

**BAYESIAN OPTIMIZATION TECHNIQUES FOR
HIGH-DIMENSIONAL AND
ADVERSARIAL SETTINGS**

HAN LIANG WEE, ERIC

NATIONAL UNIVERSITY OF SINGAPORE

2023

**BAYESIAN OPTIMIZATION TECHNIQUES FOR
HIGH-DIMENSIONAL AND
ADVERSARIAL SETTINGS**

HAN LIANG WEE, ERIC
(B.Comp. (Hons.), NUS)

**A THESIS SUBMITTED
FOR THE DEGREE OF DOCTOR OF PHILOSOPHY
DEPARTMENT OF COMPUTER SCIENCE
NATIONAL UNIVERSITY OF SINGAPORE**

2023

Thesis Advisors:

Assistant Professor Jonathan Mark Scarlett

Examiners:

Associate Professor Low Kian Hsiang
Professor Xiao Xiaokui

DECLARATION

I hereby declare that the thesis is my original work and it has been written by me in its entirety.

I have duly acknowledged all the sources of information which have been used in the thesis.

This thesis has also not been submitted for any degree in any university previously.

A handwritten signature in black ink, enclosed within a thin, hand-drawn oval. The signature is written in a cursive style and appears to read "Eric Han".

Han Liang Wee, Eric
16 August 2023

Acknowledgments

Foremost, I extend my sincere gratitude to my supervisor, Assistant Professor Jonathan Scarlett, for his invaluable mentorship and guidance throughout my PhD journey. Even though I was his first PhD student, he provided a wealth of insights and guidance, nurturing my independent thinking and influencing my research direction. I would also like to express appreciation to my colleagues and fellow Machine Learning researchers for fostering an open culture of collaboration. Their eagerness to share knowledge and open-source their code has enriched my learning experience.

I offer heartfelt thanks to the National University of Singapore, School of Computing for granting me the chance to pursue my PhD through the Graduate-Tutorship Scheme; it enabled me to impart knowledge to and inspire the next generation of Computer Scientists, while honing my research acumen. I am deeply grateful to the Agency for Science, Technology, and Research for the A*STAR Undergraduate Scholarship (AUS), which ignited the start of my PhD journey and bestowed numerous research-related opportunities. An A*STAR IHPC outreach competition first piqued my interest in Machine Learning research. I must also acknowledge the many teachers and mentors who have profoundly influenced my learning journey, providing wisdom, motivation, and guidance.

Lastly, but importantly, I extend profound thanks to my wife, family, church family (New Culture), and friends. Their unwavering love, encouragement, and support have been instrumental throughout my PhD journey. Their constant presence has been a beacon of strength and motivation, pivotal to my achievements. This milestone is not just my achievement, but a collective triumph shared with these exceptional individuals who have supported me at every turn.

Table of Contents

Summary	x
List of Tables	xii
List of Figures	xiii
List of Algorithms	xix
List of Symbols	xx
List of Abbreviations	xxi
Preface	xxiv
1 Background	1
1.1 Bayesian Optimization	2
1.1.1 Gaussian Processes	4
1.1.2 Kernels	4
1.1.3 Acquisition Functions	7
1.2 Adversarial Machine Learning	10
1.2.1 Attacks on Convolutional Neural Networks	12
1.2.2 Attacks on Bayesian Optimization	14
2 Introduction	16
2.1 High-Dimensional Bayesian Optimization	16
2.1.1 Low Effective Dimensionality	18
2.1.2 Additive Structure	18
2.1.3 Research Objective & Scope	21

2.2	Adversarial Attacks on Bayesian Optimization	22
2.2.1	Presence of outliers	23
2.2.2	Random perturbations to sampled points	24
2.2.3	Adversarial perturbations to the final point	24
2.2.4	Adversarial perturbations to samples	25
2.2.5	Research Objective & Scope	25
2.3	Black-box Adversarial Attacks on CNNs	26
2.3.1	Transfer-based methods	28
2.3.2	Gradient Estimation	28
2.3.3	Gradient-free Optimization	29
2.3.4	Research Objective & Scope	30
3	Additive GP-UCB using Tree Structures	32
3.1	Contributions	32
3.2	Setup	33
3.2.1	Prior and Posterior	34
3.3	Tree-GP-UCB Algorithm	34
3.3.1	Acquisition Function	35
3.3.2	Additive Components	38
3.4	Experiments	42
3.4.1	Experimental Setup	43
3.4.2	Metrics	45
3.4.3	Experiments with Additive GP Functions	46
3.4.4	Experiments with Non-GP Functions	51
3.4.5	BO-Based Adversarial Attack Experiments	53
3.5	Conclusion	56
4	Adversarial Attacks on Bayesian Optimization	57
4.1	Contributions	57
4.2	Setup	58
4.2.1	Knowledge Available to the Adversary	59
4.2.2	Targeted vs. Untargeted Attacks	60
4.3	Theoretical Study	60

4.3.1	Optimization Algorithms	61
4.3.2	Conditions for a Successful Attack	62
4.3.3	Applications to Specific Scenarios	65
4.4	Attack Methods	67
4.4.1	Subtraction Attack (Known f)	69
4.4.2	Clipping Attack (Known f)	69
4.4.3	Aggressive Subtraction Attack (Unknown f)	70
4.5	Experiments	71
4.5.1	Experimental Setup	72
4.5.2	Metrics	73
4.5.3	Synthetic Experiments	73
4.5.4	Robot Pushing Experiments	78
4.5.5	Defense Experiments (Without Knowledge)	78
4.5.6	Further Experiments	80
4.6	Conclusion	80
5	Black-box Adversarial Attacks on CNNs	82
5.1	Contributions	84
5.2	Setup	85
5.2.1	Adversarial Attack using Bayesian Optimization	86
5.3	Tiling Transform (ℓ_∞ Attack)	88
5.3.1	Projection on the ϵ -ball	90
5.4	Frequency-Specific Fourier Transform (ℓ_2 Attack)	90
5.4.1	Simplified-Frequency-Specific Fourier Transform	93
5.5	Tuning Hyperparameters During Attack	93
5.5.1	Bandit Hyperparameter Search	94
5.5.2	Bayesian Optimization Hyperparameter Search	95
5.6	Experiments	96
5.6.1	Experimental Setup	97
5.6.2	Metrics	98
5.6.3	Ablation Experiments for Tiling	99
5.6.4	Untargeted ℓ_∞ Attack Experiments	100

5.6.5	Untargeted ℓ_2 Attacks Experiments	101
5.6.6	Bandit Hyperparameter Search Experiments	101
5.6.7	BO Hyperparameter Search Experiments	104
5.6.8	Comparisons with Other Methods	105
5.7	Conclusion	106
6	Conclusion and Future Work	107
	Bibliography	110
A	Appendix on Additive GP-UCB using Tree Structures	127
A.1	Implementation Details	127
A.2	Experimental Details	128
A.2.1	Effect of Hyperparameters	128
A.3	Additive GP Functions Experiments	134
A.4	Non-GP Functions Experiments	138
A.4.1	Synthetic Functions	138
B	Appendix on Adversarial Attacks on Bayesian Optimization	139
B.1	Implementation Details	139
B.2	Experimental Details	139
B.3	GP-UCB Attack Experiments	142
B.3.1	Hyperparameters Discussion	142
B.3.2	Comparison of attacks	143
B.4	Kernel Experiments	144
B.5	MaxVar+Elimination Attack Experiments	144
B.6	Online-Learned Kernel Experiments	145
B.7	Dynamic Experiments	145
C	Appendix on Black-box Adversarial Attacks on CNNs	156
C.1	Implementation Details	156
C.2	Experimental Details	157
C.3	Ablation Experiments for Tiling	157
C.4	Untargeted ℓ_∞ Attack Experiments	160

C.5	Untargeted ℓ_2 Attack Experiments	163
-----	--	-----

Summary

Bayesian Optimization (BO) is an increasingly essential approach for sequentially optimizing black-box functions without a closed-form solution or those that are costly to evaluate. BO is applied to solve optimization problems in real-world applications, such as hyperparameter tuning of machine learning models, robotics, automated model selection, and more. The training of AlphaGo, where BO tuned the neural network’s hyperparameters leading to significant improvements in playing strength, is a notable example of its application (Y. Chen et al., 2018; Silver et al., 2016). However, the efficacy of BO is impacted by problem dimensionality and robustness issues. Addressing these two challenges is crucial and the subject of active research, which can enable the broader application of BO.

Firstly, we focus on the problem of scaling BO to high dimensions, studying the various approaches with differing assumptions in the literature. In particular, we examine the approach by (Rolland, Scarlett, Bogunovic, & Cevher, 2018), where the authors generalize the additive model first proposed by (Kandasamy, Schneider, & Póczos, 2015), in which low-dimensional functions represented by *arbitrary* overlapping subsets of variables are composed together to model a high-dimensional target function. By reducing model complexity and constraining the dependency graph to tree structures, we prioritize computational efficiency and ease of model learning over expressivity.

Next, we focus on the settings where BO algorithms are subject to adversarial attacks. Recent literature on BO (Gaussian Process bandits in this context) has been moving beyond random noise and devising algorithms robust to adversarial noise. Numerous methods have been proposed that defend against the proposed uncertainty model to improve robustness. However, few works study the problem

from an attacker’s perspective, i.e., which kinds of attacks would be successful against non-robust algorithms. We focus on the adversarial perturbation setting (Bogunovic, Krause, & Scarlett, 2020) and analyze the conditions for which an adversary can succeed, from which we devise several attacks, considering the various levels of information available to the adversary.

Finally, we employ BO to identify practical high-dimensional adversarial perturbations that confuse Convolutional Neural Networks (CNN). We consider the setup of (Shukla, Sahu, Willmott, & Kolter, 2021), where BO is used to find the adversarial noise in a latent space to effectively hard-label black-box attacks. We propose a tiling transformation for the ℓ_∞ setting, effectively targeting the lower-level feature detection of a neural network, leading to the propagation of incorrect signals in subsequent CNN layers. For the ℓ_2 setting, we base our method on the insights of (Yin, Lopes, Shlens, Cubuk, & Gilmer, 2019); the attack success depends on how the model was (possibly adversarially) trained. We propose a method to select the frequency in Fourier space to attack and use BO to find the adversarial noise. Our setup, which considers dimensionality techniques, demonstrates the potential of BO in executing successful adversarial attacks on CNNs.

In conclusion, our thesis advances the development of robust BO techniques tailored for high-dimensional and adversarial contexts. We also demonstrate that considering these issues, we can apply BO to performing adversarial attacks on CNN. Additionally, we delve into recent research inspired by our work and outline potential future directions in this domain.

List of Tables

1.1	Popular Kernels used in Literature.	5
1.2	Popular Acquisition Functions used in Literature.	8
4.1	Summary of our proposed adversarial attacks.	68
4.2	Synthetic functions with a summary of their $\mathcal{R}_{\text{target}}$ parameters. .	73
5.1	Attack Methods for MNIST, CIFAR-10, and VGG16-bn: Overview of settings and latent perturbation size.	105
A.1	State-of-the-art methods' parameters values.	128
A.2	Summary of both Synthetic (top) and Real (bottom) functions used in our experiments; by type, dimensionality.	133
A.3	Summary of the various structure sampling hyperparameters along with the corresponding total number of structure samples $N_{\text{iter}} \times$ N_{samples}/C	133
B.1	Summary of various experiments and functions used, sorted by type and dimensionality.	141

List of Figures

1.1	RBF Kernel, effect of changing ℓ (left) and σ^2 (right).	6
1.2	Illustration of different functions drawn from RBF posterior, learning the ground truth, $f(x) = \sin(0.9x)$; the grey region represents the confidence bounds.	7
1.3	Illustration of an instance of GP-UCB corresponding with Figure 1.2, similarly with confidence bounds (grey area) and location of next evaluation (dotted vertical line) shown.	9
2.1	Example Dependency Graph No-Overlap structure, $g(x) = g^A(x_1, x_2) + g^B(x_3, x_4) + g^C(x_5, x_6, x_7, x_9) + g^D(x_8)$	19
2.2	Example Dependency Graph Overlap structure, $g(x) = g^A(x_1, x_2) + g^B(x_1, x_3) + g^C(x_1, x_4) + g^D(x_1, x_5) + g^E(x_1, x_6) + g^F(x_1, x_7, x_8)$	20
2.3	Regression with outliers using GP (left), using Student- t (center); using Student- t to remove the outliers with GP on the remaining points; Illustration from (Martinez-Cantin et al., 2018), reproduced under Creative Commons Attribution 4.0 International License.	23
3.1	Dependency tree structure, $h(x) = h^A(x_1, x_6) + h^B(x_1, x_5) + h^C(x_1, x_4) + h^D(x_3, x_4) + h^E(x_2)$	33

3.2	Illustration of the optimization of the acquisition function on the graph $\mathcal{G}$ shown in Figure 3.1. First, the leaves of the junction tree perform partial maximization of the functions h^B , h^D , and h^A over their respective variables x_4 , x_2 and x_5 . The result of the partial maximization is a ‘conditional’ maximization over the other variables. These ‘conditional’ maximizations are passed as messages to the nodes above. In this manner, the messages are passed until the root node, and the correct answer is returned.	35
3.3	Example with two variables, grid size $N_{\text{gridsize}} = 3$, and domain $[0, 3]$. Firstly, we partition each axis evenly into 3 partitions. Next, we draw a uniformly random point from each partition. The points from each axis form a discretized domain, and we run MSG-PASSING-DISCRETE on this discrete domain. Finally, we zoom into the square, representative of the selected point. In this manner, we recursively sub-divide the grid for all $N_{\text{lvl}} = 3$ levels.	38
3.4	Synthetic Dependency Graphs Structures.	47
3.5	Star-25 Discrete and Continuous Experiments.	47
3.6	Summarized comparison of various additive methods across Grid- 3×3 and Ancestry-132 functions.	48
3.7	Scalability of Tree over dimensions.	48
3.8	Tree’s learning progress over several iterations; Tree closely approximates the underlying Grid- 3×3 dependency structure (See Figure 3.4c). Note that some of the edges are overlapping (disconnected) with vertices in 3.8a, 3.8c.	49
3.9	Tree’s learning progress over several iterations; Tree successfully recovers the underlying Star-25 dependency structure (See Figure 3.4a).	50
3.10	Results for non-GP synthetic functions.	51
3.11	Results for Lpsolve hyperparameter tuning.	52
3.12	Results for NAS-Bench-101 datasets.	54
3.13	Results for BO-based adversarial attacks.	56

4.1	Illustration of the various function constructions – bump function (left) and convolved bump function (right).	65
4.2	Difficult case where f is increasing in $\mathcal{R}_{\text{target}}$	67
4.3	Subtraction Attack for known f with different $h(\mathbf{x})$	68
4.4	Clipping attack for known f	69
4.5	Aggressive Subtraction attack for unknown f	70
4.6	Illustrations for 1D and 2D experiments; $\mathcal{R}_{\text{target}}$ is shaded.	74
4.7	Effects of hyperparameters on the different attacks.	75
4.8	Scatter plots of runs averaged over random seeds for various functions.	76
4.9	Plots of each attack with the most efficient (best success rate over normalized cost) hyperparameter.	77
4.10	Variants of GP-UCB which add a constant C to the confidence width in order to defend against the attack.	79
5.1	Image perturbation (32×32 -pixels) resulting from using NNI for the ℓ_∞ attack; Hyperparameter $\lfloor rd \rfloor$ determines the latent perturbation size.	83
5.2	Discrete Fourier transform of image data (Rao, Kim, & Hwang, 2010), showing the conjugate symmetry; Real values - white, conjugate pairs - light and dark grey. We refer to top left quadrant (0 to 24) as Q1, top right quadrant (1 to 23) as Q2, bottom left (5 to 24) as Q3 and bottom right as Q4.	83
5.3	Latent perturbation and its corresponding Image perturbation resulting from applying LF for the ℓ_2 attack when hyperparameter $\lfloor rd \rfloor = 4$	84
5.4	Example image perturbation produced by Tiling with various hyperparameters $\lfloor rd \rfloor$, corresponding to the level of feature map in CNN to attack.	89
5.5	A multi-story apartment building with a rainbow arching over it; Applying the low/high pass filter to the image, with its respective high/low-frequency components removed.	91

5.6	Example latent perturbation and its corresponding image perturbation produced by FSFT with $f_0 = (2, 2)$ and various $\lfloor rd \rfloor$	92
5.7	Heat map shows SFSFT's success rate across frequencies $f_0 = (f_x, f_y)$, with lighter (yellow) shades representing higher success rates and darker (purple) shades indicating lower rates.	96
5.8	Untargeted ℓ_∞ BO-Adv Attack on CIFAR-10, comparing NNI vs Tiling and with vs without ϵ -ball Projection.	99
5.9	Untargeted ℓ_∞ BO-Adv Attack on MNIST.	100
5.10	Intermediate feature maps across CNN layers for digit '9' in MNIST; from left (input layers with low-level features) to right (higher-level features), ending rightmost with outputs for C classes. . . .	101
5.11	Untargeted ℓ_2 BO-Adv Attack on ImageNet CNNs.	102
5.12	Bandit hyperparameter search for the best arm (<i>i.e.</i> , $\lfloor rd \rfloor$) with the best reward for untargeted ℓ_∞ Rng-Adv Attacks using Tiling. . . .	103
5.13	BO hyperparameter search for the best frequency, f_0 , with the best performance for untargeted ℓ_∞ Rng-Adv Attacks using SFSFT. . . .	104
5.14	Comparison of various methods on various query budgets.	106
A.1	Tree's learned dependency structure at Iteration 33 of a run. . . .	129
A.2	Tree's learned dependency structure at Iteration 50 of a run. . . .	130
A.3	Comparing Exploitation, Balanced and Exploration scenarios for Star-25 (Continuous).	130
A.4	Comparison of N_{samples} values [50, 100, 200] within C [4, 8, 16] for the Star-25 (Continuous) function. Each combination is denoted as 'N' and 'C', representing the specific pairing of N_{samples} and C parameters respectively.	131
A.5	Comparison of N_{gridsize} and N_{lvl} values for the Star-25 (Continuous) function. Each combination is denoted as 'GS' and 'L', representing the specific pairing of N_{gridsize} and N_{lvl} parameters respectively.	132
A.6	Ancestry-132 Dependency Graph Structure.	135
A.7	Results for synthetic additive functions on discrete domains. . . .	136

A.8	Results of synthetic additive functions on continuous domains. .	137
A.9	Results for the Ancestry-132 graph.	137
B.1	Clipping attack on Synthetic1D; see Equation 4.18 for the definition of ‘most efficient’.	143
B.2	Effect of Aggressive Subtraction’s hyperparameter Δ on the success rate and cost incurred.	147
B.3	Effect of Clipping’s hyperparameter Δ on the success rate and cost incurred.	148
B.4	Effect of Subtraction Rnd’s hyperparameter $h_{\max}$ on the success rate and cost incurred.	149
B.5	Effect of Subtraction Sq’s hyperparameter $h_{\max}$ on the success rate and cost incurred.	150
B.6	Success rate vs. cost averaged over random seeds, each point is the performance of a particular hyperparameter.	151
B.7	Success rate vs. cost with every random seed shown individually, i.e., each point is a single run.	152
B.8	Experiments comparing different kernels: Radial Basis Function (RBF), Matérn $5/2$, and Matérn $3/2$	153
B.9	We attack the MaxVar + Elimination algorithm in 3 experiments.	153
B.10	Experiments comparing the kernel parameters being learned on-line (second and forth rows) vs. learned from data sampled from f prior to the optimization (first and third rows). The top two rows average over random seeds, and the bottom two rows show every individual run.	154
B.11	Experiments applying a simple dynamic hyperparameter strategy for the Aggressive Subtraction attack. The top row averages over random seeds, and the bottom row shows every individual run. .	155
C.1	Untargeted ℓ_{∞} Rng-Adv Attack on MNIST, comparing NNI vs Tiling and with vs without ϵ -ball Projection.	158
C.2	Untargeted ℓ_{∞} Rng-Adv Attack on CIFAR-10, comparing NNI vs Tiling and with vs without ϵ -ball Projection.	158

C.3	Untargeted ℓ_∞ BO-Adv Attack on MNIST, comparing NNI vs Tiling and with vs without ϵ -ball Projection.	159
C.4	Untargeted ℓ_∞ BO-Adv Attack on CIFAR-10, comparing NNI vs Tiling and with vs without ϵ -ball Projection.	159
C.5	Untargeted ℓ_∞ Rng-Adv Attack, comparing NNI and Tiling. . . .	161
C.6	Untargeted ℓ_∞ BO-Adv Attack, comparing NNI and Tiling. . . .	162
C.7	Untargeted ℓ_2 Rng-Adv Attack on ImageNet CNN, comparing LF and FSFT.	163
C.8	Untargeted ℓ_2 BO-Adv Attack on ImageNet CNN, comparing LF and FSFT.	164

List of Algorithms

1.1	BAYESIAN-OPTIMIZATION	3
3.1	TREE-GP-UCB	35
3.2	MSG-PASSING-DISCRETE	36
3.3	MSG-PASSING-CONTINUOUS	37
3.4	TREE-LEARNING	40
3.5	GIBBS-SAMPLING at k -th iteration	41
3.6	MUTATION at k -th iteration	42
5.1	BO-ATTACK($\mathbf{x}, y$)	87
5.2	TILING-TRANSFORM(δ'_t)	89
5.3	FSFT(δ'_t)	93
5.4	BANDIT($\Theta, \mathbf{X}$)	95

List of Symbols

$\mathbb{R}$	Set of real numbers
$\mathbb{N}$	Set of natural numbers
$\mathbb{E}$	Expectation
C	Number of cycles
Bernoulli	Bernoulli distribution
$\mathcal{X}$	Domain
$\mathcal{G}$	Decomposition of functions
ϕ	Values in feature space
f	Target/Objective function $f : \mathcal{X} \rightarrow \mathbb{R}$
N_{dim}	Number of dimensions in $\mathcal{X}$
N_{samples}	Number of samples required
a_{UCB}	Upper Confidence Bound acquisition function
a_{EI}	Expected Improvement acquisition function
cov	Covariance function
$\mathbf{X}$	Input to the function f
$\mathbf{y}$	Output of the function f
$\mathcal{D}$	Observations/Samples, comprising of $\mathbf{X}, \mathbf{y}$
x^*	Location on f where it achieves global optimal value
f^*	Global Optima of f
x_{min}	Location on f where it achieves global minimum value
x_{max}	Location on f where it achieves global maximum value
$\mathcal{GP}$	Gaussian Process modelling f with mean μ and covariance K

List of Abbreviations

AI	Artificial Intelligence
API	Application Programming Interface
ARD	Automatic Relevance Determination
BFGS	Broyden-Fletcher-Goldfarb-Shanno
BO	Bayesian Optimization
CNN	Convolutional Neural Networks
CPU	Central Processing Unit
EI	Expected Improvement
FFT	Fast Fourier Transform
FMAB	Federated Multi-armed Bandits
GP	Gaussian Process
GP-UCB	Gaussian Process with Upper Confidence Bound
HDBO	High-Dimensional Bayesian Optimization
L-BFGS	Limited-Memory Broyden-Fletcher-Goldfarb-Shanno
LCB	Lower Confidence Bound
LF	Low-frequency Fourier Transform
MAB	Multi-armed Bandits
ML	Machine Learning

MLaaS	Machine-Learning-as-a-Service
MIP	Mixed Integer Linear Programming
NNI	Nearest Neighbour Interpolation
NN	Neural Network
RGB	Red, Green, and Blue
RBF	Radial Basis Function
REMBO	Random EMbedding Bayesian Optimization
RKHS	Reproducing Kernel Hilbert space
TNC	Truncated Newton algorithm
UCB	Upper Confidence Bound
UF	Union-Find

*“For since the fabric of the universe is most perfect and the work of a most wise Creator, nothing at all takes place in the universe in which some rule of the **maximum** or **minimum** does not appear.”*

— Leonhard Euler

Preface

Chapter 1 sets the stage by providing a detailed overview of Bayesian Optimization and Adversarial Machine Learning, equipping the reader with the necessary background for understanding the subsequent material. In Chapter 2, we delve into the core problem and conduct a thorough literature review of related work. This chapter outlines the key research questions driving this thesis and discusses our aim to enhance the performance of Bayesian Optimization in high-dimensional settings and bolster its robustness, resulting in broader application across various fields, and we present three research questions that align with this overarching goal. Chapters 3 and 4 build upon our previously published works, which are detailed below. Chapter 5 presents original, unpublished work. The thesis culminates in Chapter 6, where we summarize our findings and propose avenues for future research in this exciting and evolving domain.

List of Publications

1. **Han, E.** & Scarlett, J. (2022). Adversarial Attacks on Gaussian Process Bandits. *Proceedings of the 39th International Conference on Machine Learning*, 162:8304-8329. Available from <https://proceedings.mlr.press/v162/han22f.html>.
2. **Han, E.**, Arora, I., & Scarlett, J. (2021). High-Dimensional Bayesian Optimization via Tree-Structured Additive Models. *Proceedings of the AAAI Conference on Artificial Intelligence*, 35(9), 7630-7638. Available from <https://doi.org/10.1609/aaai.v35i9.16933>.

Chapter 1

Background

Optimization is inherent to human instinct and biology; we naturally desire and seek optimization (Martins & Ning, 2021). Mathematically, the problem of global optimization involves finding the global optimum (or solution) of a function. Owing to the need for continuous improvement, global optimization enjoys wide appeal and application (Martins & Ning, 2021), examples include:

1. **Finance:** Financial planning, and asset allocation (Xiang, Gubian, Suomela, & Hoeng, 2013).
2. **Biology:** Protein structure prediction, and drug discovery (Lee, Basith, & Choi, 2018).
3. **Machine learning:** Hyperparameter tuning, and model selection (Y. Chen et al., 2018).

Global optimization is an important field that continues to be challenging to solve in practice; Below, we describe some key challenges. In many practical applications, the objective function and constraints are often *black-box*. This implies that the functions are not known exactly and the optimizer has no knowledge of how the values are obtained. Another practical challenge is that the evaluations can be *expensive*, as they require extensive computing resources (*e.g.*, simulations) or take a considerable amount of time (*e.g.*, conducting experiments (Sergeyev & Kvasov, 2017)). In addition, many real-world optimization problems involve *high dimensionality*, *i.e.*, large number of variables or dimensions. As the number of dimensions increases, the search space grows exponentially,

making it difficult to explore efficiently. In addition, there is often a need for *robustness* as the objective function may be noisy or subject to uncertainties. For example, there might be limitations on obtaining precise information due to noise in measurements or incomplete data.

Our discussion focuses on Bayesian Optimization to address global optimization challenges and enhance its efficacy (Shahriari, Swersky, Wang, Adams, & Freitas, 2016). Bayesian Optimization is an approach for optimizing *black-box* and/or *expensive* functions, despite limited evaluations. We aim to advance its performance in *high-dimensional* settings and improve its *robustness*, contributing to its broader adoption in various domains; We will elaborate further on this in the subsequent chapter.

Formally, we are interested in a particular formulation where we want to find solution $x_{\max}$ achieving the globally maxima f^* . Specifically, we have a function $f: \mathcal{X} \rightarrow \mathbb{R}$ that we want to maximize on domain $\mathcal{X} \subseteq \mathbb{R}^{N_{\text{dim}}}$:

$$x^* = x_{\max} \in \arg \max_{x \in \mathcal{X}} f(x); \quad f^* = f(x^*) = \max_{x \in \mathcal{X}} f(x). \quad (1.1)$$

In the rest of our discussion, we will consider only the problem of maximization, without loss of generality, as minimization can be easily be converted to maximization and vice versa. We consider finding x^* iteratively, by evaluating f at different points until a stopping criterion is met.

1.1 Bayesian Optimization

Bayesian Optimization (BO) (Snoek, Larochelle, & Adams, 2012) is a popular and important technique for sequential global optimization of black-box functions in a query efficient manner. Kushner was one of the earliest to formulate this technique (Harold J. Kushner, 1962; H. J. Kushner, 1964) using a Gaussian Process (GP) model as a surrogate model of the objective function. BO is especially useful in situations where f is *black-box* and/or is *expensive* to evaluate, where *convex optimization* cannot be readily used. For instance, AlphaGo’s hyperparameters were tuned using BO, resulting in substantial improvements in playing strength. It resulted in an improvement of at least 50% to 66.5% in

Algorithm 1.1: BAYESIAN-OPTIMIZATION

input : Target function f
Domain $\mathcal{X}$
Number of initial points N_{init}
output: Optimal (x^*, f^*)

- 1 Initialize X with N_{init} points drawn randomly from $\mathcal{X}$
- 2 $\mathbf{y} \leftarrow f(X)$
- 3 $\mathcal{D} \leftarrow (X, \mathbf{y})$
- 4 Initialize model m using $\mathcal{D}$
- 5 Initialize acquisition function a using model m
- 6 **while** *stopping condition not met* **do**
- 7 $x_{\text{new}} \leftarrow \arg \max_{x \in \mathcal{X}} a(x)$
- 8 $y_{\text{new}} \leftarrow f(x_{\text{new}})$
- 9 Augment $\mathcal{D} \leftarrow \mathcal{D} \cup (x_{\text{new}}, y_{\text{new}})$
- 10 Update m and its parameters with updated $\mathcal{D}$
- 11 $(x^*, f^*) \leftarrow \arg \max_{(x, y) \in \mathcal{D}} y$
- 12 **return** (x^*, f^*)

self-play games (Y. Chen et al., 2018).

Our goal is to find the global optima, we need to have an accurate mapping from input variables to output variables based on samples $\mathcal{D} = \{(x_i, y_i) : x_i \in \mathcal{X}\}_{i=1}^N$. We take a Bayesian approach to modelling the x, y relationship

$$y_i = f(x_i) + \epsilon_i, \quad \epsilon_i \sim \mathcal{N}(0, \sigma^2). \quad (1.2)$$

In this setting, we only have access to noisy evaluations y_i , with ϵ_i representing additive Gaussian noise. BO finds x^* with *least* possible observations by making these assumptions:

- **Surrogate** of the objective function f , where GPs are typically used.
- **Acquisition function** that can be inexpensively optimized to select the next best x for evaluating f to find x^* .

The key idea is that BO translates the original problem to another optimization problem on a acquisition function. We note that different stopping conditions can be applied. A general description of the typical BO steps are given in Algorithm 1.1.

In many real-world applications such as vehicle design optimization (Eriksson, 2022), there is a need to apply BO to optimize for black-box outcomes

(*i.e.*, structural strength). However, such optimization may consider over 200 variables (*i.e.*, structural parts) which is prohibitively large for standard BO methods. Scaling BO to higher dimensions is challenging due to the curse of dimensionality¹, leading to sparsity of data points and increased computational costs. Efficient and effective algorithms are required to address these issues and is still an active area of research (Hoang, Hoang, Ouyang, & Low, 2018; C.-L. Li, Kandasamy, Póczos, & Schneider, 2016; Rolland et al., 2018); we will review literature in this area and discuss more in Section 2.1.

1.1.1 Gaussian Processes

BO depends on a suitable surrogate model to reason, and it heavily affects the optimization performance. Gaussian Process (GP) (Rasmussen & Williams, 2006) is a class of probabilistic models that is usually used as the surrogate model for BO. $\mathcal{GP}$ models target function f using a mean μ and covariance function K . GPs are useful for BO as we can use GP to reason about the function f .

1.1.2 Kernels

The covariance function, or kernel, is a key ingredient in a GP predictor that encode our prior beliefs about the function f 's properties. Kernels specifies the statistical relationship between inputs pairs, $x \in \mathcal{X}$ and $x' \in \mathcal{X}$, and maps into $\mathbb{R}$. The kernel function $K(x, x')$ can be chosen or learnt and is application-dependent, having the following form

$$K(x, x') = \text{cov}[\phi, \phi' | x, x'], \quad (1.3)$$

where ϕ, ϕ' are values in feature space (see Section 1.1.2 for an example). Loosely speaking, the kernel can be interpreted as a measure of similarity between x and x' , where it exhibits the property where the kernel will return a larger value when close and smaller value when far. In general, there are two types of kernels (Genton, 2001): stationary kernels, which are translation invariant, and non-

¹The curse of dimensionality refers to the phenomenon where the increase in the number of dimensions in a dataset exponentially leads to the sparsity of data points, making it challenging to explore and analyze effectively (Russell & Norvig, 2020).

stationary kernels, which depend explicitly on the points given. Some of the commonly used, popular kernels (Duvenaud, 2014; GPy, since 2012) are listed below.

Table 1.1. Popular Kernels used in Literature.

Kernel	$K(x, x')$
Radial Basis Function	$K_{\text{RBF}} : \sigma^2 \exp\left(-\frac{(x-x')^2}{2\ell^2}\right)$
Matérn- ν	$K_{\text{Matérn-}\nu} : \sigma^2 \times \frac{2^{1-\nu}}{\Gamma(\nu)} \left(\frac{\sqrt{2\nu} x-x' }{\ell}\right)^\nu J_\nu\left(\frac{\sqrt{2\nu} x-x' }{\ell}\right)$
Linear	$K_{\text{Linear}} : \sigma_b^2 + \sigma_v^2 (x - c)(x' - c)$
Periodic	$K_{\text{Period}} : \sigma^2 \exp\left(-\frac{2\sin^2(\pi x-x' /p)}{\ell^2}\right)$
Rational Quadratic	$K_{\text{RQ}} : \sigma^2 \left(1 + \frac{(x-x')^2}{2\alpha\ell^2}\right)^{-\alpha}$

Matern

The Matérn class of functions is characterized by the following complicated form,

$$K_{\text{Matérn-}\nu}(x, x') = \sigma^2 \times \frac{2^{1-\nu}}{\Gamma(\nu)} \left(\frac{\sqrt{2\nu}|x-x'|}{\ell}\right)^\nu J_\nu\left(\frac{\sqrt{2\nu}|x-x'|}{\ell}\right), \quad (1.4)$$

where Γ and J_ν are the Gamma and Bessel functions. The smoothness parameter ν plays a pivotal role in determining the degree of smoothness exhibited by the functions, with lower values yielding rougher functions and higher values resulting in smoother ones. As $\nu \rightarrow \infty$, we essentially recover the RBF kernel (see Equation 1.5). The Matérn kernel assumes a distinct structure when ν follows the pattern $\nu = d + \frac{1}{2}$, where $d \in \mathbb{N}$. This structure involves the multiplication of an exponential function and a polynomial function of order d . Notably, three widely-used selections for ν are $\nu \in \frac{1}{2}, \frac{3}{2}, \frac{5}{2}$, with their respective forms provided below:

- $K_{\text{Matérn-}1/2}(x, x') = \sigma^2 \exp\left(-\frac{|x-x'|}{\ell}\right),$
- $K_{\text{Matérn-}3/2}(x, x') = \sigma^2 \left(1 + \frac{\sqrt{3}|x-x'|}{\ell}\right) \exp\left(-\frac{\sqrt{3}|x-x'|}{\ell}\right),$
- $K_{\text{Matérn-}5/2}(x, x') = \sigma^2 \left(1 + \frac{\sqrt{5}|x-x'|}{\ell} + \frac{5|x-x'|^2}{3\ell^2}\right) \exp\left(-\frac{\sqrt{5}|x-x'|}{\ell}\right)$

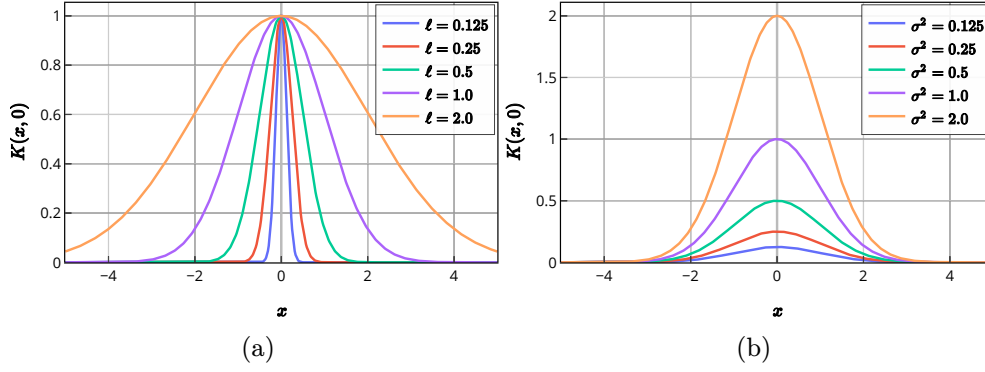


Figure 1.1. RBF Kernel, effect of changing ℓ (left) and σ^2 (right).

Radial Basis Functions

Radial Basis Functions (RBF) Kernel, also known as the Squared Exponential Kernel, is one of the most widely used², stationary kernels that satisfies a universality property (Micchelli, Xu, & Zhang, 2006), meaning it is able to approximate an arbitrary continuous function. The RBF kernel have the form:

$$K_{\text{RBF}}(x, x') = \sigma^2 \exp \left(-\frac{(x - x')^2}{2\ell^2} \right). \quad (1.5)$$

The kernel's parameters and their descriptions are as follows, see Figure 1.1 for an illustration accompanying our description:

- **Lengthscale ℓ** , determines the length of the ‘variations’ in the function. A larger lengthscale causes the covariances to be higher as the points become further apart.
- **Output Variance σ^2** , determines the average distance of the function from the mean. Increasing variance causes the covariance to increase uniformly across the domain. We sample some functions drawn from the posterior distribution of a RBF kernel after a few observations, illustrated in Figure 1.2.

Infinite-dimensional mapping It is well known that the RBF kernel's feature space has an infinite number of dimensions. To simplify our discussion here,

²Many software packages such as GPy (GPy, since 2012) and Scikit Learn (Buitinck et al., 2013) set RBF as the default kernel.

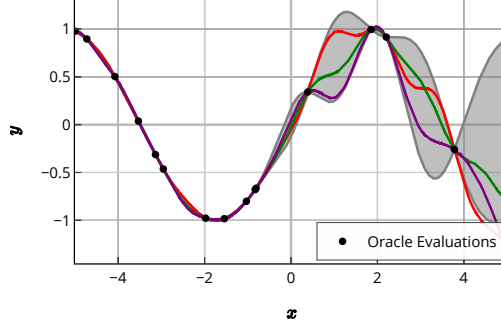


Figure 1.2. Illustration of different functions drawn from RBF posterior, learning the ground truth, $f(x) = \sin(0.9x)$; the grey region represents the confidence bounds.

we reduce the kernel to $K(x, x') = \exp(-(x - x')^2)$. Next, we simply expand the kernel using Taylor Series of e^x , at $x = 0$.

$$\begin{aligned}
 \exp(-(x - x')^2) &= \exp(-x^2) \times \exp(-x'^2) \times \exp(2xx') \\
 &= \exp(-x^2) \exp(-x'^2) \left[1 + 2xx' + \frac{2^2 x'^2 x^2}{2!} + \dots \right] \\
 &= \exp(-x^2) \exp(-x'^2) \sum_{k=0}^{\infty} \frac{2^k x'^k x^k}{k!} \\
 &= \sum_{k=0}^{\infty} \left[\sqrt{\frac{2^k}{k!}} \exp(-x^2) x^k \times \sqrt{\frac{2^k}{k!}} \exp(-x'^2) x'^k \right]
 \end{aligned} \tag{1.6}$$

We observe that it is taking a dot product (distance measure) over $k \rightarrow \infty$ dimensions. In other words, the RBF kernel is formed from a feature expansion of infinite polynomials. This explains the power behind the RBF kernels. RBF kernels implicitly map the current vector into an infinite-dimensional space and compute the distance between each vector pairing. We also note that even though this is an infinite-dimensional space, each variable is highly constrained, so the solution space is not completely unbounded. Additionally, as a consequence, GPs with this kernel have derivatives of all orders, causing the functions to be very smooth. In our discussion, we often use this kernel in our formulation and experiments.

1.1.3 Acquisition Functions

Acquisition functions are cheap to evaluate/optimize heuristic functions for which BO optimizes to find the next x to evaluate. Typical optimizers used to optimize

the acquisition functions include the well-known Broyden-Fletcher-Goldfarb-Shanno (BFGS) algorithm (Broyden, 1970; Fletcher, 1970; Goldfarb, 1970; Shanno, 1970), a quasi-Newton optimization technique. A limited-memory memory version (D. C. Liu & Nocedal, 1989), commonly referred to as L-BFGS is also widely used in the BO literature (J. Kim & Choi, 2020). Depending on the acquisition function chosen and its behavior, other optimizers may be more suitable, such as Adam (Kingma & Ba, 2015) and Truncated Newton algorithm (TNC) (Nash, 2000).

Effectively, the choice of the acquisition function and its parameters allows us to fine-tune the balance between *exploration and exploitation*. This trade-off between exploration and exploitation is a fundamental problem in decision making, where exploration is seeking new information or options while exploitation is using known solutions to maximize gains.

Different acquisition functions consider different trade-offs and metrics, but generally, we want to select the next best point to explore/exploit. Typical choices of acquisition functions are Upper Confidence Bound and Expected Improvement, they are presented in Table 1.2. Different acquisition functions will make different decisions based on each of their heuristics. We focus our discussion on Upper Confidence Bound as has strong theoretical results, commonly used and is explicit in the exploration-exploitation trade-off.

Table 1.2. Popular Acquisition Functions used in Literature.

Acquisition Function	$a(x)$
Upper Confidence Bound	$a_{\text{UCB}} = \mu(x) + \beta_t \sigma(x)$
Expected Improvement	$a_{\text{EI}} = \mathbb{E}[\max(0, f(x) - f') x, \mathcal{D}]$

Upper Confidence Bound

Upper Confidence Bound (UCB) (Srinivas, Krause, Kakade, & Seeger, 2010) is a popular acquisition function that allows us to balance between exploration and exploitation; it is also commonly referred to as GP-UCB. We also note that it can be referred to equivalently as Lower Confidence Bound (LCB) for minimization problems. UCB can be understood as the upper bound of the

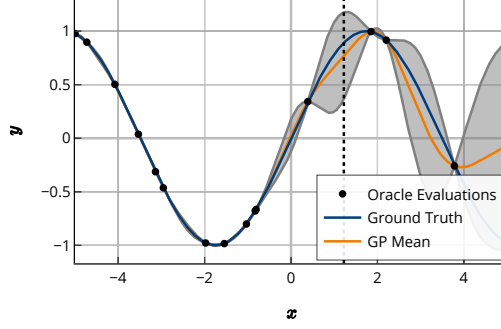


Figure 1.3. Illustration of an instance of GP-UCB corresponding with Figure 1.2, similarly with confidence bounds (grey area) and location of next evaluation (dotted vertical line) shown.

function f , with the following form

$$a_{\text{UCB}}(x) = \mu_{t-1}(x) + \beta_t \sigma_{t-1}(x), \quad (1.7)$$

where β_t is a time-dependent exploration parameter that changes over the course of optimization, trading off between exploration and exploitation components:

- **Expected reward** (mean), $\mu_{t-1}(x)$, can be interpreted as a exploitation term.
- **Uncertain reward** (standard deviation), $\sigma_{t-1}(x)$, can be interpreted as a exploration term.

The intuition behind the Upper Confidence Bound (UCB) algorithm stems from a strategy of optimism under uncertainty, which involves a balance regulated by the parameter β_t . This concept is visualized in Figure 1.3, where the focus is on points with high means (depicted by the orange line), while acknowledging that the region of high variance (represented by the grey area) might yield better outcomes. In our instance (Figure 1.3), UCB selects the subsequent observation (indicated by the dotted vertical line) from within the high variance region, prioritizing the exploration of the high variance region around the high mean points (exploitation). Our goal is to learn enough to uncover the optima, with the β_t term orchestrating the equilibrium between the exploration and exploitation aspects of the process.

Expected Improvement

Expected Improvement (EI) (Murphy, 2012) acquisition function is another widely used acquisition function. EI is formed by computing the expected marginal gain in utility, or the instantaneous improvement over the current best objective value f' by proposing x to be the next observation:

$$a_{\text{EI}}(x) = \mathbb{E}[\max(0, f(x) - f') | x, \mathcal{D}] \quad (1.8)$$

We note that if f' is greater, there is no improvement to the current best value. Hence, we use this function for every x and pick the one that gives the largest expected improvement.

1.2 Adversarial Machine Learning

The notion of adversarial attacks on ML models appears to have been started by the extensive literature on adversarial attacks on Neural Networks (NN) (Szegedy et al., 2014); some of the works in this topic are as follows:

1. **Facial Recognition:** Facial recognition systems are vulnerable to ‘fake’ images that is designed to fool the system (Sharif, Bhagavatula, Bauer, & Reiter, 2016).
2. **Malware Detection:** Malware detection systems are vulnerable to specially created malware (Grosse, Papernot, Manoharan, Backes, & McDaniel, 2017).
3. **Intrusion Detection:** Intrusion detection systems face challenges from adversaries generating network traffic that is designed to avoid detection (Jmila & Khedher, 2022).

The vulnerability of AI models, especially NN models, to adversarial attacks has raised concerns amongst practitioners. As a result, there has been a progression of adversarial machine learning as a key field of research that investigates and mitigates the susceptibilities of AI models to attacks. Unlike traditional algorithms that were created for non-adversarial settings, the adversarial learning setting operates under the assumption that an adversary is present. Traditional

settings typically only consider random noise, which has shown to be inadequate in security-critical applications (Carlini & Wagner, 2017). Earliest literature on adversarial attacks are on CNN (Szegedy et al., 2014), where the attacks target an image classifier $f : \mathbb{R}^m \rightarrow \{1 \dots k\}$, with varying objectives:

- **Targeted** attacks refer to the scenario where the adversary has a specific goal in mind, such as causing misclassification of a particular input into a pre-determined class (Carlini & Wagner, 2017).
- **Untargeted** attacks aims to instead mislead the prediction to any labels, but the true label, causing misclassification without a specific class in mind (Carlini & Wagner, 2017).

Both targeted and untargeted attacks can be performed under different settings for which we will elaborate below, such as type of attack and with various levels of information. Untargeted attacks are more challenging to defend against since the adversary does not have a specific goal, and any misclassification would be considered a success. Untargeted attacks are also often easier to execute compared to targeted attacks because the adversary does not need to have specific knowledge of the target. However, targeted attacks can be effective and damaging as the adversary can drive the classification towards a specific (destructive) goal.

Next, we discuss two example various types of attack; poisoning attacks and evasion attacks are the most common:

- **Poisoning** attacks are a type of adversarial attack where an adversary intentionally introduces malicious data into the dataset used to train a machine learning model, such that the model produces incorrect predictions on future data (Biggio, Nelson, & Laskov, 2012).
- **Evasion** attack setting is where the adversary attacks a trained model during inference (*i.e.*, input perturbation attack); The adversary manipulates the input (otherwise correctly classified) with some perturbation causing misclassification (Biggio et al., 2013).

Poisoning attacks corrupt the training data, while evasion attacks target inference by manipulating the input data at test time; these types of adversarial

attacks are different and can be applied to different scenarios. Lastly, an additional crucial factor to consider is the extent of information available to the adversary, which directly correlates with the complexity and difficulty of the attack:

- **White-box** attacks assume full access to the model along with complete knowledge of the target model’s architecture, parameters, and internal workings, etc; the adversary would be able to leverage any information to cause misclassification (Carlini & Wagner, 2017).
- **Grey-box** attacks are a type of attack on machine learning models where the adversary has partial knowledge of the model’s parameters or architecture, but not complete access to all details (Athalye, Carlini, & Wagner, 2018).
- **Black-box** attacks refer to attacks where the adversary has no access to the model’s architecture or parameters; The adversary only has access to the queries (Papernot et al., 2017).

White-box attacks assume the adversary possesses complete knowledge of the system, which can be overly idealistic and not always practical. Conversely, black-box attacks, are often more realistic, and have the weakest assumption as adversaries have limited information. Black-box attacks, devoid of any model-related information, are frequently categorized based on the level of feedback accessible from the model, encompassing scenarios such as soft-label and hard-label settings; we will discuss these scenarios in greater detail in Section 2.3. Grey-box attacks are between these extremes, and are often more challenging than white-box attacks, as adversaries must deduce missing information.

1.2.1 Attacks on Convolutional Neural Networks

Convolutional Neural Networks (CNN) are vulnerable to carefully crafted perturbations that can lead to unexpected and potentially unsafe behaviors of the system that they control (Goodfellow, Shlens, & Szegedy, 2015; Kurakin, Goodfellow, & Bengio, 2017; Ren, Zheng, Qin, & Liu, 2020). As an alarming example, it has been demonstrated that an adversary can add a patch (See (Sharma, Bian,

Munz, & Narayan, 2022) for illustration of patch attack) on a Stop sign, confusing a CNN classifier to misclassify the sign to be ‘Speed Limit 45’; Consequently an autonomous system that rely on the faulty classification would make an unsafe decision (Eykholt et al., 2018).

Given the widespread use of CNN models in real-world applications, ensuring their trustworthiness and robustness is of paramount importance. The deployment of these models in practical settings can give rise to various issues, such as the lack of security in the model’s predictions. Adversarial attacks have become practically applied to NNs:

- Machine learning systems are vulnerable to adversarial examples even in physical world scenarios. Adversarial images from a cell-phone camera were fed to an ImageNet Inception classifier and the classification accuracy was measured. When viewed through a camera, a significant proportion of adversarial examples were incorrectly classified (Kurakin et al., 2017).
- (Papernot et al., 2017) introduces the first practical demonstration of a black-box attack on a remotely hosted CNN. The authors showed general applicability of their strategy to many ML techniques by conducting the attacks against models hosted by Amazon and Google.
- (Athalye, Engstrom, Ilyas, & Kwok, 2018) present the existence of robust 3D adversarial objects and propose an algorithm for synthesizing adversarial examples over a chosen distribution of transformations. They demonstrate the existence of 3D adversarial objects in the physical world.

In order to perform an attack, the adversary needs to formulate an adversarial perturbation in the image space. However, the dimensionality of the image space of commonly used datasets are high - $32 \times 32 \times 3 = 3072$ for CIFAR-10 (Krizhevsky, 2009) and $256 \times 256 \times 3 = 196608$ for ILSVRC2012 (Russakovsky et al., 2015). Moreover, images captured by modern cameras possess dimensions that are at least a magnitude larger when compared to those used in literature - $8256 \times 6192 \times 3 = 153363456$ for Sony A1 (Sony Corporation, 2021). The adversary would need to formulate a high-dimensionality adversarial perturbation, and potentially even higher in the near future. Existing methods use dimensionality

reduction techniques (such as principal component analysis) to successfully perform the attack (Mondal et al., 2020; Simon-Gabriel, Ollivier, Schölkopf, Bottou, & Lopez-Paz, 2018). Hence, considerations to overcome higher dimensionality continue to be important.

1.2.2 Attacks on Bayesian Optimization

We now return to the BO problem. In certain applications, corruptions in the measurements may be insufficiently captured by random noise (*i.e.*, ϵ_i in Equation 1.2) alone. Such corruptions can be caused by (but not limited to):

- **Outliers** - Extreme values in a dataset that do not fit the expected pattern of the rest of the data, that can arise from measurement errors, data entry errors, or real but rare events (Duraj & Szczepaniak, 2021).
- **Nonlinear effects** - Measurements may be affected by nonlinear effects that cannot be fully explained by random noise alone. For example, a sensor may respond differently to small and large inputs (Ayadi, Ghorbel, Obeid, & Abid, 2017; Sieberer, McWilliam, & Popov, 2019).
- **Others** - Potential sources of corruption involve instances such as node failures or intentional malicious actions within the context of a distributed optimization setting (Sundaram & Ghahesifard, 2019).

Robustness considerations are important because they help ensure that the models and systems we create perform reliably and consistently, even in the face of unexpected or adversarial situations. In many real-world applications, such as autonomous vehicles, medical diagnosis, and financial trading, even small errors or failures can have serious consequences, so it is important to design models and systems that are as robust as possible. The performance of machine learning models can be adversely affected by outliers; For example, (Nyitrai & Virág, 2019) examines the impact of handling outliers on the performance of bankruptcy prediction models. The results show that outlier handling improves the performance of the bankruptcy prediction models, indicating that significance of robustness in sensitive real-world applications.

In the context of dealing with potential sources of uncertainties, a prudent

strategy is to adopt a cautious approach and prepare for the ‘worst-case’ scenario, which involves addressing adversarial noise. As an illustration, the work presented in (J. Tu et al., 2021) focuses on introducing an adversary within a multi-sensor perception system, in contrast to addressing multiple sources of uncertainties across diverse input modalities. Hence, we consider addressing these issues via the consideration of an adversary in the BO problem; we assume that function observations are not only subject to random noise, but also adversarial noise. With this additional requirement, the optimization not only becomes more robust to uncertainty, but can also maintain robustness in the presence of malicious adversaries. We will discuss more in Section 2.2.

Chapter 2

Introduction

This chapter explores the literature on BO in high-dimensional and adversarial settings. Firstly, we will conduct a comprehensive review of state-of-the-art methods and techniques that deal with high-dimensional optimization problems. Next, we will examine recent advancements in adversarial settings where observations are intentionally manipulated to undermine the BO process. Finally, we will study the practicality of employing BO to perform adversarial attacks on CNNs. This study aims to provide insights into the strengths and limitations of existing methods, and identify the research direction for our thesis in this exciting and rapidly evolving field.

2.1 High-Dimensional Bayesian Optimization

High-Dimensional Bayesian Optimization (HDBO) refers to the application of BO techniques to problems that have a large number of dimensions, typically more than ten but potentially even hundreds, thousands, or more. In such problems, the exponential growth of the search space makes it challenging to find the optimal solution. HDBO methods attempt to overcome this challenge by leveraging the structure of the problem and reducing the search space. This is achieved through techniques such as dimensionality reduction, feature selection, and surrogate modeling. HDBO has a wide range of applications, including hyperparameter tuning of machine learning models, sensor network design, and optimization of complex engineering systems.

A key difficulty associated with HDBO is the curse of dimensionality (Spruyt, 2014), *i.e.*, exponentially many observations are needed to find the global optimum in the absence of structural assumptions. Accordingly, two significant opposing challenges include the incorporation of suitable structural assumptions, and computationally efficient acquisition function optimization.

Here, we will review the different approaches that exist in the literature that tackles high dimensionality. We agree with Kandasamy that there are two key challenges in scaling BO to HDBO (Kandasamy et al., 2015):

1. **Regression is fundamentally especially challenging in high dimensions.** (Györfi, Kohler, Krzyzak, & Walk, 2002) showed that the lower bounds of regression are exponentially dependant in dimension. In addition, he showed that for high dimensions, the lower bound is not remotely close to zero even when large datasets are given. He concluded that the only way to overcome the curse of dimensionality is to *incorporate additional assumptions* about the regression function.
2. **Computational challenge in high dimensions.** BO translates the original problem of global optimization on f to another on acquisition function a . An effective acquisition function for HDBO must be computationally tractable. Contrastingly, many of the acquisition functions for BO have a computational complexity that's exponential in dimensions.

The literature highlights at least two distinct approaches to HDBO, each based on its own set of *additional* assumptions, which will be discussed in greater detail below.

- **Low effective dimensionality**, few dimensions significantly affect f .
- **Additive structure**, small subsets of variables interact with each other.

In addition to the methods described above, other approaches have been taken to tackle high dimensionality. (C. Li et al., 2017) proposed a dropout strategy to optimize on a smaller subset of variables for every iteration. (Oh, Gavves, & Welling, 2018) proposed Bayesian Optimization with Cylindrical Kernels, which tackles high-dimensionality via a cylindrical transformation of the search space.

(Eriksson, Pearce, Gardner, Turner, & Poloczek, 2019) proposed an approach based on running several local search procedures in parallel, and giving more samples to the most promising ones. Other methods use deep neural networks combined with BO, such as (Cui, Yang, & Hu, 2019; Snoek et al., 2015).

2.1.1 Low Effective Dimensionality

One early approach was based on the assumption that the problem exhibits low effective dimensionality. This assumption was based on observations by numerous researchers, that most dimensions do not change the objective function significantly. With such an assumption, (B. Chen, Castro, & Krause, 2012) performed joint variable selection and optimization using GP-UCB. This method is similar to performing feature selection and running the original problem on the reduced effective dimensionality. (Djolonga, Krause, & Cevher, 2013) applied low-rank matrix recovery techniques to learn the underlying effective subspace, and (Zhang, Li, & Su, 2019) proposed a related approach based on sliced inverse regression. (Ziyu Wang, Zoghi, Hutter, Matheson, & Freitas, 2013) solved the problem through random embedding; the method is known as Random Embedding Bayesian Optimization (REMOBO).

More recently, (Kirschner, Mutny, Hiller, Ischebeck, & Krause, 2019) proposed LineBO, decomposing the problem into a sequence of one-dimensional sub-problems. As such, BO can be implemented efficiently in a one-dimensional domain. The algorithm’s local and global convergence bounds rely on the existence of some invariant subspace. Recent proposals have also suggested the use of *non-linear* low-dimensional embeddings (Lu, Gonzalez, Dai, & Lawrence, 2018; Moriconi, Kumar, & Deisenroth, 2019). In short, these methods consider only low-dimensional functions and thereby avoiding both of the key challenges altogether as they have reduced the problem to a lower-dimensional subspace.

2.1.2 Additive Structure

Building on the formulation by (Kandasamy et al., 2015; Rolland et al., 2018), additive models assume that f can be decomposed into sums of lower-dimensional

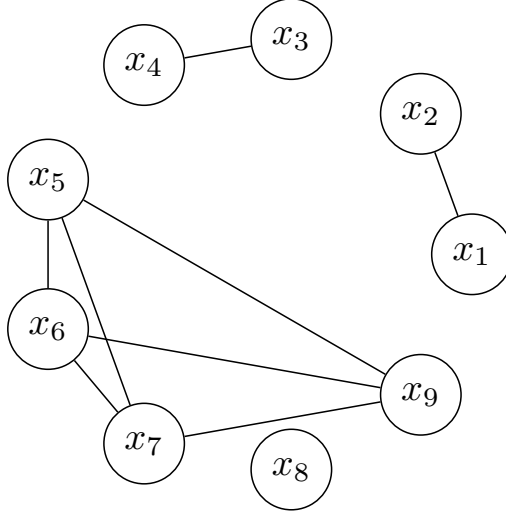


Figure 2.1. Example Dependency Graph No-Overlap structure, $g(x) = g^A(x_1, x_2) + g^B(x_3, x_4) + g^C(x_5, x_6, x_7, x_9) + g^D(x_8)$.

components:

$$f(x) = \sum_{G \in \mathcal{G}} f^G(x^G), \quad (2.1)$$

where $G \subseteq \{1, \dots, N_{\text{dim}}\}$ denotes one subset of variables, and $\mathcal{G}$ represents the additive structure. Each $f^G : \mathcal{X}^G \rightarrow \mathbb{R}$ is a low dimensional function defined on the variables in G , where $\mathcal{X}^G = \prod_{v \in G} \mathcal{X}_v$. Consequently, $|\mathcal{G}|$ is the number of low dimensional functions. Our formulation bears resemblance to the approach outlined in (Hoang et al., 2018), and we will discuss more shortly. In (Kandasamy et al., 2015)’s original formulation, the variables constructing a particular lower-dimensional function are not present in the other decomposed functions (i.e., the variables of each function are pairwise disjoint), which we refer to as **Graph No-Overlap**.

$$\forall_{I \in \mathcal{G}} \forall_{J \in \mathcal{G} \setminus \{I\}} : I \cap J = \emptyset \quad (2.2)$$

With this assumption, the acquisition function can be efficiently maximized along each $\mathcal{X}^G$ independently. We can visualize this formulation in a graph, a possible non-overlapping solution to can be visualized in Figure 2.1. Note that each maximal clique¹ represents a low-dimensional function.

¹A maximal clique in graph theory is a set of vertices in a graph where every vertex is directly connected to all other vertices in the set (Akkoyunlu, 1973).

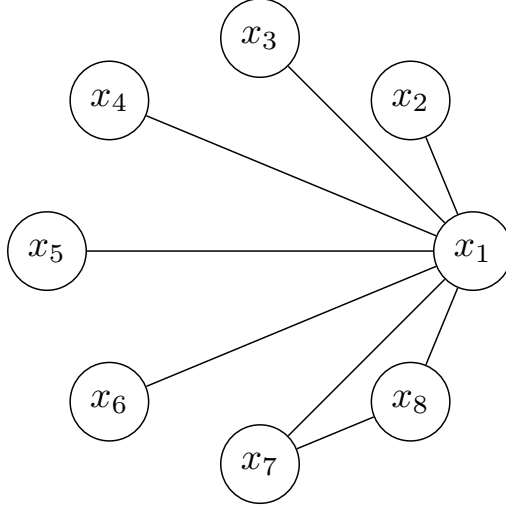


Figure 2.2. Example Dependency Graph Overlap structure, $g(x) = g^A(x_1, x_2) + g^B(x_1, x_3) + g^C(x_1, x_4) + g^D(x_1, x_5) + g^E(x_1, x_6) + g^F(x_1, x_7, x_8)$.

Overlapping Groups

The non-overlapping assumption is highly restrictive as it assumes that the variables of some f^G have absolutely no impact on other $f^{G'}$. This assumption may fail in practice (Naveršnik & Rojnik, 2012) as input components depend on each other. If the interdependence effects are not accounted for, it can degrade BO’s performance. (Rolland et al., 2018) generalized the additive model to allow for an arbitrary dependency graph, removing the restriction of pairwise disjointness (*i.e.*, Equation 2.2), which we refer to as **Graph Overlap**. However, with the additive model generalized, the optimization of the acquisition function will be over the entire domain of $\mathcal{X}$. In high dimensions, the optimization will not be scalable. Also considering overlapping groups, (Hoang et al., 2018) assumed that f can be decomposed into several sparse factor functions, allowing for distributed acquisition function approximation. As a result, a sparse factor graph representation of f can be used to optimize (approximately) the acquisition function in a distributed fashion. Impressively, he has shown the method to be able to scale to tuning a model with 1811 parameters.

(C.-L. Li et al., 2016) generalized to a projected-additive assumption; the model proposed by (Kandasamy et al., 2015) is a special case when there is no projection. Besides, the optimization for acquisition functions is restricted by a box constraint, allowing the optimization to be performed tractably. We

note that the non-overlapping group variant proposed earlier is an axis-aligned particular case of this model. (Rolland et al., 2018) extends (Kandasamy et al., 2015) work, formulating a tractable and generalized method to perform the optimization efficiently by message passing. The idea behind message passing is similar to dynamic programming, where the algorithm uses the structure of the problem to its advantage. It will perform the optimization of the acquisition function over the smallest possible domain. However, the computational complexity of message passing is still exponential in the size of the maximal clique as the smallest possible domain in a maximal clique is the entire domain. We provide an example in Figure 2.2.

Ensemble BO (Zi Wang, Gehring, Kohli, & Jegelka, 2018) seeks to not only exploit additive structures, but also use an ensemble of GP models through a divide and conquer strategy. (Mutny & Krause, 2018) combined additive GPs with approximations based on Fourier features, with the notable advantage of also establishing rigorous regret bounds.

2.1.3 Research Objective & Scope

The assumptions of low effective dimension vs. additive structure are complementary. The performance of the optimization is dependent on the structure of the high-dimensional function, and trade-offs exist between computation time and accuracy. Methods that assume low effective dimensionality are often computationally faster than additive methods; *e.g.*, due to scalability concerns, (Eriksson et al., 2019) omitted methods that attempt to learn an additive decomposition from their experiments. While the models that assume low effective dimensionality scale well, it works well in problems such as hyperparameter optimization (Bergstra, Bardenet, Bengio, & Kégl, 2011; Ziyu Wang et al., 2013). To the best of our knowledge, none of the existing works have scaled **Graph Overlap** past 20 to 30 dimensions, except for (Hoang et al., 2018) who scaled a similar setup using decentralization and relies on convergence for message passing.

Our interest is on additive structures; in particular, we seek to scale **Graph Overlap**. **Graph Overlap** maintains computational tractability by using a message passing algorithm to optimize the acquisition function efficiently. However,

the message passing algorithm runs exponentially in the size of the maximum clique of the triangulated dependency graph (Rolland et al., 2018), impeding its scalability.

We see in the above-outlined works (Hoang et al., 2018; C.-L. Li et al., 2016; Rolland et al., 2018) that the trend in the recent research of additive models has been to increase the model expressiveness. An important caveat to such approaches is that a suitable model may be *much harder to find* given limited samples. Since one of the main premises of BO is optimizing with few samples, we contend that *simpler* models should also be sought to facilitate model learning with fewer samples, as well as reduced computation. Philosophically, our method follow Occam’s razor, to prefer the simplest hypothesis (Russell & Norvig, 2020). Richer models are potentially more susceptible to noise in the data, whereas simpler models may model the problem better. Besides, simpler models are computationally easier to learn, allowing the method to scale to higher dimensions.

We delve into this matter in greater depth and introduce our methodology in Chapter 3. Our method involve trading off model expressiveness for scalability and ease of learning through the utilization of tree structures, along with introducing a zooming technique and a hybrid method for efficient learning with competitive performance.

Research Question

It is possible to trade-off model expressiveness for scalability for Graph Overlap by introducing constraints on the graph structure?

2.2 Adversarial Attacks on Bayesian Optimization

Numerous GP optimization methods have been developed in the literature to overcome the various forms of uncertainty, and achieve some associated notion of robustness:

- *Presence of outliers*, where some function evaluations are highly unreliable (Martinez-Cantin, Tee, & McCourt, 2018).

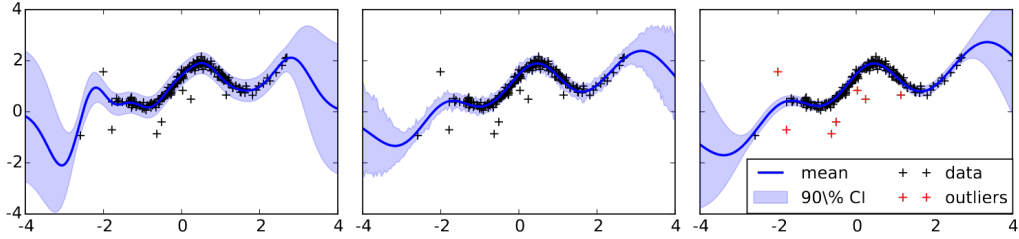


Figure 2.3. Regression with outliers using GP (left), using Student- t (center); using Student- t to remove the outliers with GP on the remaining points; Illustration from (Martinez-Cantin et al., 2018), reproduced under Creative Commons Attribution 4.0 International License.

- *Random perturbations to sampled points*, where the sampled points are subject to random uncertainty (Beland & Nair, 2017; Nogueira, Martinez-Cantin, Bernardino, & Jamone, 2016).
- *Adversarial perturbations to the final point*, where the final recommendation x may be perturbed up to some level ϵ (Bertsimas, Nohadani, & Teo, 2010; Bogunovic, Scarlett, Jegelka, & Cevher, 2018).
- *Adversarial perturbations to samples*, where the observations are adversarially corrupted up to some maximum budget (Bogunovic et al., 2020).

2.2.1 Presence of outliers

(Martinez-Cantin et al., 2018) observed that typical BO uses an observation model for noisy data, $\epsilon_i \sim \mathcal{N}(0, \sigma^2)$ as defined in Equation 1.2. When there are outliers in the data, the underlying GP model would be affected by the outliers, which can result in biased estimates and high variance; See Figure 2.3. Outliers are modelled with α , where α is the predefined level of outliers outside the upper and lower α -percentile of the predicted Student- t distribution. In the BO framework, new observations arrive every iteration and because it can help improve the model – the model is refitted with all the data on some schedule.

(Martinez-Cantin et al., 2018) focused on proposing their method which uses the Student- t to remove any points that are in the α part of the tail that belongs to outliers. From an experimental perspective, this characterization of uncertainty leans towards a larger α to enhance performance, even if it reduces the effective sample size. As one might expect, (Martinez-Cantin et al., 2018) found that an aggressive α worked better in practice as a misclassification of a

non-outliers does not affect the underlying GP significantly and are often classified correctly in subsequent iterations. We note that although this work gives a method for experimentally identifying outliers, it does not attempt to describe an explicit mathematical model.

2.2.2 Random perturbations to sampled points

Here we group the methods (Beland & Nair, 2017; Nogueira et al., 2016) which attempt to model uncertainty by considering input random noise. For example, (Beland & Nair, 2017) considered the following setup:

$$f(x_1, x_2 + \delta, \xi) + \tau, \quad (2.3)$$

where δ, ξ are random variables with specified joint probability density $p(\delta, \xi)$, along with the $\tau \sim \mathcal{N}(0, \sigma^2)$ modelling measurement noise. Similarly by considering input noise, (Nogueira et al., 2016) assumes that the input distribution corresponds to the input noise in each query.

Similarly, (Beland & Nair, 2017; Nogueira et al., 2016) focuses on proposing their corresponding method that deals with the specified setup. We note that this type of uncertainty is quite different from the presence of outliers.

2.2.3 Adversarial perturbations to the final point

(Bogunovic et al., 2018) considers a slightly different optimization objective (also similar, building on non-BO works such as (Bertsimas et al., 2010)), seeking to find the optimum point that is robust against errors that is bounded; he considers the objective to find an ϵ -stable input x_ϵ^* :

$$x_\epsilon^* \in \arg \max_{x \in \mathcal{D}} \min_{\delta \in \Delta_\epsilon} f(x + \delta), \quad (2.4)$$

where the set of perturbations of x where the adversarial perturbations δ are bounded by some ϵ distance from x . We bound δ by some distance to make the problem meaningful. An adversary must find an adversarial perturbation δ in the absence of *any* knowledge. In order to make the problem meaningful, δ would be bounded by some ϵ distance; typical distance norms include ℓ_2 and ℓ_∞

norms.

2.2.4 Adversarial perturbations to samples

(Bogunovic et al., 2020) considers a problem instance that is C -corrupted with $\tilde{y}_i$ corrupted observations, on top of the uncorrupted observation y_i as per Equation 1.2. Furthermore, it is noteworthy that this work builds upon analogous investigations in the context of multi-armed bandits, such as the study by (Lykouris, Mirrokni, & Leme, 2018).

$$\tilde{y}_i = y_i + c_i(x_i), \quad \sum_{i=1}^T \max_{x \in \mathcal{D}} |c_i(x)| \leq C \quad (2.5)$$

We note that in this setting, the observations are adversarially corrupted and are adaptive, which is significantly different from the settings previously discussed. The work focused on introducing an algorithm for this setting along with theoretical analysis upper bounding the cumulative regret.

2.2.5 Research Objective & Scope

The aforementioned studies, along with others, have developed methods that go beyond traditional uncertainty modeling using Random Noise. They typically defend against the proposed uncertainty model to improve robustness for GP optimization. It can be difficult to know and tailor for the specific issue (*i.e.*, outliers, nonlinear effects), so the cautious approach is to simply ‘prepare for the worst’ and consider worst-case adversarial perturbations. Though works on defense have been devised in the literature, there are minimal work studying the problem from an attacker’s perspective, *i.e.*, what kinds of attacks would be successful against non-robust algorithms. We examine such a perspective in this work, focusing on adversarial perturbations to samples (Bogunovic et al., 2020).

Our study is related to that of attacks on stochastic linear bandits (Garcelon et al., 2020), but we move to the GP setting which is inherently non-linear and poses substantial additional challenges. We focus in particular popular algorithms into selecting from a specific set of (typically sub-optimal) *target actions* as much as possible, using GP-UCB (Srinivas et al., 2010) and a related

elimination-based algorithm as representative examples. Prior to (Garcelon et al., 2020), analogous works studied attacks and defenses for multi-armed bandits (Jun, Li, Ma, & Zhu, 2018; Lykouris et al., 2018). These are less related here, since they assume finite domains with independent arms.

In Chapter 4, we study the problem from an attacker’s perspective, and introducing various practical attacks inspired by our study. We applied the attacks and demonstrated the practicality of such attacks.

Research Question

It is possible construct practical attacks on common BO algorithms based on the adversarial perturbations to samples setting in (Bogunovic et al., 2020)?

2.3 Black-box Adversarial Attacks on CNNs

Having explored the nuances of adversarial attacks within BO, it’s pertinent to also address another critical domain where adversarial vulnerabilities are prominent: Convolutional Neural Networks (CNNs). Given the foundational role CNNs play in many of today’s AI applications, understanding their adversarial landscape is of paramount importance. Continuing the discussion from Section 1.2.1, we shift our focus to black-box attacks, where access to model information is absent, and these attacks are often distinguished by the degree of feedback available from the model. In the literature, feedback types available to adversaries are primarily categorized into two main groups (Galstyan & Cohen, 2007; Shukla et al., 2021):

- **Soft-label** provides more information than just the label; The adversary has access to the score (*i.e.*, probability or likelihood) over the possible labels, indicating the model’s confidence in each possible class.
- **Hard-label** refers to the output label assigned to the input. The adversary has no access to any other information about how the output label was derived.

We assume that x is a D -dimensional input and the model is learning the map-

ping to one of the class $c \in C = \{\forall_{i=1,\dots,K} : c_i\}$. Soft-label setting assume access to the attacked model classifier F_s as a function:

$$F_s(x) : \mathbb{R}^D \rightarrow [0, 1]^K, \quad c = \arg \max_{\forall c_i} [F_s(x)]_i \quad (2.6)$$

The majority of research on black-box adversarial attacks has focused on soft-label attacks, in which the queries provide the adversary with the entire final output layer of the network in the form of a score $[0, 1]^K$. Contrastingly, hard-label setting assume access to the attacked model classifier F_h :

$$F_h(x) : \mathbb{R}^D \rightarrow C \quad (2.7)$$

In the context of black-box setting where information is already scarce, soft-label setting affords a considerable amount of information to the adversary. In contrast, hard-label setting is most restrictive and often more realistic.

For instance, since the advent of cloud computing and Machine-Learning-as-a-Service (MLaaS), enterprises can easily integrate and apply machine learning capabilities via Application Programming Interfaces (API) or web interfaces, eliminating the need for implementation and maintenance of infrastructure and models (Ribeiro, Grolinger, & Capretz, 2015). MLaaS exposes the ML model to model stealing, where an adversary infers sensitive information about a machine learning model by analyzing its outputs, potentially leading to unauthorized replication or intellectual property theft (Tramèr, Zhang, Juels, Reiter, & Ristenpart, 2016). In some cases, disturbingly, near-perfect recovery rate of stolen models is possible and can be achieved (Y. He, Meng, Chen, Hu, & He, 2019). Many defence methods have been devised to protect the ML model against theft, however, restricting information, such as allowing access to hard-labels instead of soft-labels is a useful strategy (Juuti, Szyller, Marchal, & Asokan, 2019).

Hence, we focus on the hard-label setting as it is the most restrictive setting and most general practically; any attack that works in the hard-label setting can be applied in the soft-label setting, but not vice versa. We detail an overview of the various approaches that exist in literature.

2.3.1 Transfer-based methods

Early works in this setting have relied on training a substitute model that approximates the classification behaviour of the target model. As the substitute is fully accessible, the adversary is able to attack the substitute using any existing white-box methods; and then transfers these attacks to the original target model. This approach depends on the assumption of transferability [Szegedy et al. (2014);goodfellow2015explaining], which implies that adversarial examples that successfully fool the substitute are also likely to deceive the target model, as both models have similar classification performance on the same data. (Brunner, Diehl, Truong-Le, & Knoll, 2019) exploits biases from domain knowledge: image frequency, regional masks and surrogate gradients to formulate the attack. These methods require access to significant additional resources to create some proxy, such as additional queries and computation to train a substitute model in (Papernot et al., 2017), and access to an ensemble of models in (Y. Liu, Chen, Liu, & Song, 2017). The requirement for access to additional information can be restrictive in real-world applications, especially when a large number of queries is required for large and complicated models (Brendel, Rauber, & Bethge, 2018).

2.3.2 Gradient Estimation

Instead of training a substitute model, an alternative is to estimate the gradient and then use this estimated gradient information to generate attacks. A naive approach would be to perform coordinate-wise gradient estimation, requiring large number of queries, *i.e.*, at least 2 queries per each coordinate. To address this issue, (P.-Y. Chen, Zhang, Sharma, Yi, & Hsieh, 2017) employed stochastic coordinate gradient descent and importance sampling to select batches of coordinates, and demonstrated that his method (ZOO) is computationally tractable and achieved comparable attack success rates on high-dimensional image data. AutoZOOM (C.-C. Tu et al., 2019), which is an improvement over ZOO, significantly improves the query efficiency by utilizing random vectors to estimate the full gradient, and adapts the estimation parameter to balance query efficiency and input perturbation cost. Sign-Opt (Cheng et al., 2020) applies zeroth order optimization to efficiently estimate the sign of the gradient to perform an hard-

label attack. Alternative approach to estimating the gradient involves using a modified version of natural evolution strategy (Ilyas, Engstrom, Athalye, & Lin, 2018), and using time and data-dependent priors to improve the estimate (Ilyas, Engstrom, & Madry, 2019). In general, gradient estimation approaches tend to require many queries and are often less robust to gradient-manipulating defenses (Athalye, Carlini, et al., 2018; Brendel et al., 2018; Guo, Rana, Cissé, & Maaten, 2018).

2.3.3 Gradient-free Optimization

To move beyond the attack’s reliance on gradient data and enhance query efficiency, GenAttack (Alzantot et al., 2019) employs a genetic algorithm to progressively generate a population of potential adversarial examples, using dimensionality reduction techniques to improve query efficiency. More recently, (Vo, Abbasnejad, & Ranasinghe, 2022) proposed an evolution-based algorithm for the sparse setting. Alternative methods bypassing gradient estimation involve various strategies: employing constrained random sampling (Brendel et al., 2018; Guo, Gardner, You, Wilson, & Weinberger, 2019), discretizing via combinatorial optimization to identify potential perturbations (Moon, An, & Song, 2019), and reframing the problem through a boundary-based approach (Cheng et al., 2019). However, such approaches tend to use a large number of queries. Enhancing the random sampling approach, (Andriushchenko, Croce, Flammarion, & Hein, 2020a) use a randomized search with square-shaped updates, which offers a notable improvement to the query efficiency.

Our interest in gradient-free methods, especially BO based methods, arises from their query efficiency in challenging settings. BO’s successful applications include successful low-dimensional ($D < 100$) attacks (Suya, Tian, Evans, & Papotti, 2017), to image space optimization at the cost of higher distortion (Zhao et al., 2019). Combining BO with Bayesian optimal stopping and latent space dimension selection (Kharkovskii, 2020), or integrating it with additive GP models for search space reduction (Ru, Cobb, Blaas, & Gal, 2020), have demonstrated successful attacks requiring fewer queries. Furthermore, (Shukla et al., 2021) showed the effectiveness of combining BO with various dimensionality reduction

techniques. BO identifies a *latent perturbation*, which, after applying dimensionality reduction, is transformed into an *image perturbation* for the attack.

2.3.4 Research Objective & Scope

Although most research focuses on the simpler soft-label setting, hard-label approaches face challenges like needing many queries (Brendel et al., 2018, p. 9008375; Cheng et al., 2019, p. vo2022query) or causing notable distortion (Zhao et al., 2019). We focus on the challenging query-limited black-box hard-label setting (Shukla et al., 2021).

The dimensionality techniques used in (Shukla et al., 2021) were Nearest Neighbour Interpolation (NNI) and Low-frequency Fourier Transform (LF), for ℓ_∞ , and ℓ_2 untargeted attacks respectively; We will discuss these methods in greater detail in Chapter 5. We can also better exploit domain knowledge (Brunner et al., 2019) via improving the dimensionality techniques used. For CNNs, it is understood in literature that feature maps at the initial layers encode simpler features such as edges and direction; as we go deeper in layers the feature maps encode more complex parts of the objects that it is trying to classify (see (LeCun, 2016)). We can potentially tailor the ℓ_∞ attacks by targeting the lower level features. From (Yin et al., 2019)’s statistical study the robustness of different neural networks to ℓ_2 attacks on different frequencies, they found that adversarial training and Gaussian data augmentation improve robustness in higher frequencies but sacrifice it in lower frequencies, while naturally trained models are sensitive to noise in all but the lowest frequencies. This work motivates the use of a frequency-adaptive approach in ℓ_2 attacks, instead of only attacking the lower frequency spectrum in (Shukla et al., 2021).

In Chapter 5, we propose transformations of the latent space for CNNs (for ℓ_2, ℓ_∞ attacks) that works by exploiting domain knowledge of CNNs. We also discuss approaches for query-efficient hyperparameter selection during attacks.

Research Question

It is possible to exploit domain knowledge in order to improve the dimensionality techniques in (Shukla et al., 2021) to attack CNNs?

Chapter 3

Additive GP-UCB using Tree Structures

We consider the sequential global optimization problem in Equation 1.1, where we seek $x_{\max} = \arg \max_{x \in \mathcal{X}} f(x)$ for a N_{dim} -dimensional black-box function $f : \mathcal{X} \rightarrow \mathbb{R}$, where $\mathcal{X} = \times_{i=1}^{N_{\text{dim}}} \mathcal{X}_i$ with each $\mathcal{X}_i$ being an interval in $\mathbb{R}$. At the t -th observation, the algorithm selects x_t and observes a noisy observation $y_t = f(x_t) + \epsilon_t$, with $\epsilon_t \sim \mathcal{N}(0, \eta^2)$.

3.1 Contributions

The main contributions are as follows:

1. We make a trade-off between expressiveness for scalability and ease of learning by reducing the complexity of the additive model, constraining the dependency structure to tree structures. As the function class is simpler, it reduces overfitting of the GP kernel, and we are also able to reap computational efficiencies in both acquisition function optimization and dependency structure learning.
2. We propose a zooming technique for extending the message passing algorithm of (Rolland et al., 2018) to continuous domains, thus benefiting additive methods in general, and in particular our tree-based approach.
3. We propose a hybrid method to learn the additive tree structures, composed of the following two techniques:

- a. an algorithm that grows tree structures that efficiently discovers edges that do not form cycles via Gibbs sampling;
 - b. an edge mutation algorithm that obtains a new generation of trees from the current tree efficiently.
4. Although limiting to tree structures may seem potentially risky due to the reduced expressivity, we show this approach to be highly effective in a wide range of experiments, indicating a highly competitive trade-off between expressive power and ease of model learning.

It should be noted that the application of tree structures in BO has been explored in previous research (Jenatton, Archambeau, González, & Seeger, 2017; Ma & Blaschko, 2020), but with a very different type of model and motivation. These works aim to handle structured domains instead of real-valued domains, and in contrast with our work, the tree represents binary decisions with only the leaves corresponding to Gaussian Processes.

3.2 Setup

We use a Gaussian Process (GP) model to reason about the target function f . Following (Rolland et al., 2018), we model f as a sum of several lower-dimensional components:

$$f(x) = \sum_{G \in \mathcal{G}} f^G(x^G) \quad (3.1)$$

where $G \subseteq \{1, \dots, N_{\text{dim}}\}$ denotes one subset of variables, and $\mathcal{G}$ represents the additive structure (see Figure 3.1 for an example). The additive dependency structure is assumed to be tree-structured, which could include forests. In contrast with (Rolland et al., 2018), in our setting the additive structure associated with

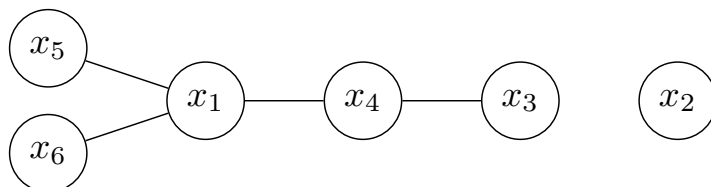


Figure 3.1. Dependency tree structure, $h(x) = h^A(x_1, x_6) + h^B(x_1, x_5) + h^C(x_1, x_4) + h^D(x_3, x_4) + h^E(x_2)$.

any given graph is unique: Each lower-dimensional component $f^G : \mathcal{X}^G \rightarrow \mathbb{R}$ is either a 1 or 2-dimensional function defined on the variables in G , where $\mathcal{X}^G = \bigtimes_{v \in G} \mathcal{X}_v$. Each vertex represents a unique variable, and an edge exists between each pair of variables of every 2-dimensional function. Each edge represents a 2-dimensional function, and each disconnected vertex represents a 1-dimensional function. It follows from the tree constraint that every pairwise combination of variables can only be connected by at most one path.

3.2.1 Prior and Posterior

We model $f \sim \mathcal{GP}(\mu, \kappa)$, with each component f^G being an independent sample from a Gaussian Process $\mathcal{GP}(\mu^G, \kappa^G)$, and

$$\begin{aligned}\mu(x) &= \sum_{G \in \mathcal{G}} \mu^G(x^G), \\ \kappa(x, x') &= \sum_{G \in \mathcal{G}} \kappa^G(x^G, x'^G).\end{aligned}\tag{3.2}$$

We know from (Rolland et al., 2018) that the posterior can be inferred via $(f_*^G | \mathbf{y}) \sim \mathbb{N}(\mu_{t-1}^G, (\sigma_{t-1}^G)^2)$ for each f_*^G at an arbitrary point x_* given $\mathcal{D}_t = \{(x_i, y_i)\}_{i=1}^t$, where $\mathbf{y} = (y_1, \dots, y_t)$ correspond to $\mathbf{x} = (x_1, \dots, x_t)$, and the posterior mean and variance are given by

$$\begin{aligned}\mu_{t-1}^G &= \kappa^G(x_*^G, \mathbf{x}^G) \Delta^{-1} \mathbf{y}, \\ (\sigma_{t-1}^G)^2 &= \kappa^G(x_*^G, x_*^G) \\ &\quad - \kappa^G(x_*^G, \mathbf{x}^G) \Delta^{-1} \kappa^G(\mathbf{x}^G, x_*^G).\end{aligned}\tag{3.3}$$

Here we define the matrix $\Delta = \kappa(\mathbf{x}, \mathbf{x}) + \eta^2 I_t \in \mathbb{R}^{t \times t}$, $\kappa(x_i, x_j)$ is the (i, j) -th entry of $\kappa(\mathbf{x}, \mathbf{x})$, and $\kappa^G(\mathbf{x}^G, x_*^G)$ is of length t , with i -th entry $\kappa^G(x_i^G, x_*^G)$.

3.3 Tree-GP-UCB Algorithm

We present Tree-GP-UCB (Tree for short). Here, the total number of observations is $N = N_{\text{init}} + N_{\text{iter}}$, where N_{init} is the number of initial random samples X_{init} drawn uniformly from $\mathcal{X}$ and N_{iter} is the number of iterations. For efficiency, $\mathcal{G}$ and its hyperparameters are learned every C iterations, for some $C > 0$.

Algorithm 3.1: TREE-GP-UCB

```
1 Initialize  $\mathcal{D}_{N_{\text{init}}} \leftarrow \{(x_t, y_t)\}_{x_t \in X_{\text{init}}}$ 
2 for  $t = N_{\text{init}} + 1, \dots, N$  do
3   if  $t \bmod C = 0$  then
4     Learn  $\mathcal{G} \leftarrow \text{TREE-LEARNING}$  (Algorithm 3.4)
5     Update  $\mu_t^G, \sigma_t^G : \forall G \in \mathcal{G}$  (3.3)
6     Optimize  $x_t \leftarrow \arg \max_{x \in \mathcal{X}} \phi_t(x)$  (Algorithm 3.3)
7     Observe  $y_t \leftarrow f(x_t) + \epsilon$ 
8     Augment  $\mathcal{D}_t \leftarrow \mathcal{D}_{t-1} \cup \{(x_t, y_t)\}$ 
9 return  $\arg \max_{(x,y) \in \mathcal{D}} y$ 
```

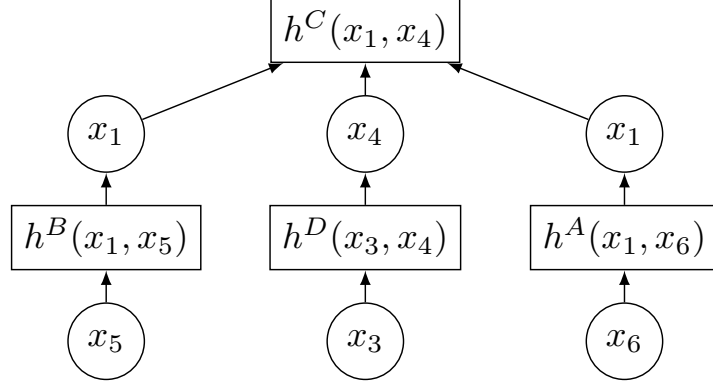


Figure 3.2. Illustration of the optimization of the acquisition function on the graph $\mathcal{G}$ shown in Figure 3.1. First, the leaves of the junction tree perform partial maximization of the functions h^B , h^D , and h^A over their respective variables x_4 , x_2 and x_5 . The result of the partial maximization is a ‘conditional’ maximization over the other variables. These ‘conditional’ maximizations are passed as messages to the nodes above. In this manner, the messages are passed until the root node, and the correct answer is returned.

3.3.1 Acquisition Function

We focus on upper confidence bound (UCB) based algorithms (Auer, 2002; Srinivas et al., 2010). Specifically, following (Kandasamy et al., 2015) and (Rolland et al., 2018), we define the global acquisition function $\phi_t(x)$ be the sum of the individual acquisition functions with respect to the dependency structure $\mathcal{G}$:

$$\begin{aligned} \phi_t(x) &= \sum_{G \in \mathcal{G}} \phi_t^G(x^G), \\ \phi_t^G(x^G) &= \mu_{t-1}^G(x^G) + \beta_t^{1/2} \sigma_{t-1}^G(x^G). \end{aligned} \tag{3.4}$$

Maximization over Discrete Domains

For BO to be effective in high dimensions, the underlying acquisition function needs to be optimized efficiently. In Algorithm 3.2, we present a simplified vari-

Algorithm 3.2: MSG-PASSING-DISCRETE

```

1 Select a node  $R$  to root the junction tree
2  $D_{\text{tree}} \leftarrow$  depth of the rooted tree
3 Initialize  $\Phi \leftarrow \emptyset$ 
4 for  $d = D_{\text{tree}}, \dots, 1$  do
5   for every node  $C$  at distance  $d$  from  $R$  do
6      $C_p = \text{parent}(C)$ 
7     Common variables  $I \leftarrow C \cap C_{\text{parent}}$ 
8     Marginal variables  $J \leftarrow C \setminus C_{\text{parent}}$ 
9     for  $x^I \in \mathcal{X}^I$  do
10       $M_{C \rightarrow C_p}[x^I] =$ 
11       $\max_{x^J \in \mathcal{X}^J} \psi^C(x^I, x^J) + \sum_{C_c \in \text{children}(C)} M_{C_c \rightarrow C}[x^{C \cap C_c}]$ 
12      Update  $\Phi$  to include all  $\phi^G$  that were optimized in  $\psi^C$ 
12 return  $\max_{x^R \in \mathcal{X}^R} \psi^R(x^R) + \sum_{C_c \in \text{children}(R)} M_{C_c \rightarrow R}[x^{R \cap C_c}]$ 

```

ant of the algorithm proposed by (Rolland et al., 2018) for generalized additive models. We deal with the optimization of the acquisition function of a given dependency structure $\mathcal{G}$ and discrete domain $\mathcal{X} = \times_{i=1}^{N_{\text{dim}}} \mathcal{X}_i$ with $\mathcal{X}_i$ discretized uniformly with N_{gridsize} discrete values, between a_i and b_i inclusive. The algorithm uses the message passing technique on a triangulated graph, similar to the efficient maximization of probability in Markov random fields (Wainwright, Fosson, & Ravazzi, 2015). We will shortly discuss a simplified variant in the case of tree-structured graphs.

Algorithm 3.2 makes use of a junction tree, with each node in the junction tree representing a maximal clique of the triangulated dependency graph. Since we have changed the structure of the dependency graph by triangulation, we need to carefully account for the individual acquisition functions ϕ^G (see Equation 3.4), because they are no longer directly equivalent to the function that we are optimizing on each node of the junction tree. We simplify (Rolland et al., 2018)’s method by excluding the acquisition functions ϕ^G that have been previously maximized:

$$\psi^C(x^C) = \sum_{\forall G \in \mathcal{G}: (\phi^G \notin \Phi \wedge G \subseteq C)} \phi^G(x^G) \quad (3.5)$$

where Φ denotes the set of acquisition functions that have been previously maximized. Figure 3.2 shows an example.

Algorithm 3.3: MSG-PASSING-CONTINUOUS

```
1 Initialize  $(\mathbf{a}, \mathbf{b})$  with the bounds of  $\mathcal{X}$ 
2 for  $l = 1, \dots, N_{\text{lvl}}$  do
3   for  $d = 1, \dots, N_{\text{dim}}$  do
4     Discretize  $\mathcal{X}_d \leftarrow [[a_d, b_d]]_{N_{\text{gridsize}}}$  //  $|\mathcal{X}_d| = N_{\text{gridsize}}$ 
5    $\mathcal{X} \leftarrow \bigtimes_{d=1}^{N_{\text{dim}}} \mathcal{X}_d$ 
6    $(x, y) \leftarrow \text{MSG-PASSING-DISCRETE}(\mathcal{X})$ 
7   Select  $(\mathbf{a}, \mathbf{b}) \leftarrow \text{ZOOM-STRATEGY}(x)$ 
8 return  $(x, y)$ 
```

Complexity Algorithm 3.2 applies to arbitrary graph structures, allowing this technique to be applied to general additive models. The complexity of running the algorithm is exponential in the size of the maximum clique of the triangulated graph (Rolland et al., 2018). Hence, the algorithm can benefit from choosing a triangulation that minimizes the size of the maximal clique (Arnborg, Corneil, & Proskurowski, 1987; Cano & Moral, 1994). In *Tree*, the dependency structure is a tree. As a result, there are no cycles, and there is no need to triangulate the tree $\mathcal{G}$. Moreover, the function of ψ^C can be simplified to $\psi^C = \phi^C$. Hence, the complexity of the maximization is reduced to quadratic in N_{gridsize} .

Maximization over Continuous Domains

The message passing approach proposed by (Rolland et al., 2018) works on discrete domains. A naive approach to handle continuous domains would be to discretize the continuous domain uniformly (i.e., a grid with equal spacing). However, this may require large amounts of computation, especially when the discretization is performed using a small spacing. Here, we present a refined message passing algorithm specifically designed for continuous domains.

The optimization of the acquisition function over continuous domains is presented in Algorithm 3.3; it begins with the entire continuous domain $\mathcal{X} = \bigtimes_{d=1}^{N_{\text{dim}}} \mathcal{X}_d$, where $\mathcal{X}_d \in [a_d, b_d] \subseteq \mathbb{R}$. Firstly, we discretize each variable’s domain to a finite subset, and let N_{gridsize} denote the size of the subset. Thereafter, we use a simplified version of the message passing algorithm MSG-PASSING-DISCRETE of (Rolland et al., 2018) — Algorithm 3.2 — to perform optimization over the discretized domain. As the dependency graph is a tree, the complexity of mes-

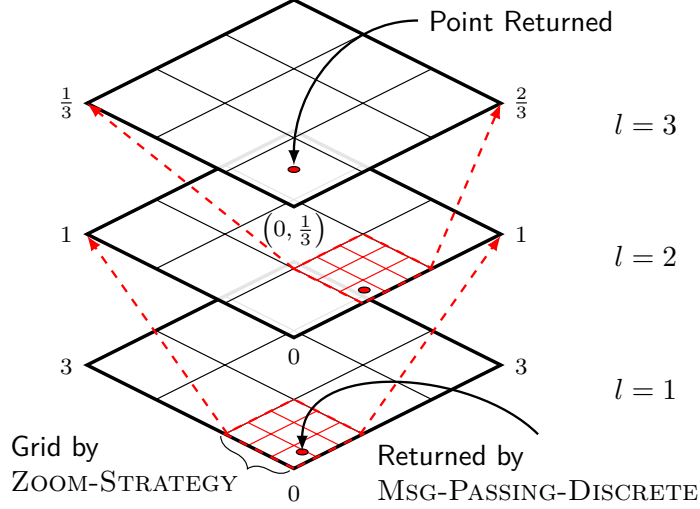


Figure 3.3. Example with two variables, grid size $N_{\text{gridsize}} = 3$, and domain $[0, 3]$. Firstly, we partition each axis evenly into 3 partitions. Next, we draw a uniformly random point from each partition. The points from each axis form a discretized domain, and we run MSG-PASSING-DISCRETE on this discrete domain. Finally, we zoom into the square, representative of the selected point. In this manner, we recursively sub-divide the grid for all $N_{\text{lvl}} = 3$ levels.

sage passing is quadratic in N_{gridsize} . The bounds $(\mathbf{a}, \mathbf{b})$ for the next level are picked by ZOOM-STRATEGY (see below) given the selected point. We perform the steps iteratively for some number N_{lvl} of levels.

Zoom Strategy Different strategies can be employed in choosing the bounds and their representative points for the next level. We adopt a simple randomized strategy exemplified in Figure 3.3: At each level, we divide each current interval $[a_d, b_d]$ uniformly onto a grid of size N_{gridsize} , and choose a uniformly random point within that interval as its representative. We refer to this discretization of the domain as $[[a_d, b_d]]_{N_{\text{gridsize}}}$. We use MSG-PASSING-DISCRETE restricted to these representatives, and for the one chosen, we recursively zoom into the corresponding sub-domain. Henceforth, we use MSG-PASSING-CONTINUOUS with this zoom strategy.

3.3.2 Additive Components

The choice of an appropriate kernel and the learning of its parameters are critical to the success of BO. In high-dimensional additive BO, the problem compounds, as we need to learn the dependency structure along with kernel parameters for

every kernel in the additive model.

As mentioned previously, an additive decomposition $\mathcal{G}$ corresponds to a dependency graph; the additive function $f(x) = \sum_{G \in \mathcal{G}} f^G(x^G)$ is the sum over its additive components in $\mathcal{G}$. It will be convenient to work with the equivalent representation of an adjacency matrix $Z \in \{0, 1\}^{N_{\text{dim}} \times N_{\text{dim}}}$, where $Z_{ij} = 1$ if variables i and j are connected on the (tree-structured) graph. We enforce that Z must be tree-structured (including forests), *i.e.*, there must be no cycles in $\mathcal{G}$. Assuming that each function's kernel κ^G is parameterized by some kernel parameters θ^G (e.g., lengthscale etc.), the overall collection of parameters is $\Theta_{\mathcal{G}} = \{\theta^G\}_{G \in \mathcal{G}}$ given a decomposition $\mathcal{G}$. We note that learning the kernel parameters along with the decomposition $\mathcal{G}$ is difficult, given the large search space and we may encounter problems with overfitting. We tackle this problem by defining a fixed set of kernel parameters Θ that are independent of the decomposition and defining the kernel parameters over them; see Section 3.4.1 for details.

Maximum likelihood For model learning, we make use of the maximum log-likelihood score as given by

$$\begin{aligned} \rho(Z, \theta) = & -\frac{1}{2} \mathbf{y}^T (K + \sigma_n^2 I)^{-1} \mathbf{y} \\ & -\frac{1}{2} \log |K + \sigma_n^2 I| - \frac{n}{2} \log 2\pi, \end{aligned} \tag{3.6}$$

where $K \in \mathbb{R}^{n \times n}$ is the kernel matrix of the observed data points n , assuming a dependency graph $\mathcal{G}$ with an equivalent adjacency matrix Z and parameters θ .

Dependency Structure Learning

Following (Rolland et al., 2018; Zi Wang, Li, Jegelka, & Kohli, 2017), we use a Bayesian approach to structure learning, on which we place a prior distribution on Z and seek to sample from the posterior distribution. We use Gibbs sampling to sample approximately, avoiding the difficult task of sampling directly from the high-dimensional distribution over tree structures.

Specifically, we use such sampling to update the presence/absence of edges from variable i to j , but to maintain the tree structure, we discard edges that

Algorithm 3.4: TREE-LEARNING

```

1  $\mathcal{Z} \leftarrow \{Z^{\text{current}}\}$ 
2  $Z^{(k)} \leftarrow Z^{\text{current}}$ 
3 while  $k < N_{\text{samples}}$  do
4   if  $\text{Number-Of-Edges}(Z^{(k)}) < N_{\text{dim}} - 1$  then
5      $\text{Update}(\mathcal{Z}, k)$  via GIBBS-SAMPLING (Algorithm 3.5)
6   else
7      $\text{Update}(\mathcal{Z}, k)$  via MUTATION (Algorithm 3.6)
8 return  $Z \in \mathcal{Z}$  with the highest likelihood score
```

would create a cycle. We assume a prior with Bernoulli random variables with parameter γ , $Z_{ij} \sim \text{Bernoulli}(\gamma)$. We can use this model to formulate the posterior for Z_{ij} ; letting $\mathcal{D}$ denote the data collected, and letting $Z_{-(ij)}$ be the adjacency variables excluding (i, j) , we have the following (Rolland et al., 2018):

$$\begin{aligned}
P(Z_{ij} = 1 \mid Z_{-ij}, \theta, \mathcal{D}; \gamma) &\propto P(\mathcal{D} \mid Z_{ij} = 1, Z_{-ij}, \theta; \gamma) \\
&\quad \times P(Z_{ij} = 1 \mid Z_{-ij}, \theta; \gamma) \\
&\propto \gamma P(\mathcal{D} \mid Z_{ij} = 1, Z_{-ij}, \theta; \gamma) \\
&\propto \gamma \times e^{\rho(Z_{ij}=1 \cup Z_{-ij}, \theta)}.
\end{aligned} \tag{3.7}$$

For each Z_{ij} , we compare the log of the posterior for two cases: $\log(\gamma) + \rho(Z_{ij} = 1 \cup Z_{-ij}, \theta)$ vs. $\log(1 - \gamma) + \rho(Z_{ij} = 0 \cup Z_{-ij}, \theta)$. The parameter γ can be set to 1/2 if there is no prior information about Z .

The efficient learning of the graph is essential to scaling BO to higher-dimensions, and we devise this method to learn the tree structure efficiently. We use the log-likelihood in two ways, combining them to learn the structure in Algorithm 3.4. First, we use Gibbs Sampling to build a connected tree from an empty graph iteratively. We use Gibbs Sampling as we want to sample the possible connections to connect the disconnected tree structures into a connected tree. Once the dependency graph is a connected tree, we apply mutation in subsequent iterations. Thus, we grow the empty graph into a tree and then seek improvements via mutation.

Adding Edges Algorithm 3.5 samples from the marginal posteriors, while only adding edges that maintain that Z is still a tree. The Union-Find (UF)

Algorithm 3.5: GIBBS-SAMPLING at k -th iteration

```

1 Initialize UF data structure
2 for  $j = 1, \dots, N_{\text{dim}}$  do
3   for  $i = 1, \dots, j - 1$  do
4      $Z^{(k+1)} \leftarrow Z^{(k)}$ 
5     if cycle not formed by  $Z_{ij}^{(k+1)} = 1$  then
6       Sample  $Z_{ij}^{(\text{new})}$  from posterior
7        $Z^{(k+1)} \leftarrow Z_{ij}^{(\text{new})}$ 
8       Update UF via union operation
9       Add  $\mathcal{Z} \leftarrow \mathcal{Z} \cup \{Z^{(k+1)}\}$ 
10     $k \leftarrow k + 1$ 

```

data structure tracks a set of disjoint sets, providing the operations *union* and *find*. Both operations can be performed in (amortized) time $O(\alpha(N_{\text{dim}}))$ when implemented using weights with path compression (Cormen, Leiserson, Rivest, & Stein, 2009; Sedgewick & Wayne, 2016), where $\alpha(N_{\text{dim}})$ is the inverse Ackermann function. In short, both operations can be performed in nearly constant time (amortized). In our algorithm, we use UF to track the connected components of $\mathcal{G}$, represented by disjoint subsets of variables. We use the find operation to check for cycles. After adding the edge, we update UF by performing the union operation.

Mutation Algorithm 3.6 describes the mutation operation that we perform when the dependency graph $\mathcal{G}$ is a connected tree. We borrow the idea of mutation from genetic algorithms; the mutation operation can maintain tree structure diversity from one generation to another. The purpose of the mutation operation is to preserve and introduce diversity, wherein genetic algorithms, a mutation helps to avoid getting stuck in local maxima by making minor changes to the previous generation. In our context, the population is a new generation of trees in each iteration, and the fitness function is the log-likelihood. Using mutation, we can simultaneously avoid local maxima and efficiently maintain a tree structure.

We note that one could simply use the Gibbs sampling approach or the mutation approach separately rather than using the former followed by the latter, but experimentally, we found this combined approach to be more effective.

Algorithm 3.6: MUTATION at k -th iteration

```
1  $Z^{(k+1)} \leftarrow Z^{(k)}$ 
2  $i, j \leftarrow$  Sample random edge for which  $Z_{ij}^{(k+1)} = 1$ 
3 Remove edge:  $Z_{ij}^{(k+1)} = 0$ 
4  $i', j' \leftarrow$  Sample nodes from the disconnected sub-trees
5 Sample  $Z_{i'j'}^{(\text{new})}$  using posterior
6  $Z^{(k+1)} \leftarrow Z_{i'j'}^{(\text{new})}$ 
7 Augment the dataset:  $\mathcal{Z} \leftarrow \mathcal{Z} \cup \{Z^{(k+1)}\}$ 
8  $k \leftarrow k + 1$ 
```

3.4 Experiments

For each of our experiments¹, we compare our method, `Tree`, to several state-of-the-art black-box global optimization methods, particularly BO methods including `Graph No-Overlap` Kandasamy et al. (2015), `Graph Overlap` Rolland et al. (2018), `LineBO` (Kirschner et al., 2019), and `REMBO` (Ziyu Wang et al., 2013).

To avoid clutter, we avoid including every algorithm and baseline in our charts. Instead, we make an effort to compare against the *best* algorithm for the function at hand, leveraging on prior works’ experimental results to complete our discussion. For example, in cases where `LineBO` was already shown to outperform standard GPs and `REMBO` in (Kirschner et al., 2019), we omit these worse-performing methods. From their results, both `REMBO` and standard GP are found to be ineffective even when compared to a problem with few ‘important’ dimensions.

We run all additive methods using the zooming-based message passing algorithm, analogous to Algorithm 3.3. In addition, we compare to `Random`, which evaluates points at random. Where possible, we also compare our results to `Oracle`, which has access to the true dependency graph along with the true kernel parameters. The functions and datasets used in our analysis are detailed in Table A.2, and the competing algorithms with their hyperparameters are discussed in Table A.1, both located in the appendix.

¹Source code available at <https://github.com/eric-vader/HD-BO-Additive-Models>.

3.4.1 Experimental Setup

Whenever possible, we used identical parameters across all competing algorithms and functions. We ran each algorithm 25 times for every function with varied conditions.² We ran all experiments with $N_{\text{init}} = 10$ initial points and $N_{\text{iter}} = 1000$ total points. The same conditions are used across all algorithms to ensure a fair comparison.

Kernel

We adopt the widely-used Radial Basis Function (RBF) kernel, more specifically using a variant known as the RBF-ARD kernel (Murphy, 2012), which consists of a dimensional lengthscale ℓ_i for every dimension i . In addition, we decompose the scale parameter $\sigma^G = \sqrt{\sum_{i \in G} \sigma_i^2}$ into its dimensional components σ_i , so that we can learn the parameters tractably. Each low-dimensional kernel corresponds to a low-dimensional function with set of variables G :

$$\kappa_{\text{RBF}}^G(x, x') = \sigma^G \exp \left(-\frac{1}{2} \sum_{i \in G} \frac{(x_i - x'_i)^2}{\ell_i^2} \right). \quad (3.8)$$

In this manner, the kernel parameters $\Theta_{\mathcal{G}}$ are defined over the dimensional kernel parameters $\Theta = \{(\ell_i, \sigma_i)\}_{i=1}^{N_{\text{dim}}}$. We learn these parameters using the procedure described in Section 3.4.1, independent from the dependency graph. We adopt the established gradient-based approach to learning Θ , detailed below. We initialize the dimensional lengthscale and scale parameters as $\sigma_i = 0.5$, and $\ell_i = 0.1$ for all i . We set $\eta = 0.1$ in Equation 3.3 to account for noisy observations.

Learning the Kernel Parameters We consider selecting the kernel parameters by maximizing the marginal likelihood, and specifically adopt a well-established gradient approach. To implement gradient-based methods, we compute the partial derivatives of the log-likelihood with respect to the dimensional hyperparameters $\Theta = \{\theta_i : (\ell_i, \sigma_i)\}_{i=1}^{N_{\text{dim}}}$ as (Rasmussen & Williams, 2006):

$$\frac{\partial}{\partial \theta} \rho(Z, \theta) = \frac{1}{2} \text{Tr} \left[(\alpha \alpha^T - K^{-1}) \sum_{G \in \mathcal{G}} \left(\frac{\partial \kappa^G}{\partial \theta} \right) \right]. \quad (3.9)$$

²Conditions include initial points, instances of the objective function, and random seeds used by the algorithm.

The inner partial derivative $\partial\kappa^G/\partial\theta$ depends on the choice of kernel. The partial derivative with respect to each lengthscale ℓ_i can be computed directly, and for the kernels we consider, this is facilitated by the availability of closed-form kernel expressions. Following our definition of the scale parameter $\sigma^G = \sqrt{\sum_{i \in G} \sigma_i^2}$, the partial derivative with respect to each dimensional scale parameter σ_j is

$$\frac{\partial\kappa^G}{\partial\sigma_j} = \frac{\partial\kappa^G}{\partial\sigma^G} \left(\frac{\partial}{\partial\sigma_j} \sqrt{\sum_{i \in G} \sigma_i^2} \right) = \frac{\partial\kappa^G}{\partial\sigma^G} \times \frac{\sigma_j}{\sigma^G}. \quad (3.10)$$

Again, $\partial\kappa^G/\partial\sigma^G$ is computed directly from the closed-form kernel expression. Following common practice (GPy, since 2012; Pedregosa et al., 2011), we restrict the optimization of the dimensional parameters to $\ell_j \in [10^{-2}, 10^5]$ and $\sigma_j \in [\sqrt{0.1}, 10^5]$, ensuring that the kernel parameters are positive and avoid extreme values that can lead to numerical issues. In addition, we avoid making the lower bound on each σ_j too small, accounting for the equivalent effect in the graph learning of not creating an edge between two variables. There exist several gradient-based optimization techniques that can be used, e.g., Limited-Memory Broyden-Fletcher-Goldfarb-Shanno algorithm (L-BFGS), Truncated Newton algorithm (TNC), and others. We adopt TNC for all experiments except BO-based adversarial attacks, in which we use L-BFGS for consistency with a previous baseline. In addition, we impose limits on the maximum number of function evaluations in terms of the iteration index t according to the following:

$$m(t) = M_0 \exp \left(\frac{\log(M_{N_{iter}})}{N_{iter}} \times t \right). \quad (3.11)$$

We found that by increasing the maximum number of function evaluations over iterations, we can get a good trade-off between exploration and exploitation, avoiding overfitting of kernel parameters. In Equation 3.11, we set $M_0 = 1$ and $M_{N_{iter}} = 2D$.

Additive Models

All additive models start with an empty graph of the appropriate size for the given function. Concerning the learning of the dependency structure, we assume no prior knowledge ($\gamma = 0.5$). We sample the structure for $N_{\text{samples}} = 250$ times

every $C = 15$ iterations. After learning the structure, we choose the best kernel parameters using the gradient approach mentioned above. We set the trade-off parameter in UCB to be $\beta(t) = 0.5 \log(2t)$, as suggested in (Rolland et al., 2018). For *discrete* experiments, we discretize each dimension to 50 levels, with the maximum number of individual acquisition function evaluations capped at 1000. For *continuous* experiments, we let each level’s grid size be $N_{\text{gridsize}} = 4$ and the number of levels be $N_{\text{lvl}} = 4$ (see Figure 3.3) with no maximum evaluation limits.

3.4.2 Metrics

Following (Ziyu Wang et al., 2013), we plot the mean and $1/4$ standard deviation confidence intervals of the metrics over all 25 runs of the algorithm. For ease of analysis, each plot’s legend is ordered according to the curves’ final y -value.

Graph Learning Performance

We measure how close the estimate G is from its target graph G_{opt} by calculating

$$F_1\text{score}(G) = 2 \frac{\text{Precision}(G) \times \text{Recall}(G)}{\text{Precision}(G) + \text{Recall}(G)}, \quad (3.12)$$

by computing Precision and Recall over the set of edges:

$$\begin{aligned} \text{Precision}(G) &= \frac{|\text{Edges}(G) \cap \text{Edges}(G_{\text{opt}})|}{|\text{Edges}(G)|}, \\ \text{Recall}(G) &= \frac{|\text{Edges}(G) \cap \text{Edges}(G_{\text{opt}})|}{|\text{Edges}(G_{\text{opt}})|}, \end{aligned} \quad (3.13)$$

where $\text{Edges}(G)$ denoting the set of edges in graph G . We note that a larger $F_1\text{score}$ indicates better graph learning performance.

Optimization Performance

In accordance with the ultimate goal of BO, we compute the *best regret* to measure closeness to the best value f_{max} at iteration i :

$$R_t = f_{\text{max}} - f_i^*, \quad (3.14)$$

where f_i^* denotes the best $f(x)$ value sampled up to iteration i . For functions where $f_{\max}$ is unknown, we instead consider f_i^* , i.e., the *best value* found.

Discussion on Computation Time

In general, it is difficult to compare the amount of computational resources by various algorithms, as it is very much affected by many factors such as implementation, hardware, underlying GP backends, etc. However, we provide a brief discussion of the general trends observed. We generally found LineBO to be one of the faster approaches due to the use of 1D subroutines, though we also found its optimization performance to be limited in several cases. A fairly similar discussion applies to REMBO. On the other hand, the computational requirements of the *additive* methods are somewhat easier to compare in a fair manner, as we now discuss.

Cost Efficiency

For the additive methods, we compute the *Message Passing Cost* counting the number of individual acquisition function (ϕ^G) evaluations; see Equation 3.5. This metric is a proxy for the computational resources used in the optimization of the acquisition function. While it may not always correspond exactly to the total computation time, we expect that each message passing operation for *Tree* is *at least* as fast as in *Graph No-Overlap* and *Graph Overlap*. This is because *Tree* only works with functions containing only one or two variables, whereas the others may contain a larger number of variables.

3.4.3 Experiments with Additive GP Functions

We first compare *Tree* to other additive methods for functions drawn from a GP with additive structure. We focus our discussion on understanding the additive methods' scaling ability and performance. Afterwards, we compare *Tree* to other methods using various non-GP functions. On all synthetic experiments, we add Gaussian $\mathcal{N}(0, 0.15^2)$ noise to simulate noise that occurs in real-world applications.

Similar to (Rolland et al., 2018), we test our algorithm on synthetic data by

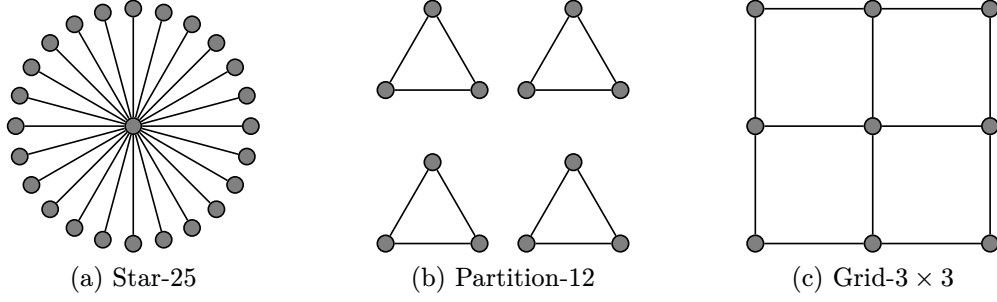


Figure 3.4. Synthetic Dependency Graphs Structures.

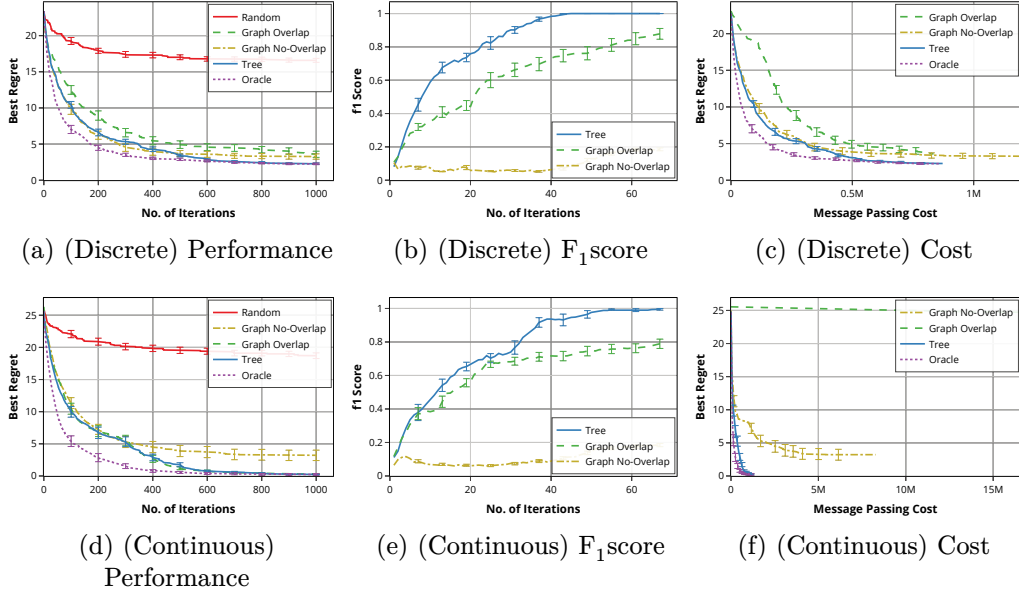


Figure 3.5. Star-25 Discrete and Continuous Experiments.

sampling functions from GPs with several different additive dependency structures. We use an RBF kernel with corresponding dimensional lengthscale and scale parameters set to $\sigma_i^{\text{opt}} = 1$ and $l_i^{\text{opt}} = 0.2$ for all i . We tested several dependency structures; three notable examples are illustrated in Figure 3.4, and a full list is given in the Table A.2.

In Figure 3.5a, it is unsurprising that *Tree* outperforms the other additive methods for Star-25. The dependency graph of Star-25 is indeed a tree, enabling our method to be effective in learning the dependency structure. From Figure 3.5b, by plotting F_1 score over iterations, we observe that it is efficient in learning the dependency structure. The dependency structure learned by *Tree* is closest to the ground-truth throughout the experiment, when compared with other additive models. This efficiency is also reflected in Figure 3.5c, where *Tree*

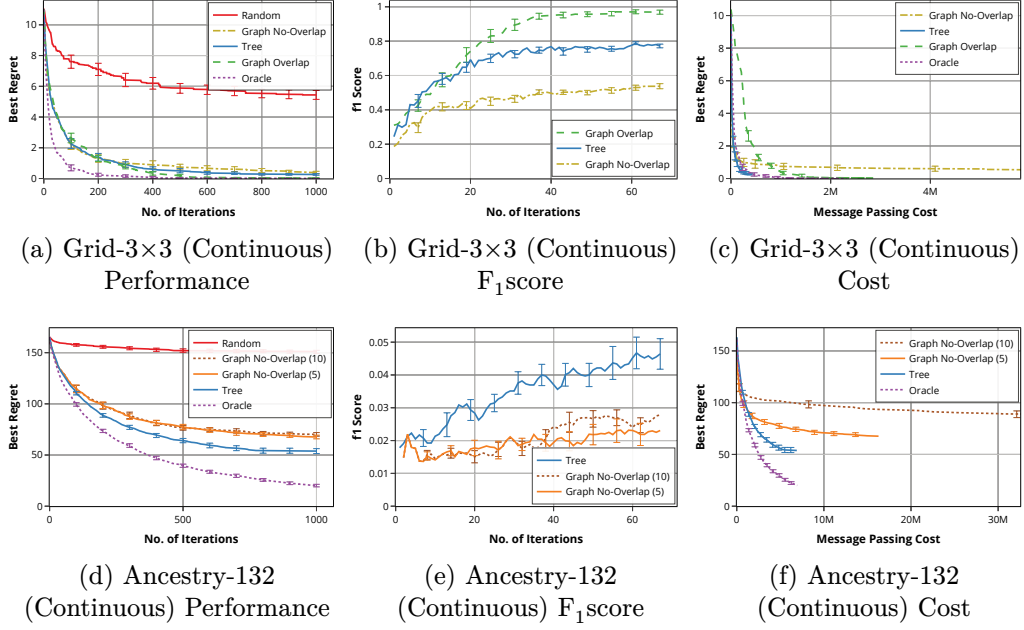


Figure 3.6. Summarized comparison of various additive methods across Grid-3 $\times$ 3 and Ancestry-132 functions.

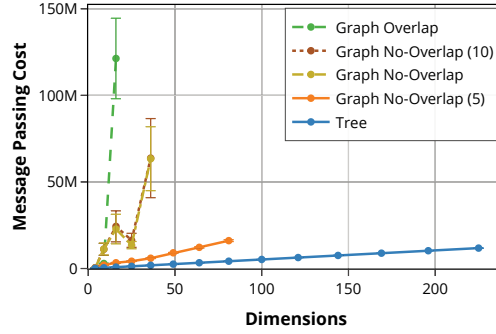


Figure 3.7. Scalability of Tree over dimensions.

achieves the best performance as a function of the message passing cost. We note that the reduction in cost becomes significantly higher in the case of continuous domains, achieving better performance return on cost than other additive methods.

Next, we turn to the case that the underlying graph is not a tree (*i.e.*, not realizable). Figures 3.6a, 3.6b corresponds to the Grid-3 $\times$ 3 structure, and we find that Graph Overlap performs the best in terms of learning the dependency structure. This is because, for the grid graph model, only Graph Overlap’s underlying structural assumptions are correct. Both Tree and Graph No-Overlap face difficulty learning the graph accurately, albeit worse for Graph No-Overlap. We illustrate the learned Tree structure and the associated learning process in

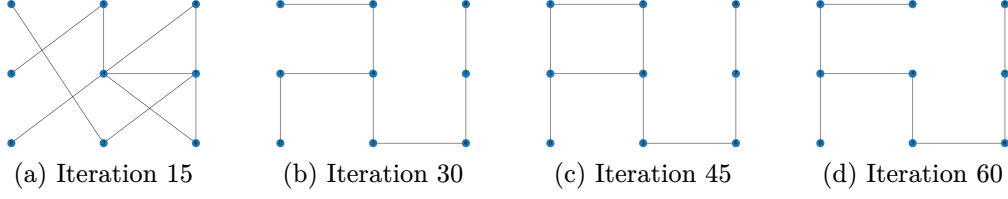


Figure 3.8. **Tree**’s learning progress over several iterations; **Tree** closely approximates the underlying Grid- 3×3 dependency structure (See Figure 3.4c). Note that some of the edges are overlapping (disconnected) with vertices in 3.8a, 3.8c.

Figure 3.8. Though not realizable, **Tree** manages to learn the closest possible dependency structure (4 errant non-connections), approximating the interaction between variables. The learned **Tree** structure can be seen as an approximation to the underlying structure, where only the most important connections are kept. Despite this, **Tree** still remains competitive in terms of optimization performance despite poorer graph learning. That is, when **Tree** makes errant connections (or errant non-connections) in the dependency graph, the performance does not degrade significantly, and the algorithm can tweak other parameters (e.g., σ_i and l_i) to minimize the effect of any errant connections.

In the case where the underlying dependency structure is realizable, **Tree** is almost always able to recover the correct Star-10 and Star-25 graphs within our experiment. We see in Figures 3.5d, 3.5e, both the performance and graph learning performance of **Tree** is expectedly good in the realizable dependency structure Star-25. We further illustrate this process in Figure 3.9, showing the progression of **Tree** algorithm successfully learning the ground truth dependency structure of the underlying Star-25. **Tree** starts by exploring many different connections and subsequently discovering the Star-25 structure over the next few iterations. The learning performance of **Tree** is particularly remarkable in the realizable case, indicating its effectiveness in accurately (and completely) recovering the dependency structure.

From Figure 3.6a, despite all additive algorithms being mutually competitive in terms of regret, both **Graph No-Overlap** and **Graph Overlap** needed more acquisition function evaluations to achieve the same performance as **Tree** (more than triple for **Graph No-Overlap**); see Figure 3.6c. In this instance, **Graph No-Overlap**’s pairwise disjoint assumption not only results in worse graph learning,

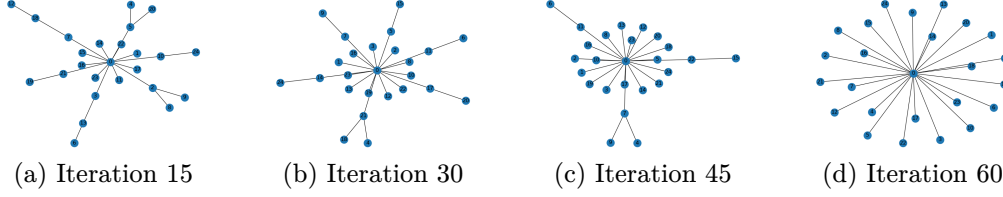


Figure 3.9. Tree’s learning progress over several iterations; Tree successfully recovers the underlying Star-25 dependency structure (See Figure 3.4a).

but also worse cost efficiency. Next, we compare Graph No-Overlap and Tree using an Ancestry-132 dependency structure (132D)³. We found that Graph Overlap was unable to complete such high-dimensional experiments in a reasonable time. For Graph No-Overlap to work efficiently, we limited the maximum clique size, consider limits of both 5 and 10, represented by Graph No-Overlap (5) and Graph No-Overlap (10) respectively. In the absence of such a restriction, Graph No-Overlap would face difficulty in higher dimensions, as discussed below. In Figures 3.6d, 3.6f, we find Tree performing best in high-dimensions, and being the most cost efficient.

Scalability

Here, we test Tree’s scalability to higher dimensions up to 225D, focusing on studying how the total cumulative message passing cost incurred scales as dimension increases. We used the same setup and parameters as Section 3.4.3, across additive grid structures of varying sizes – Grid- $i \times i$ for $i \in [2, 15]$. We again include Graph No-Overlap (5) and Graph No-Overlap (10) for this experiment.

From Figure 3.7, we can see that the amount of cost needed for Graph Overlap and Graph No-Overlap quickly increases as the dimensionality increases. In fact, we were unable to complete the experiment for larger grids in a reasonable amount of time. Recalling that Graph Overlap runs in time exponential in the size of the maximum clique of the triangulated dependency graph (Rolland et al., 2018), we note that even if that clique size does not grow large for the *true* graph, it may still tend to increase for the *estimated* graph. Similarly, Graph No-Overlap may be slow due to the consideration of large cliques, unless the

³See Section A.3 for more details on Ancestry-132.

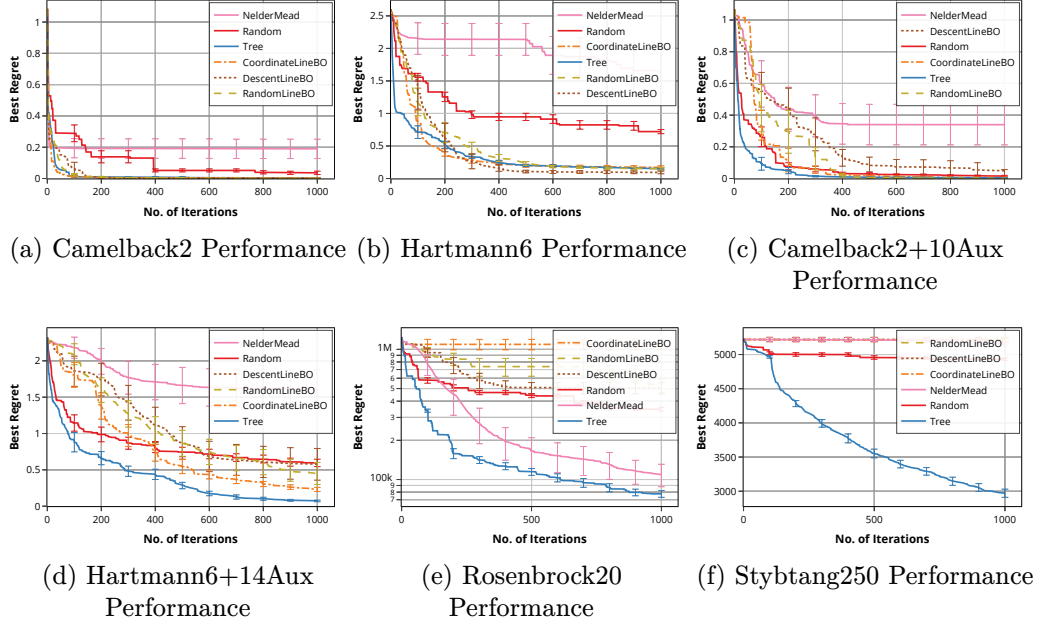


Figure 3.10. Results for non-GP synthetic functions.

clique size is explicitly limited. Even after imposing the limits, we found that **Tree** still incurs the lowest cost when compared with both **Graph No-Overlap (5)** and **Graph No-Overlap (10)**. This is because, for tree structures, the message passing cost is quadratic in the number of discretization levels of a single dimension. Hence, the message passing cost required for **Graph Overlap** and **Graph No-Overlap** would grow exponentially to become computationally intractable as dimensionality increases.

3.4.4 Experiments with Non-GP Functions

Non-GP Synthetic Functions

Here we test our algorithm against commonly used BO synthetic function benchmarks (Kirschner et al., 2019; Oh et al., 2018), including **Hartmann6 (6D)** and **Stybtang250 (250D)**. We also tested **Tree** on benchmarks with invariant subspace. Following the setup in (Kirschner et al., 2019), **Hartmann6+14Aux (20D)** was obtained by augmenting the synthetic functions with 14 auxiliary dimensions. In Figure 3.10d, we see that the regret of **Tree** reduces rapidly compared to other methods, with variants of **LineBO** catching up in later iterations. In Figure 3.10f, we see that **Tree** again manages to scale well in higher-dimensional synthetic

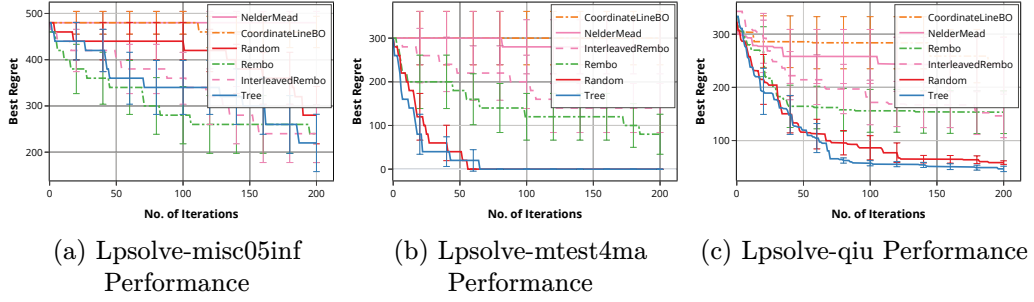


Figure 3.11. Results for Lpsolve hyperparameter tuning.

functions. *Tree* is also competitive against LineBO variants across both lower and higher dimensional settings, even in cases with invariant subspace. From Figures 3.10a, 3.10f, we observe that *Tree* consistently performs well across these synthetic functions. In contrast, the LineBO algorithms perform better on certain functions than others, and can face certain limitations. For instance, from Figure 3.10e, we see that LineBO algorithms can incur higher regret to perform when the range of the function is large, which applies to both *Rosenbrock20* and *Stybtang250*.

Linear Programming Solver

We consider tuning the parameters of *lpsolve*, an open-source Mixed Integer Linear Programming (MIP) solver (Berkelaar, Eikland, & Notebaert, 2004). The parameters within each algorithm typically have some relationship with each other; tweaking a parameter can potentially affect another. We consider a similar configuration problem as defined by (Hutter, Hoos, & Leyton-Brown, 2010; Ziyu Wang et al., 2013), focusing on tuning *lpsolve*’s 74 parameters - 59 binary, 10 ordinal and 5 categorical. Our objective is to find the set of parameters of *lpsolve* that minimize the *optimality gap* it can achieve with a time limit of five seconds for the MIP encodings ‘misc05inf’, ‘mtest4ma’ and ‘qiu’ found in the benchmark MIPLIB (Gleixner et al., 2021).

From Figure 3.11, we observe that REMBO is competitive in performance for optimizing the linear programming solver, as parameter optimization problems often have low effective dimensionality (Hutter, Hoos, & Leyton-Brown, 2014; Ziyu Wang et al., 2013). REMBO and its interleaved variant are effective in some

cases (*e.g.* Figure 3.11a), but are slow to converge in others (*e.g.* Figures 3.11a, 3.11b). In contrast, **Tree** performs well across all three datasets. Despite being based on a very different notion of structure, **Tree** attains better performance than REMBO in this example, with both clearly outperforming **Random**.

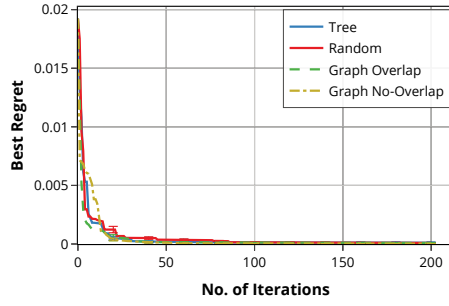
NAS Benchmarks

We run experiments on the NAS-Bench-101 (NAS) dataset (Klein & Hutter, 2019; Ying et al., 2019). This dataset serves as a convenient hyperparameter benchmark that allows us to quickly run hyperparameter optimization over a fully connected neural network. Traditionally, when we run hyperparameter optimization over a neural network, it would be slow, as training and testing a neural network would take substantial time. This benchmark pre-computes the results for all possible 62208 configurations (from 9 hyperparameters) defined in (Klein & Hutter, 2019; Ying et al., 2019). Consequently, we have access to $f_{\max}$. We ran the experiments as described in (Klein & Hutter, 2019), across all four NAS benchmarks (see Table A.2).

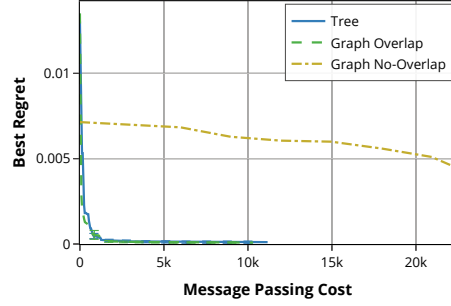
The results from the NAS benchmarks suggest that most functions in this dataset have little additive structure. We can observe this from the lack of significant difference between the graph-based algorithms across the NAS-naval, NAS-parkinsons and NAS-slice as shown in Figure 3.12. There is a small performance edge that **Tree** achieves over the other algorithms in NAS-protein shown in Figure 3.12c. The performance edge is gained efficiently, requiring lower cost when compared with the rest of the algorithms. Overall, these experiments suggest that **Tree** does not break down in datasets with little or no additive structure while retaining efficiency.

3.4.5 BO-Based Adversarial Attack Experiments

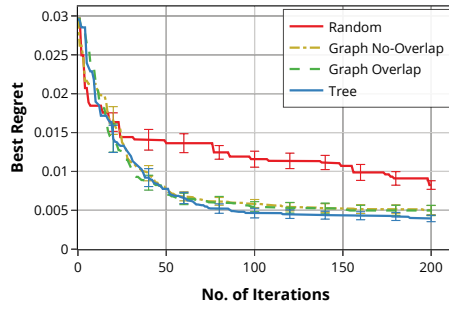
BO has recently been applied to *targeted* adversarial black-box attacks on neural networks. In particular, (Ru et al., 2020) proposed an approach based on the non-overlapping additive model from (Kandasamy et al., 2015). Their method achieves comparable success rates with fewer queries than certain state-of-the-art black-box attack methods for attacks on Convolutional Neural Network (CNN)



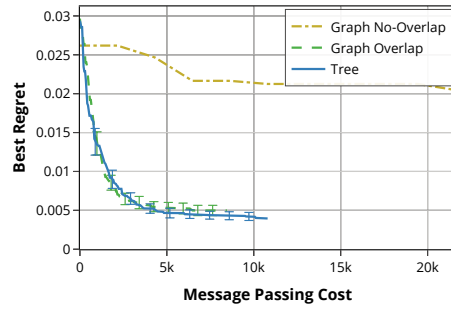
(a) NAS-naval Performance



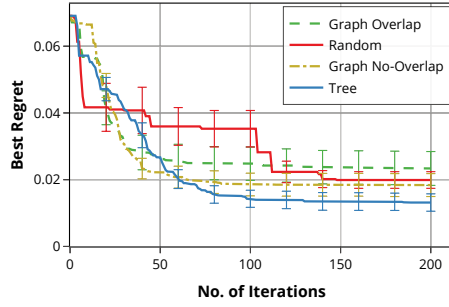
(b) NAS-naval Cost



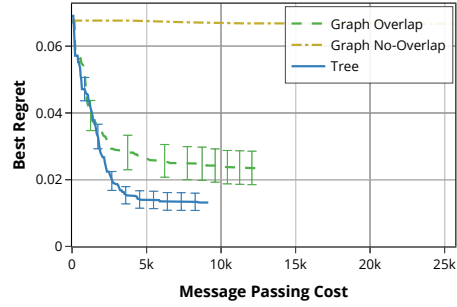
(c) NAS-parkinsons Performance



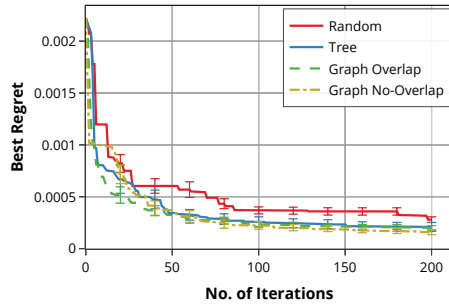
(d) NAS-parkinsons Cost



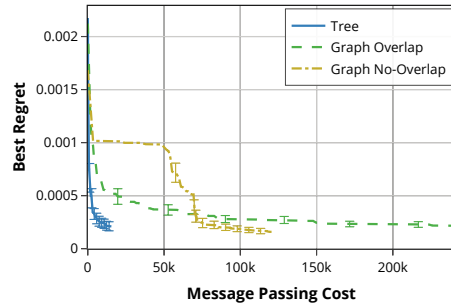
(e) NAS-protein Performance



(f) NAS-protein Cost



(g) NAS-slice Performance



(h) NAS-slice Cost

Figure 3.12. Results for NAS-Bench-101 datasets.

models. We adopt the same black-box function as (Ru et al., 2020), in which the input is an image perturbation and the output is the classification accuracy.

In this experiment, we adapt *Tree* in an analogous manner to (Ru et al., 2020) in order to perform adversarial black-box attacks on the CNN models. To maintain a fair comparison, we adopted the same experimental choices and parameters as (Ru et al., 2020) wherever possible, in particular using the Matérn- $5/2$ kernel, applying L-BFGS for kernel parameter optimization, performing graph and parameter learning according to the same regularly-spaced intervals (length 40 and 5 respectively), and using $N_{\text{init}} = 50$ initial random points.

The CNN models that we attack are also the same as (Alzantot et al., 2019; Ru et al., 2020; C.-C. Tu et al., 2019). The image classifiers for the dataset MNIST (99.5% Accuracy) is taken from the author’s repository. Using the same dimensionality reduction technique as (Ru et al., 2020; C.-C. Tu et al., 2019), we perform the search on a reduced image dimension of $14 \times 14 \times 1$ for MNIST (196D). For the attack, we randomly select 50 correctly-classified images; then, for each image, we perform the attack for all target labels except the true label. We record the Attack Success Rate (i.e., the proportion of times the attack succeeds in making the classifier output the target label, averaged across 450 runs) against the number of iterations. We used process time from Python’s standard library, recording the total of both system and user process-wide CPU time⁴. Considering that we executed the experiments over a heterogeneous cluster (randomly distributed) and other experimental considerations, this metric allows us to glean a rough measure of the algorithm’s efficiency.

We compare *Tree* to the implementations available in (Ru et al., 2020)’s repository. We note that the additive BO implementation (*ADDGP-BO*) is different from our implementation of *Graph No-Overlap*. In particular, they select the graph according the highest likelihood among a number of randomly-generated graphs, whereas we adopt the Gibbs sampling approach of (Rolland et al., 2018; Zi Wang et al., 2017).

We focus on comparing against *ADDGP-BO*, as it was the best overall

⁴https://docs.python.org/3/library/time.html#time.process_time

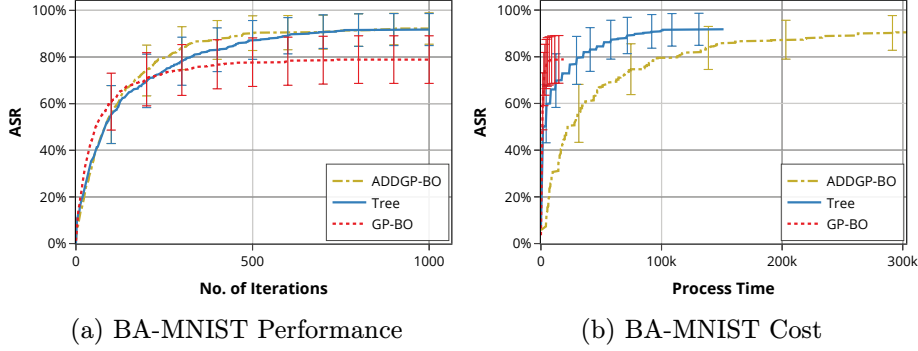


Figure 3.13. Results for BO-based adversarial attacks.

method reported in (Ru et al., 2020). The results are presented⁵ in Figure 3.13. We see in Figure 3.13a that *Tree* is able to attack MNIST (196D) CNN model with a comparable success rate as *ADDGP-BO*, but with significantly reduced process time. The process time is lowest for *GP-BO* (an GP-UCB based baseline without additive structure, available in the repository of (Ru et al., 2020)), but at the cost of a significantly worse success rate.

3.5 Conclusion

For the problem of GP optimization with generalized additive models, we traded off expressivity for computational efficiency and ease of model learning by reducing the model complexity, constraining the dependency graph to tree structures. Our method efficiently learns the additive tree structure using Gibbs Sampling and edge mutation. This approach is suitable for resource-limited settings, aligning with the primary motivation of BO. Besides, we presented a zooming-based message passing approach that can benefit BO with generalized additive models in continuous domains, with or without tree structures. We demonstrated that *Tree* is competitive on both synthetic functions and real datasets, and that the computation can be significantly reduced compared to more complex graph structures, without sacrificing the optimization performance.

⁵Attacks that took at least 3 times longer than 99% of the runs were marked as unsuccessful.

Chapter 4

Adversarial Attacks on Bayesian Optimization

Considering (Bogunovic et al., 2020), a problem instance as per Equation 2.5, where the observations are adversarially corrupted up to some maximum budget, *i.e.*, adversarial perturbations added to uncorrupted samples. In Section 2.2, we see that there are many methods developed for *defending* against the specific uncertainty model. There are minimal work studying the problem from an attacker’s perspective, considering the ‘worst-case’ – adversarial perturbation. Our work delves into the challenges posed by imposing demanding requirements, exploring questions like the difficulty of causing standard algorithms to fail, the effectiveness of different types of attacks, and the possibility of achieving optimal non-robust regret while maintaining robustness.

4.1 Contributions

Our main contributions are as follows:

1. We theoretically characterize conditions under which an adversarial attack can succeed against GP-UCB or elimination-based algorithm even with an attack budget far smaller than the time horizon, both in the cases of the function being known (white-box attack) and unknown (black-box attack) to the attacker.

2. We present various attacks inspired by our analysis:

- a. knowing f : **Subtraction Attack** (two variants), **Clipping Attack**.
- b. not knowing f : **Aggressive Subtraction Attack** (two variants).

We demonstrate the effectiveness of these attacks via experiments on a diverse range of objective functions.

More broadly, we believe that our work fills an important gap in the literature by moving beyond finite domains and/or linear functions, and giving the first study of robust GP bandits from the attacker’s perspective.

4.2 Setup

We consider the setup proposed in (Bogunovic et al., 2020), described as follows. The player seeks to maximize an unknown function $f(\mathbf{x})$ over $\mathbf{x} \in D$, and we model the smoothness of f by assuming that it has Reproducing kernel Hilbert space (RKHS) norm at most B according to some kernel k , *i.e.*, $f \in \mathcal{F}_k(B)$ where $\mathcal{F}_k(B) = \{f : \|f\|_k \leq B\}$. We focus primarily on the widely-adopted Matérn kernel, defined as

$$k_{\text{Mat}}(x, x') = \frac{2^{1-\nu}}{\Gamma(\nu)} \left(\frac{\sqrt{2\nu}\|x - x'\|}{l} \right)^\nu J_\nu \left(\frac{\sqrt{2\nu}\|x - x'\|}{l} \right), \quad (4.1)$$

where $l > 0$ denotes the lengthscale, $\nu > 0$ is the smoothness parameter, and J_ν denotes the modified Bessel function. This kernel provides useful properties that we can exploit in our theoretical analysis, and is also one of the most *widely-adopted* kernels in practice. In addition, it closely resembles the RBF kernel in the case that ν is large (Rasmussen & Williams, 2006).

In the uncorrupted setting (*e.g.*, see (Chowdhury & Gopalan, 2017a; Srinivas et al., 2010)), the player samples $\mathbf{x}_t$ and observes $y_t = f(\mathbf{x}_t) + z_t$, where $z_t \sim \mathcal{N}(0, \sigma^2)$ is random noise. We also note that regret at time t is $r_t = f(\mathbf{x}^*) - f(\mathbf{x}_t)$ (where $\mathbf{x}^*$ is any maximizer of f), and the cumulative regret up to time T is $R_T = \sum_{t=1}^T r_t$. In the presence of adversarial noise, we consider corrupted observations taking the form

$$y_t = f(\mathbf{x}_t) + c_t + z_t, \quad (4.2)$$

where $z_t \sim \mathcal{N}(0, \sigma^2)$, and c_t is adversarial noise injected at time t . We consider c_t as being chosen by an *adversary* or *attacker*. In order to make the problem meaningful, the adversary’s power should be limited, so we constrain

$$\sum_{t=1}^n |c_t| \leq C \quad (4.3)$$

for some total corruption budget C . Following (Bogunovic et al., 2020), we adopt the same definition of regret as the uncorrupted setting: $R_T = \sum_{t=1}^T r_t$, where $r_t = f(\mathbf{x}^*) - f(\mathbf{x}_t)$, and where $\mathbf{x}^*$ is any maximizer of f .

We note that c_t could also potentially be considered as part of the objective when defining regret; the two notions are closely related, and differ by at most $O(C)$ (Bogunovic et al., 2020; Lykouris et al., 2018). For all of our results, this connection ensures that a successful attack (*i.e.*, linear regret in T) with respect to the above notion implies the same with respect to the alternative regret notion.

4.2.1 Knowledge Available to the Adversary

Naturally, the ability to attack or defend in the preceding setup varies; it significantly depends on what is assumed to be known to the adversary. For instance, the adversary may or may not know f , know $\mathbf{x}_t$, know which algorithm the player is using, and so on. In addition, as noted in the literature on robust bandit problems (Bogunovic, Losalka, Krause, & Scarlett, 2021; Lykouris et al., 2018), one may consider the case that the player can randomize $\mathbf{x}_t$, and the adversary knows the distribution but not the specific choice.¹

In this paper, our focus is on attacking widely-adopted deterministic algorithms such as GP-UCB (Srinivas et al., 2010), so we do not consider such randomization. In this case, knowing the “distribution of $\mathbf{x}_t$ ” becomes equivalent to knowing $\mathbf{x}_t$, and we assume that such knowledge is available throughout the paper. We consider both the cases of f being known and unknown to the attacker.

¹In such a case, c_t in Equation 4.3 is typically replaced by $\max_{\mathbf{x}} |c_t(\mathbf{x})|$, where $c_t(\cdot)$ is the adversary’s corruption function at time t .

4.2.2 Targeted vs. Untargeted Attacks

At this stage, we find it useful to distinguish between two types of attack. In an *untargeted attack*, the adversary’s sole aim is to make the player’s cumulative regret as high as possible (e.g., $R_T = \Omega(T)$). In contrast, in a *targeted attack*, the adversary’s goal is to make the player choose actions in a particular region $\mathcal{R}_{\text{target}} \subseteq D$. This generalizes the finite-arm notion of seeking to make the player pull a particular target arm (Jun et al., 2018).

Of course, a targeted attack can also be used to ensure a large regret: If $\mathcal{R}_{\text{target}}$ satisfies the property that every $\mathbf{x} \in \mathcal{R}_{\text{target}}$ has $r(\mathbf{x}) = \Omega(1)$, then any attack that forces $\Omega(T)$ selections in $\mathcal{R}_{\text{target}}$ also ensures $R_T = \Omega(T)$.

More generally, to assess the performance of a targeted attack, we define the quantity after T rounds

$$N_T^{\text{target}} = \sum_{t=1}^T \{\mathbf{x}_t \in \mathcal{R}_{\text{target}}\}, \quad (4.4)$$

counting the number of arm pulls within the target region.

4.3 Theoretical Study

We introduce some attacks and gives conditions under which they provably succeed even when the budget C is small compared to the time horizon T . These results are not only of interest in their own right, but will also motivate several of our other attacks (without theory) in Section 4.4.

Motivated by successful attacks on bandit problems (Garcelon et al., 2020; Jun et al., 2018), we adopt the idea of perturbing the function *outside* the $\mathcal{R}_{\text{target}}$ in a manner such that the perturbed function’s maximizer is in the $\mathcal{R}_{\text{target}}$. Then, a (non-robust) optimization algorithm will steer towards $\mathcal{R}_{\text{target}}$ and stay there, with any points sampled in $\mathcal{R}_{\text{target}}$ remaining unperturbed.

This idea is somewhat trickier to implement in the GP bandit setting than the finite-arm or linear bandit setting, and the details are given below.

4.3.1 Optimization Algorithms

Our attack methods can be applied to any GP bandit algorithm, and Theorem 4.1 below states general conditions under which the attack succeeds. As two specific examples, we will consider the following widely-used algorithms that are representative of broader techniques in the literature:

- *GP-UCB* (Srinivas et al., 2010): The t -th point is selected according to

$$\mathbf{x}_t = \arg \max_{\mathbf{x} \in D} \mu_{t-1}(\mathbf{x}) + \beta_t^{1/2} \sigma_{t-1}(\mathbf{x}), \quad (4.5)$$

for some suitably-chosen exploration parameter β_t , where $\mu_{t-1}(\cdot)$ and $\sigma_{t-1}(\cdot)$ are the posterior mean and standard deviation after sampling $t - 1$ points (Rasmussen & Williams, 2006).

- *MaxVar + Elimination* (Contal, Buffoni, Robicquet, & Vayatis, 2013): At each time, define the set of *potential maximizers* M_t to contain all points whose UCB (*i.e.*, $\mu_{t-1}(\mathbf{x}) + \beta_t^{1/2} \sigma_{t-1}(\mathbf{x})$) is at least as high as the highest LCB (*i.e.*, $\mu_{t-1}(\mathbf{x}) - \beta_t^{1/2} \sigma_{t-1}(\mathbf{x})$). Then, select the point in M_t with the highest posterior variance.

For both of these algorithms, the exploration parameter β_t is set to the value used in theoretical studies (e.g., (Chowdhury & Gopalan, 2017a)):

$$\beta_t^{1/2} = B + \sigma \lambda^{-1/2} \sqrt{2(\gamma_{t-1} + \ln(1/\delta))}. \quad (4.6)$$

where λ is a free parameter (e.g., $\lambda = 1$), δ is the target error probability, and γ_t is the maximum information gain at time t (Srinivas et al., 2010). The latter quantity scales as $\gamma_t = \tilde{O}(T^{\frac{d}{2\nu+d}})$ for the Matérn kernel (Vakili, Khezeli, & Picheny, 2021).

The reason for focusing on these algorithms is that they have well-known guarantees on the regret in the uncorrupted setting, and such guarantees turn out to be a key ingredient in establishing the success of attacks in the corrupted setting. While these algorithms are far from exhaustive, they serve as representative examples of general non-robust algorithms.

4.3.2 Conditions for a Successful Attack

Our main theoretical result motivating our attacks is stated as follows. This result applies to *any* algorithm that has non-robust guarantees of not playing too many strictly sub-optimal actions. The asymptotic notation $o(1)$ is defined with respect to the limit $T \rightarrow \infty$.

Theorem 4.1. *Consider an adversary that performs an attack shifting the original function $f \in \mathcal{F}_k(B)$ to another function $\tilde{f}$ (i.e., set $c_t = \tilde{f}(\mathbf{x}_t) - f(\mathbf{x}_t)$ at time t as long as the corruption budget permits it). Suppose that the following properties hold for some $\Delta > 0$ and $B_0 > 0$:*

- (i) *For any $\mathbf{x}$ that is Δ -optimal for $\tilde{f}$, it holds that both $\mathbf{x} \in \mathcal{R}_{\text{target}}$ and $\tilde{f}(\mathbf{x}) = f(\mathbf{x})$;*
- (ii) *For all $\mathbf{x} \in D$, it holds that $|f(\mathbf{x}) - \tilde{f}(\mathbf{x})| \leq B_0$;*
- (iii) *It holds that $\|\tilde{f}\|_k \leq B$ (i.e., $\tilde{f} \in \mathcal{F}_k(B)$).*

In addition, suppose that in the absence of an adversary, the optimization algorithm guarantees, for any $f \in \mathcal{F}_k$ and with probability at least $1 - \delta$, that at most N_0 played actions are Δ -sub-optimal, for some N_0 depending on (T, Δ, δ) .

Then, in the presence of the adversary, with probability at least $1 - \delta$, the attack succeeds in forcing $T(1 - o(1))$ actions to be played from $\mathcal{R}_{\text{target}}$, while using an attack budget C of at most $B_0 N_0$.

Proof. By the first assumed property, whenever the algorithm plays actions that are Δ -optimal *with respect to $\tilde{f}$* , it holds that $c_t = 0$, and no budget is spent. The third property allows us to bound the number of actions failing to satisfy such a property by N_0 , where this bound holds with probability at least $1 - \delta$ by the assumption on the algorithm. The second property ensures that each action uses an amount of budget satisfying $|c_t| \leq B_0$, and the result follows. $\square$

Combining this result with recently-established results from (Cai, Gomes, & Scarlett, 2021), we obtain the following corollary for the GP-UCB and elimination algorithms.

Corollary 4.1. *Under the setup of Theorem 4.1, with probability at least $1 - \delta$ (for δ used in the algorithms), the attack succeeds in forcing $T(1-o(1))$ actions to be played from $\mathcal{R}_{\text{target}}$ against GP-UCB (respectively, the elimination algorithm) using an attack budget C of at most $B_0 N_{\max}(\Delta, T)$ (respectively, $B_0 N'_{\max}(\Delta)$), where*

$$N_{\max}(\Delta, T) = \max \left\{ N : N \leq \frac{C_1 \gamma_N \beta_T}{\Delta^2} \right\}, \quad (4.7)$$

$$N'_{\max}(\Delta) = \max \left\{ N : N \leq \frac{4C_1 \gamma_N \beta_N}{\Delta^2} \right\}, \quad (4.8)$$

with $C_1 = \frac{8\lambda^{-1}}{\log(1+\lambda^{-1})}$.

Proof. This result follows readily by combining Theorem 4.1 with existing bounds on the number of Δ -sub-optimal actions (i.e., having $f(\mathbf{x}) < \max_{\mathbf{x}'} f(\mathbf{x}') - \Delta$) from (Cai et al., 2021); in the standard (non-adversarial) GP bandit setting, we have the following:

- (i) For GP-UCB, the number of actions chosen in T rounds that are Δ -sub-optimal is at most $N_{\max}$ with probability at least $1 - \delta$.
- (ii) For the elimination algorithm, this can further be reduced to $N'_{\max}$.

□

While the preceding results are phrased as though the adversary had perfect knowledge of f , we will highlight further special cases below where this need not be the case.

As discussed in (Cai & Scarlett, 2021), as long as the smoothness parameter ν is not too small, $N_{\max}(\Delta, T)$ scales slowly with T (e.g., $\sqrt{T}$ or $T^{0.1}$), and $N'_{\max}(\Delta)$ has no dependence on T at all. Hence, for such sufficiently smooth functions with the Matérn kernel, Theorem 4.1 gives conditions under which an attack succeeds with a far smaller budget than the time horizon. Note that as ν grows large for the Matérn kernel, $N_{\max}$ behaves as $\frac{T^{\epsilon'}}{\Delta^{2+\epsilon}}$ for some small ϵ and ϵ' , and $N'_{\max}$ behaves as $\frac{1}{\Delta^{2+\epsilon}}$ for some small ϵ .

Moreover, in view of Theorem 4.1, the stronger the guarantee on the number of Δ -sub-optimal actions in the uncorrupted setting, the smaller the attack budget will be in a counterpart of Corollary 4.1. For instance, if we were to attack

the recent *batched* elimination algorithm of (Z. Li & Scarlett, 2022), the required budget would be sub-linear in T for all $\nu > \frac{1}{2}$, regardless of d . However, we prefer to focus on the above non-batched algorithm, since it is simpler and more standard in the literature.

For completeness, we mention a recent study (Whitehouse, Wu, & Ramdas, 2023); the authors showed that using tightened confidence bounds, GP-UCB achieves nearly optimal regret rates, including sub-linear regret for the Matérn kernel. Applying these to our results is a potential avenue for future work.

Sufficiency vs. Necessity

We note that Theorem 4.1 only states sufficient conditions for a successful attack; analogous necessary conditions are difficult, and appear to be absent even in simpler settings studied previously (e.g., linear bandits). We note that condition (ii) is not always necessary, because the two functions could differ drastically at some far-away sub-optimal point that is never queried. Condition (iii) is also not always necessary, because **Clipping** succeeds without it. We believe that condition (i) is “closest to necessary”, but even so, although the attacker’s budget would get exhausted without it, it could be the case that the “damage has already been done”, and the attack still succeeds.

The Role of RKHS Norm

As noted by a reviewer, the role of RKHS on the attack success is subtle. On the one hand, a higher RKHS norm could be associated with more local optima that the attacker can exploit. Furthermore, it could additionally help the attacker if they can perturb f using a constant fraction of the total budget B . On the other hand, if one asks which functions are the hardest to attack, then a higher B implies more flexibility in coming with such a function. In general, we expect that there is no direct correspondence between B and the attack difficulty; in particular, among functions with a large RKHS norm, some are easier to attack, and some are harder.

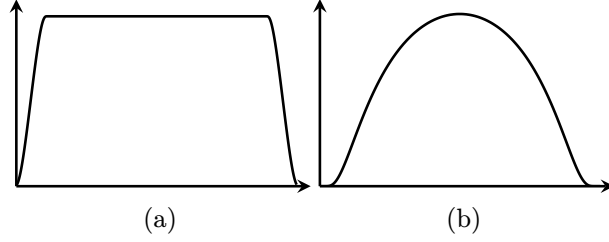


Figure 4.1. Illustration of the various function constructions – bump function (left) and convolved bump function (right).

4.3.3 Applications to Specific Scenarios

Before we discuss two natural approaches to ensuring the conditions in Theorem 4.1, we describe some useful function constructions from the existing literature (see Figure 4.1):

- *Bump function in spatial domain:* It is known from (Bull, 2011; Cai & Scarlett, 2021) that the function

$$h(\mathbf{x}) = \exp\left(\frac{-1}{1 - \|\mathbf{x}\|^2}\right) 1\{\|\mathbf{x}\| \leq 1\}, \quad (4.9)$$

has bounded RKHS norm under the Matérn kernel, and is non-negative with bounded support and maximum at $\mathbf{x} = 0$. Moreover, we can form a scaled version of h with height ϵ and support of radius w , and the resulting RKHS norm scales as $O(\frac{\epsilon}{w^\nu})$ (*i.e.*, it can be made at most B with $w = \Theta((\frac{\epsilon}{B})^{1/\nu})$).

- *Convolved bump function:* Following (Cai & Scarlett, 2021), we consider taking the width- w height-1 bump function from the previous dot point, and convolving it with a function that equals 1 for $\mathbf{x} \in S$, and 0 for $\mathbf{x} \notin S$, where S is some compact subset of $\mathbb{R}^d$. Note that in (Cai & Scarlett, 2021, Lemma 6), S was ball-shaped, but the analysis extends to the general case. Then, after suitable scaling to make the new function’s maximum height equal to some value τ , we obtain a function $g(\mathbf{x})$ satisfying the following:

- If the ball of radius w around $\mathbf{x}$ is contained in S , then $g(\mathbf{x}) = \tau$;
- If the ball of radius w around $\mathbf{x}$ is completely outside S , then $g(\mathbf{x}) = 0$;
- In all cases in between these, $g(\mathbf{x}) \in (0, \tau)$.

Moreover, the RKHS norm satisfies $\|g\|_k \leq O(\frac{\tau \cdot \text{vol}(S)}{w^\nu})$ (Cai & Scarlett, 2021), so if $\text{vol}(S) = O(1)$ we can ensure an RKHS norm of at most B with a choice of the form $w = \Theta((\frac{\tau}{B})^{1/\nu})$.

We now discuss two general attack approaches, on which we will expand later in Section 4.4.

Approach 1

To guarantee that $\|\tilde{f}\|_k \leq B$, it is useful to adopt the decomposition

$$\|\tilde{f}\|_k \leq \|f\|_k + \|\tilde{f} - f\|_k, \quad (4.10)$$

which follows from the triangle inequality. Hence, if the “RKHS norm budget” B is not entirely used up by f (e.g., $\|f\|_k = \frac{B}{2}$), then we can maintain $\|\tilde{f}\|_k \leq B$ by setting

$$\tilde{f}(\mathbf{x}) = f(\mathbf{x}) - h(\mathbf{x}) \quad (4.11)$$

for some $h(\mathbf{x})$ with suitably bounded norm (e.g., $\|h\|_k \leq \frac{B}{2}$). The convolved bump function described above (which approximates a rectangular function) is useful for constructing h .

This approach is immediately applicable to the case that the interior of $\mathcal{R}_{\text{target}}$ contains a local maximum, e.g., see Figure 4.3. In this case, we can simply form bumps to “swallow” any higher maxima (or maxima within Δ of the target peak). As long as the convolved bump functions used for this purpose have a small enough height and transition width to maintain that $\|h\|_k \leq B - \|f\|_k$, the resulting function will satisfy the conditions of Theorem 4.1.

However, finding the precise locations of such bumps and setting their parameters may be difficult, or even entirely impossible if f is unknown, even if there is plenty of RKHS norm budget that can be utilized.

Approach 2

In view of the above discussion, we propose a more aggressive approach that seeks to push the function values downward equally (or approximately equally) for *all*

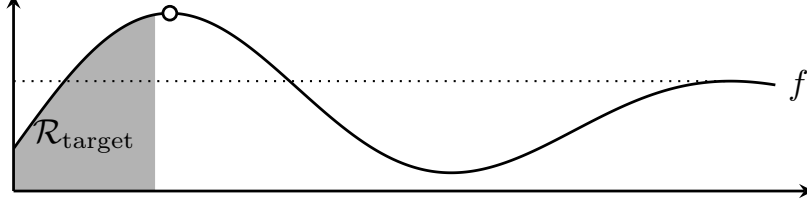


Figure 4.2. Difficult case where f is increasing in $\mathcal{R}_{\text{target}}$.

points outside $\mathcal{R}_{\text{target}}$. This can be done by taking a convolved bump function that covers the entire domain with some height $h_{\max}$ (i.e., $h(\mathbf{x}) = -h_{\max}$), but then “re-adding” a bump that covers $\mathcal{R}_{\text{target}}$. Depending on which is more convenient, the transition region could lie either inside or outside $\mathcal{R}_{\text{target}}$. See Figure 4.5 for an illustration.

Once again, the preceding attack can be applied while maintaining the conditions Theorem 4.1 in cases where $\mathcal{R}_{\text{target}}$ contains a local maximum, provided that the bumps used in its construction do not exceed the RKHS norm budget.

Example Difficult Case

To highlight the consideration of cases with local maxima in $\mathcal{R}_{\text{target}}$ above, consider the 1D (counter-)example in Figure 4.2. Here, there is no apparent way to construct $h(\mathbf{x})$ to satisfy the conditions of Theorem 4.1; the function continues to increase when leaving $\mathcal{R}_{\text{target}}$, and performing any *smooth* perturbation to the function to push those values downward would either keep the maximizer outside $\mathcal{R}_{\text{target}}$, or amount to having Δ -optimal points for $\tilde{f}$ such that $\tilde{f}(\mathbf{x}) \neq f(\mathbf{x})$. Despite this difficulty, we will see that our attacks can still be effective in such situations experimentally, albeit requiring a larger attack budget.

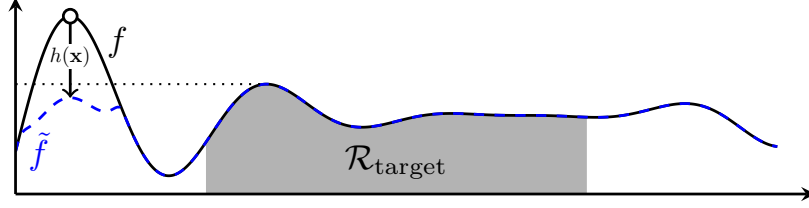
4.4 Attack Methods

We introduce our main proposed adversarial attacks on GP bandits. Based on Theorem 4.1, we adopt the idea of perturbing the function f to construct another function $\tilde{f}$ such that ideally the following properties hold for some set $\mathcal{R}_0 \subseteq \mathcal{R}_{\text{target}}$ (often taking the full set, $\mathcal{R}_0 = \mathcal{R}_{\text{target}}$):

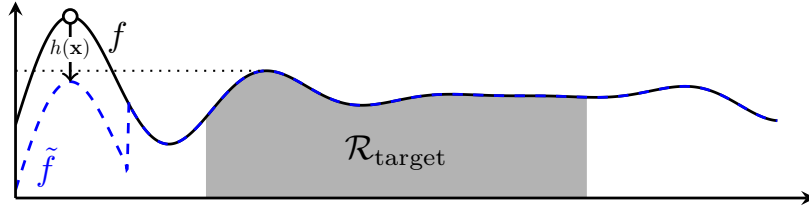
- (i) $\tilde{f}(\mathbf{x}) = f(\mathbf{x})$ for all $\mathbf{x} \in \mathcal{R}_0$;

	Subtraction		Clipping	Aggressive Subtraction	
	Rnd	Sq		w Trans.	w/o Trans.
Fit Theory	Yes	No	No	Yes	No
Knowledge	f		$(\tilde{\mathbf{x}}^*, f(\tilde{\mathbf{x}}^*))$	-	-
Practical	Low		High	Medium	High
Hyperparameter	$h(x)$		Δ	$h_{\max}$	

Table 4.1. Summary of our proposed adversarial attacks.



(a) Subtraction Rnd, with the bump function.



(b) Subtraction Sq, with the indicator function.

Figure 4.3. Subtraction Attack for known f with different $h(\mathbf{x})$.

- (ii) All maximizers of $\tilde{f}$ lie inside $\mathcal{R}_0$, and ideally all points outside $\mathcal{R}_{\text{target}}$ are sub-optimal by at some strictly positive value $\Delta > 0$;
- (iii) $\tilde{f}$ satisfies the RKHS norm constraint, *i.e.*, $\|\tilde{f}\|_k \leq B$.

To see why these properties are useful, note that property (iii) ensures that algorithms such as GP-UCB eventually choose most actions to near the optimum of $\tilde{f}$ (Chowdhury & Gopalan, 2017a; Srinivas et al., 2010), and hence must sample $T(1 - o(1))$ points in $\mathcal{R}_{\text{target}}$ due to property (ii). Then, property (i) ensures that most actions played don't need to be perturbed at all, so that $C = o(T)$.

We use the Synthetic1D objective function in Equation 4.17 to illustrate and discuss all of our attack methods. In Table 4.2, we provide a summary comparing each of our proposed adversarial attacks, comparing each attack based on factors such as whether it is *practical* to implement, level of *knowledge* needed.

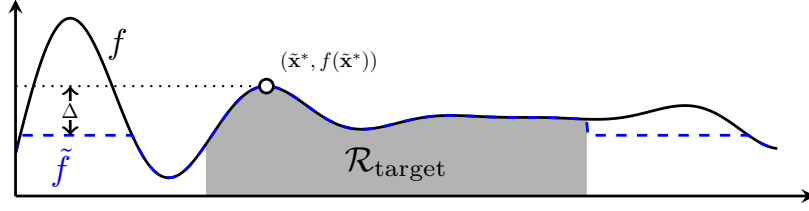


Figure 4.4. Clipping attack for known f .

4.4.1 Subtraction Attack (Known f)

For this attack, we follow the ideas discussed in Section 4.3.3. To maintain property (iii), *i.e.*, $\tilde{f} \in \mathcal{F}_k(B)$, we assume that $f \in \mathcal{F}_k(B/2)$,² and seek to find a perturbation function $h \in \mathcal{F}_k(B/2)$ such that

$$\tilde{f}(\mathbf{x}) = f(\mathbf{x}) - h(\mathbf{x}) \quad (4.12)$$

satisfies properties (i) and (ii). The triangle inequality then gives that $\|\tilde{f}\|_k \leq B$, as desired for property (iii).

We let $h(\mathbf{x})$ be a sum of support-bounded, with the support lying entirely outside $\mathcal{R}_0$ designed to “swallow” the peaks outside $\mathcal{R}_0$. For **Subtraction Rnd** we construct these functions using the bump function $h_{\text{bump}}(\mathbf{x}) = \exp\left(\frac{-1}{1-\|\mathbf{x}\|^2}\right)1\{\|\mathbf{x}\| \leq 1\}$ (see Section 4.3.3 for details), whereas for **Subtraction Sq** we use the simpler indicator function $h_{\text{ind}}(\mathbf{x}) = 1\{\|\mathbf{x}\| \leq 1\}$. Examples are shown in Figure 4.3.

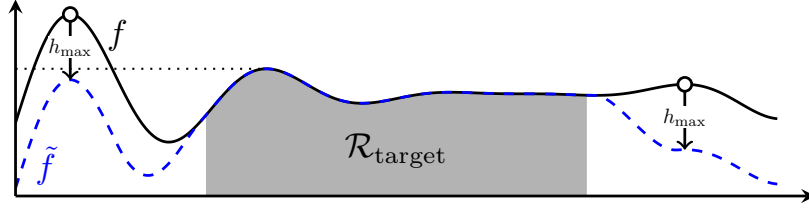
4.4.2 Clipping Attack (Known f)

The subtraction attack in Section 4.4.1 was developed for the primary purpose of obtaining theoretical guarantees, but in practice, finding $h(\cdot)$ with the desired properties may be challenging. Here we provide a more practical attack without theoretical guarantees, but with a similar general idea. Specifically, we propose to directly attain properties (i) and (ii) above by setting

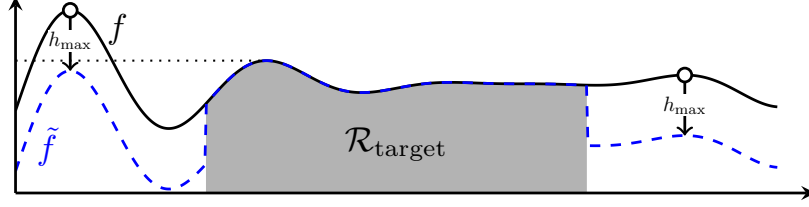
$$\tilde{f}(\mathbf{x}) = \begin{cases} f(\mathbf{x}) & \mathbf{x} \in \mathcal{R}_{\text{target}} \\ \min\{f(\mathbf{x}), f(\tilde{\mathbf{x}}^*) - \Delta\} & \mathbf{x} \notin \mathcal{R}_{\text{target}}, \end{cases} \quad (4.13)$$

where $\tilde{\mathbf{x}}^* = \arg \max_{\mathbf{x} \in \mathcal{R}_{\text{target}}} f(\mathbf{x})$ (illustrated in Figure 4.4).

²The factor $\frac{1}{2}$ can be replaced by other constants in $(0, 1)$.



(a) With “transition region”, corresponding with theory.



(b) Without “transition region”, simplified and practical to implement.

Figure 4.5. Aggressive Subtraction attack for unknown f .

Here, unlike for the subtraction attack, property (iii) does not hold, which is why the theoretical analysis does not follow. Despite this disadvantage (and the requirement of knowing f), this attack has the clear advantage of being simple and easy to implement, and we will see in Section 4.5 that it can be highly effective in experiments.

4.4.3 Aggressive Subtraction Attack (Unknown f)

In the case that f is unknown, we propose to build on the idea of the subtraction attack, but to be highly aggressive and subtract *all* points outside $\mathcal{R}_{\text{target}}$ by roughly the same value h_{max} . This is a special case of Equation 4.12, but here we consider $h(\cdot)$ having a much wider support than described above, so we find it best to highlight separately.

To ensure that $\|h\|_k \leq \frac{B}{2}$, we need to consider a “transition region” where $h(\cdot)$ transitions from zero to h_{max} . Overall, we are left with $h(\mathbf{x})$ equaling zero within $\mathcal{R}_0$, h_{max} outside $\mathcal{R}_{\text{target}}$, and intermediate values within $\mathcal{R}_{\text{target}} \setminus \mathcal{R}_0$.

Again based on Section 4.3, this attack has strong theoretical guarantees. A notable advantage is not requiring precise knowledge of f ; it suffices that $\mathcal{R}_{\text{target}}$ has a suitable local maximum and h_{max} is large enough, but no specific details of the function need to be known. In addition, this attack is fairly straightforward to implement, only requiring the selection of h_{max} and a transition region width.

A disadvantage is that a higher budget C may be needed in practice due to being highly aggressive. On the other hand, similar aggressive approaches have shown success in related bandit problems (Garcelon et al., 2020; Jun et al., 2018).

Simplified Aggressive Subtraction Attack

In the same way that Subtraction Sq simplifies Subtraction Rnd, we can simplify the aggressive subtraction attack as follows, for some $h_{\max} > 0$:

$$\tilde{f}(\mathbf{x}) = \begin{cases} f(\mathbf{x}) & \mathbf{x} \in \mathcal{R}_{\text{target}} \\ f(\mathbf{x}) - h_{\max} & \mathbf{x} \notin \mathcal{R}_{\text{target}}. \end{cases} \quad (4.14)$$

This can again be implemented without knowledge of f .

Similar to above, we would like to choose $h_{\max}$ large enough so that the maximizer of $\tilde{f}$ is in $\mathcal{R}_{\text{target}}$. This again has the disadvantage of potentially requiring a large corruption budget, but compared to the clipping attack, we gain the advantage of not needing to know f .

Both versions of Aggressive Subtraction are illustrated in Figure 4.5. In the rest of the paper, whenever we mention this attack, we are referring to the simplified variant.

4.5 Experiments

For convenience, our experiments³ consider targeted attacks and do not constrain the attack to any particular budget; we instead explore the trade-off between attack success rate (*i.e.*, fraction of actions played that fall in the $\mathcal{R}_{\text{target}}$) and attack cost (e.g., sum of perturbation levels used).

To the best of our knowledge, our work is the first for this setting, so there are no existing attacks to compare against. Hence, our focus is only comparing our proposed attacks to each other (*i.e.*, Subtraction Rnd, Subtraction Sq, Clipping, Aggressive Subtraction) along with two trivial baselines. The baselines are Random, which perturbs the function evaluation with $\mathcal{N}(\mu_a, \sigma_a^2)$ and, No Attack

³Source code available at <https://github.com/eric-vader/Attack-B0>.

which does not perturb the function evaluation at all.

We exclude Subtraction Rnd, Subtraction Sq from experiments on higher dimensional functions (*i.e.*, ≥ 3), due to the difficulty discussed in Section 4.4.1.

We compare the attacks on a variety of objective functions of up to 6 dimensions. For the experiments on synthetic functions, we manually add $z_t \sim \mathcal{N}(0, 0.01^2)$ noise. Detailed experimental details are given in Appendix B and our complete code is included in the supplementary.

4.5.1 Experimental Setup

In order to understand the behavior of each attack, we run each attack 300 times, varying 30 different hyperparameter choices (*i.e.*, $\Delta, h_{\max}, (\mu_a, \sigma_a), h(\mathbf{x})$) with 10 differing conditions (initial points, instances of the objective function, and random seeds). For both the Subtraction Rnd and Subtraction Sq attacks, we vary only the y -transformation hyperparameter, which we refer to as $h_{\max}$. The different hyperparameter settings are chosen such that, as much as possible, we aim to cover the entire spectrum of the behavior (starting with a lower success rate and ending with a high success rate) of the attack method for each experiment. We run 100 or 250 optimization steps depending on the dimensionality; we adopt the Matérn $5/2$ kernel to match our theory (see Sections B.2, B.4 for further details).

We focus on attacking the GP-UCB algorithm and defer the elimination algorithm to Appendix B, since GP-UCB is much more widespread and Corollary 4.1 indicates that it should be the harder algorithm to attack. (One reason for this is that MaxVar explicitly removes points from further consideration, so it can be forced to permanently discard the true optimum.) Following typical choices adopted in existing works' experiments (e.g., (Rolland et al., 2018; Srinivas et al., 2010)), we select the exploration parameter as $\beta_t = 0.5 \log(2t)$. There are very few defense algorithms in the literature that are appropriate here; in particular, the main algorithm in (Bogunovic et al., 2020) (Fast-Slow GP-UCB) was introduced for theoretical purposes and seemingly not intended to be practical. However, (Bogunovic et al., 2020) also shows that GP-UCB with *larger choices of β_t depending on C* can be provably more robust, and we provide experimental

Name	Centroid	Length
Synthetic1D	(0)	1
Forrester1D	(0.25)	0.5
Levy1D	(−2.915)	4
Levy-Hard1D	(−11.56)	6.881
Bohachevsky2D	(55, 55)	90
Bohachevsky-Hard2D	(75, 75)	50
Branin2D	(−1, 11)	8
Camelback2D	(0.090, −0.713)	1.425
Robot3D	(2.5, 2.5, 20)	(5, 5, 20)
Robot4D	(2.5, 2.5, 20, $\pi/2$)	(5, 5, 20, π)
Hartmann6D	(0.6, $\dots$, 0.6)	0.8

Table 4.2. Synthetic functions with a summary of their $\mathcal{R}_{\text{target}}$ parameters.

support for this in Section 4.5.5.

4.5.2 Metrics

We measure the following up to iteration t , with $X_t = (\mathbf{x}_1, \dots, \mathbf{x}_t)$ and adversarial noise $A_t = (|c_1|, \dots, |c_t|)$.

Success Rate

We measure the success of each attack using the proportion of actions played that lie within $\mathcal{R}_{\text{target}}$:

$$\text{Success-Rate}(t) = \frac{|\mathcal{R}_{\text{target}} \cap X_t|}{t}. \quad (4.15)$$

Normalized Cost

We measure the cost up to iteration t by summing the history of adversarial noise, and normalize by the function range for easier comparison across experiments:

$$\text{Normalized-Cost}(t) = \sum_{a \in A_t} \frac{a}{f_{\max} - f_{\min}}. \quad (4.16)$$

4.5.3 Synthetic Experiments

For convenience, we explicitly append the dimensionality to the name of the function. Firstly, we consider the 1D function

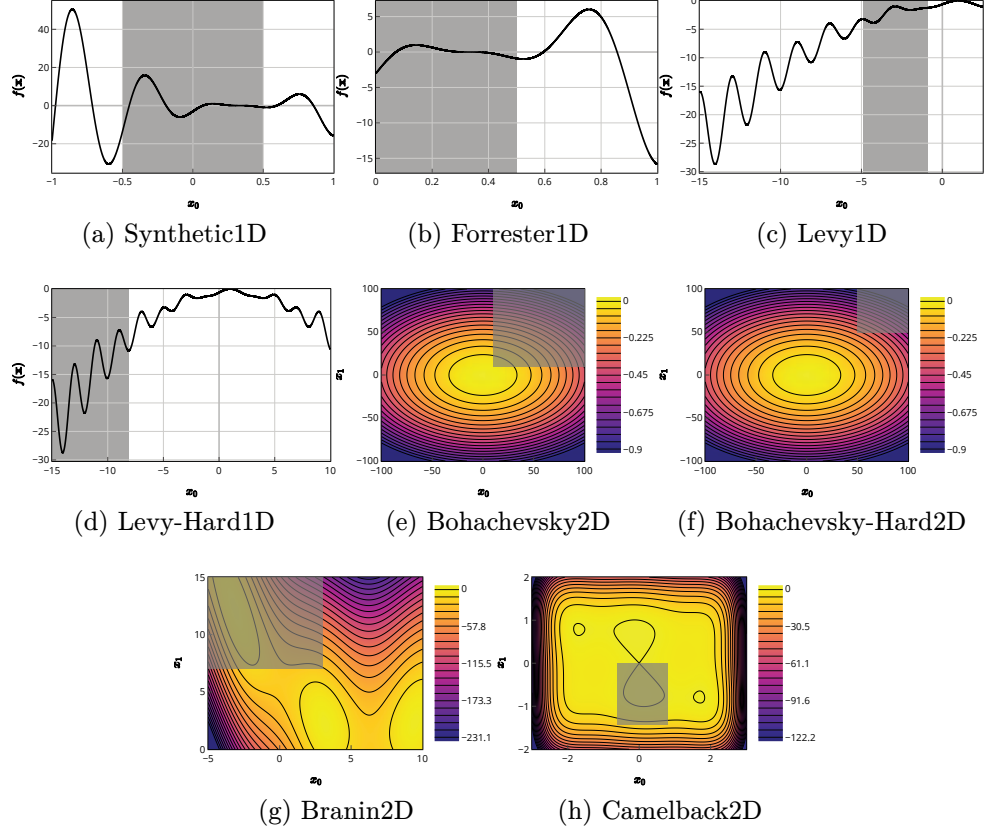


Figure 4.6. Illustrations for 1D and 2D experiments; $\mathcal{R}_{\text{target}}$ is shaded.

$$f(x) = (6x - 2)^2 \sin(12x - 4), \quad (4.17)$$

which we refer to as Synthetic1D. Then, we compare the attacks on commonly used optimization synthetic function benchmarks (Eggenberger et al., 2013) as listed in Table 4.2. We have chosen these benchmarks so that we attack under a diverse range of conditions. The 1D and 2D functions are illustrated in Figure 4.6, each target region $\mathcal{R}_{\text{target}}$ is rectangular, and defined by its centroid and length; the choices for each experiment are given in Table 4.2. Levy-Hard1D and Bohachevsky-Hard2D are variants of Levy1D and Bohachevsky2D, but with different $\mathcal{R}_{\text{target}}$, chosen to increase the difficulty of the attack.

Synthetic1D and Forrester1D are relatively simple examples where the second-best peak falls inside of the $\mathcal{R}_{\text{target}}$ and there are few local maxima. Levy1D is an experiment with a periodic function, which increases the difficulty by the introduction of multiple local maxima. Levy-Hard1D uses the same function, but with a different domain and $\mathcal{R}_{\text{target}}$ such that the second-best peak falls outside

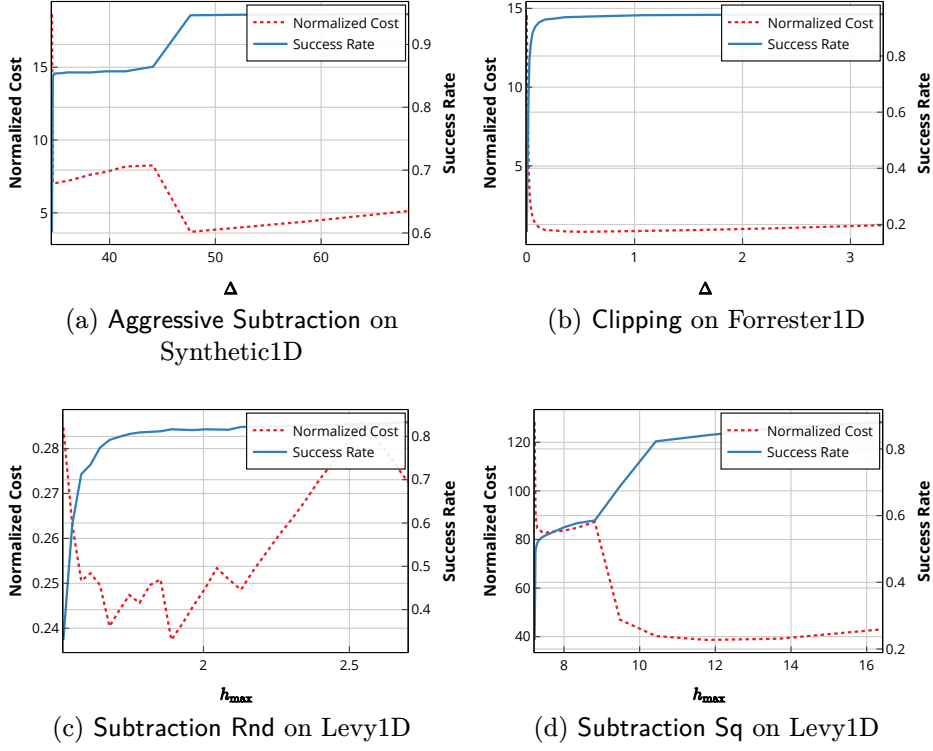


Figure 4.7. Effects of hyperparameters on the different attacks.

of $\mathcal{R}_{\text{target}}$. Bohachevsky2D has a characteristic bowl shape with a single maximum, which is easy to optimize but difficult to attack. Here, we use a variant of the Bohachevsky2D function that is scaled by a constant factor to better manage the function range. Bohachevsky-Hard2D uses the same Bohachevsky2D function but with a smaller $\mathcal{R}_{\text{target}}$ to further increase the difficulty. Specifically, the function increases throughout $\mathcal{R}_{\text{target}}$ and when leaving $\mathcal{R}_{\text{target}}$, causing the attack to be especially difficult as explained in Section 4.3.3.

Branin2D has 3 global maxima, and $\mathcal{R}_{\text{target}}$ contains one of the global maxima. Camelback2D has 2 global maxima and several local maxima, and $\mathcal{R}_{\text{target}}$ contains one of the global maxima. As an example with higher dimensionality, Hartmann6D is a 6-dimensional function with a global maximum lying outside $\mathcal{R}_{\text{target}}$.

Finally, we attack the 3-dimensional and 4-dimensional functions – Robot3D and Robot4D respectively (Zi Wang & Jegelka, 2017). In both of these experiments, $\mathcal{R}_{\text{target}}$ is again rectangular, but with different lengths in each dimension.

The performance of each attack method depends on the associated hyper-

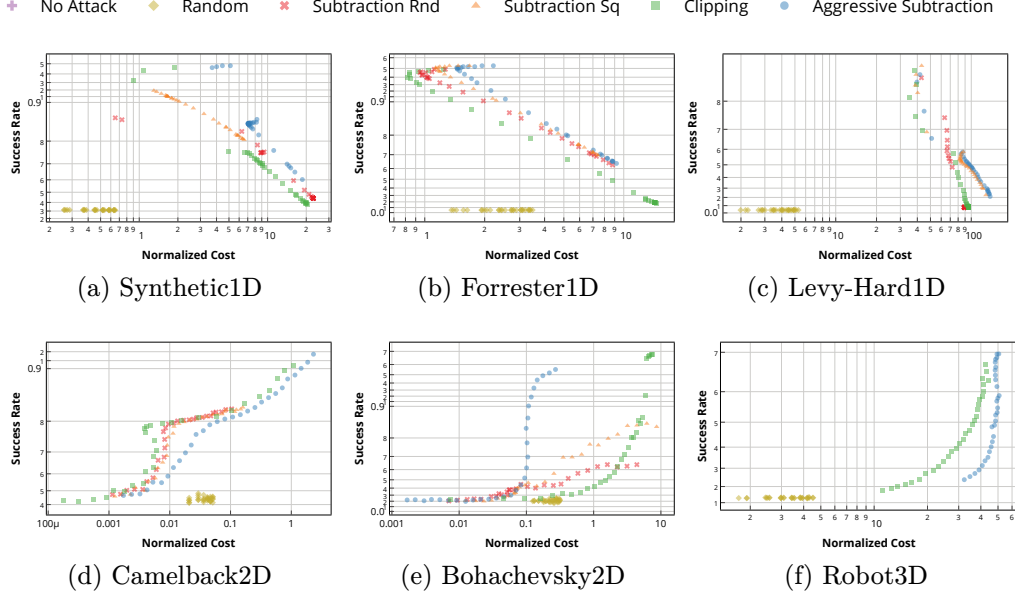


Figure 4.8. Scatter plots of runs averaged over random seeds for various functions.

parameter (e.g., $h_{\max}$ or Δ). Ideally, we want the best hyperparameter for the attack where it spends the least cost for the highest success rate. One might expect that a higher cost is always associated with a higher success rate, but this is not quite the case, as we observe in Figure 4.7. The subtlety is that a poorly-chosen hyperparameter can be poor in both regards. Typically, what happens is that when the hyperparameter is at its ‘boundary value’ (e.g., $\Delta = 0$ in Figure 4.4) the attack is not very successful, because GP-UCB explores both the clipped regions and $(\tilde{\mathbf{x}}^*, f(\tilde{\mathbf{x}}^*))$. However, going carefully beyond the boundary value can quickly lead to a near-maximal success rate with a fairly modest cost. Perhaps less intuitively, even attacking more aggressively (e.g., increasing Δ) sometimes maintains a similar cost-success trade-off, suggesting that it may be preferable to be ‘over-aggressive’ than ‘under-aggressive’.

From Figure 4.8, we observe that Clipping performs consistently well across the various functions, providing competitive success rates and costs. Both Subtraction Rnd and Subtraction Sq tend to have a performance ‘in between’ that of Clipping and Aggressive Subtraction. Due to Subtraction Rnd’s smooth $h(\mathbf{x})$, Subtraction Rnd tends to narrowly beat Subtraction Sq in most experiments, e.g., see Figure 4.8b. The trend is also observed in Levy1D, the easier version of Levy-Hard1D, as seen in Figure B.6c. However, these aforementioned meth-

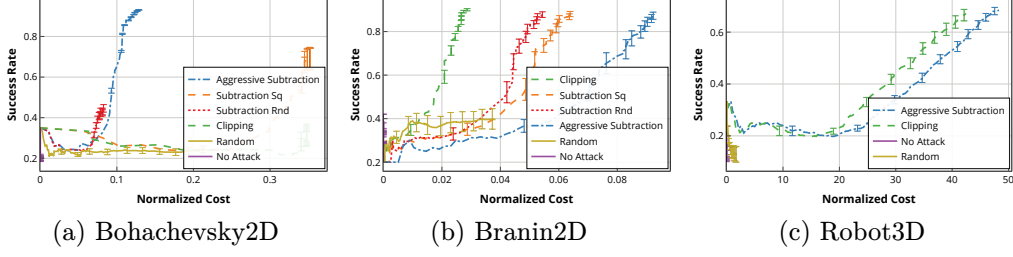


Figure 4.9. Plots of each attack with the most efficient (best success rate over normalized cost) hyperparameter.

ods require prior knowledge of f , whereas **Aggressive Subtraction** works well in practice without knowledge of f .

We additionally observe in Figure 4.8 that across all of the functions considered, there typically exists a successful attack with a relatively low normalized cost, e.g., on the order of magnitude 1 or less, though more difficult cases also exist (e.g., Levy-Hard1D). This highlights the general lack of robustness of standard solutions to the GP bandit problem.

In Figure 4.9, we plot the performance of each attack method with its most efficient hyperparameter. The hyperparameter with the largest success rate over cost is the most efficient,

$$\theta_{\text{efficient}} = \arg \max_{\theta} \frac{\text{Success-Rate}(T)_{\theta}}{\text{Normalized-Cost}(T)_{\theta}}. \quad (4.18)$$

Considering the most efficient hyperparameter, **Clipping** continues to be successful and cost effective compared to others for most functions as seen in Figures 4.9b, 4.9c. **Subtraction Rnd** and **Subtraction Sq** tend to perform in between **Clipping** and **Aggressive Subtraction**, e.g., see Figure 4.9b.

There are also some interesting cases where this trend is not observed, **Subtraction Rnd** and **Subtraction Sq** struggle to achieve a high success rate in Camelback2D, while **Aggressive Subtraction** and **Clipping** achieve a higher success rate after spending more cost. Camelback2D has a symmetric surface with multiple global and local optima in different regions, which increases the difficulty in constructing $h(\mathbf{x})$ for the subtraction attacks. Similarly, for the attacks with the most efficient hyperparameter in Bohachevsky2D, **Aggressive Subtraction** is the most efficient, spending the least but having the highest success rate. Similar

findings are observed in Figure 4.8e. Bohachevsky2D has a bowl shape with a single maximum, which is easy to optimize but hard to attack.

As a result, Bohachevsky2D increases throughout $\mathcal{R}_{\text{target}}$ and when leaving $\mathcal{R}_{\text{target}}$, causing the attack to be especially difficult as explained in Section 4.3.3 with Figure 4.2. Hence, there is no obvious way to construct $h(\mathbf{x})$ for both Subtraction Rnd and Subtraction Sq (also for Camelback2D). We believe that Clipping is less effective because it creates $\tilde{f}$ which is largely flat (equal to the clipped value), the algorithm may spend too long exploring this flat region. In contrast, Aggressive Subtraction maintains the bowl shape (shifted downwards) outside $\mathcal{R}_{\text{target}}$, which the algorithm can more readily stop exploring due to believing it to be sub-optimal. Hence, Aggressive Subtraction performs best here in terms of success rate and cost.

4.5.4 Robot Pushing Experiments

We consider the deterministic robot pushing objective from (Zi Wang & Jegelka, 2017) to find the best pre-image to push an object towards a target location. We test both the 4-dimensional variant $f_4(r_x, r_y, r_t, r_\theta)$ and 3-dimensional variant f_3 , where $r_\theta = \arctan(r_y/r_x)$. Both functions compute the distance between the pushed object and the target location. Here, $r_x, r_y \in [-5, 5]$ is the robot location, pushing duration $r_t \in [1, 30]$ and pushing angle $r_\theta \in [0, 2\pi]$. The results from Figure 4.8f and Figure 4.9c are consistent with our findings from synthetic experiments; in particular, Clipping performs better than Aggressive Subtraction.

4.5.5 Defense Experiments (Without Knowledge)

We consider defending against our attacks using a combination of two techniques. Firstly, we use a technique proposed in (Bogunovic et al., 2020), namely, adding a constant C to the exploration parameter:

$$\beta_t = 0.5 \log(2t) + C. \quad (4.19)$$

Secondly, we increase the model’s robustness to adversarial noise via the model’s noise variance η^2 in the posterior update equations:

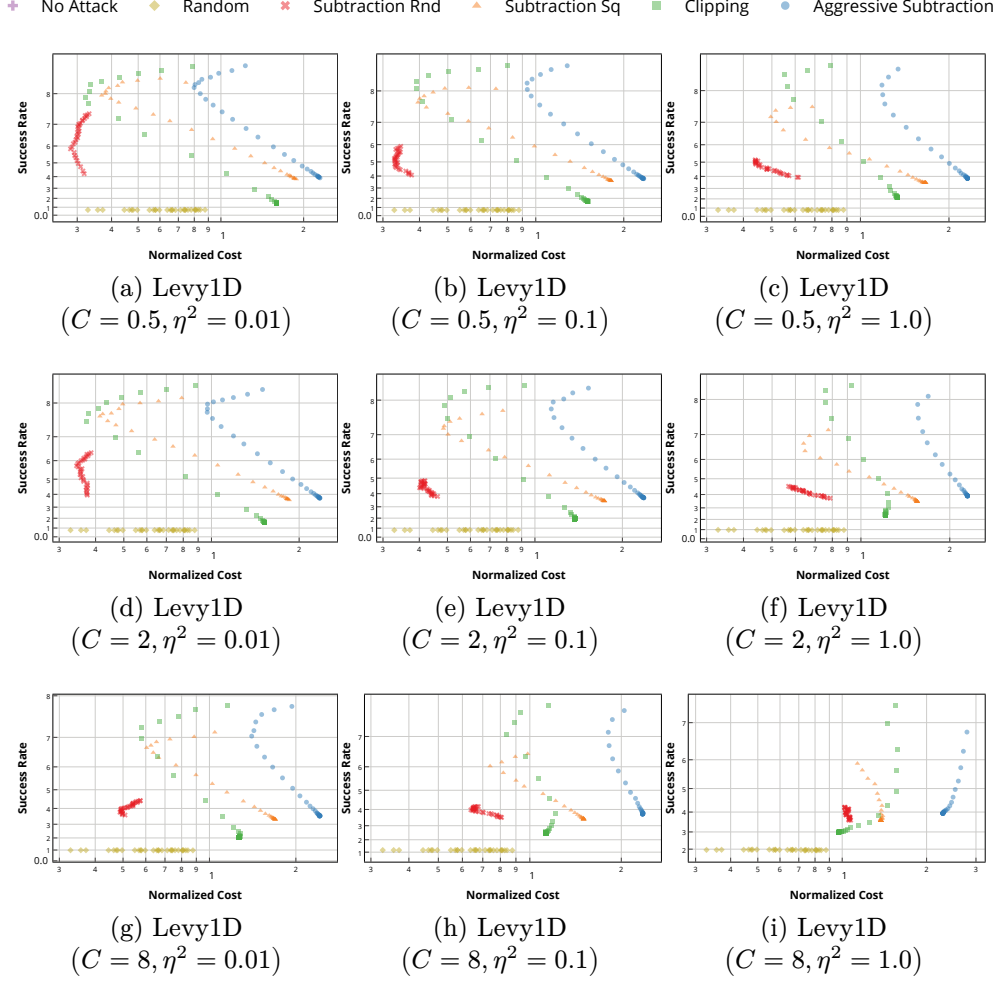


Figure 4.10. Variants of GP-UCB which add a constant C to the confidence width in order to defend against the attack.

$$\begin{aligned}\mu_{t+1}(\mathbf{x}) &= \mathbf{k}_t(\mathbf{x})^\top (\mathbf{K}_t + \eta^2 \mathbf{I}_t)^{-1} \mathbf{y}_t, \\ \sigma_{t+1}(\mathbf{x}) &= k(\mathbf{x}, \mathbf{x}) - \mathbf{k}_t(\mathbf{x})^\top (\mathbf{K}_t + \eta^2 \mathbf{I}_t)^{-1} \mathbf{k}_t(\mathbf{x}).\end{aligned}\tag{4.20}$$

We vary $C \in \{0.5, 2, 8\}$ with $\eta^2 \in \{0.01, 0.1, 1\}$, resulting in nine different parameter combinations, allowing us to study their combined effect. We note that these defense techniques are unaware of the attack method nor its budget.

We observe from Figure 4.10 that increasing C indeed decreases the attack success rate and increases the cost for each run, though seemingly not in a very major way. Essentially, increasing the confidence width causes GP-UCB to become more explorative and less exploitative, making it harder for the attacker to quickly steer the algorithm towards $\mathcal{R}_{\text{target}}$, and hence increasing the attack cost.

Similarly, we see that increasing η^2 tends to increase the cost for each run. By increasing η^2 , we make the model expect more noise, and thus be more cautious against ‘unusual’ observations. Thus, the adversary needs to spend more cost to perform the attack. This defense works significantly better on the Subtraction Attacks (Subtraction Rnd, Subtraction Sq) than on Clipping and Aggressive Subtraction. We believe that this is because Clipping and Aggressive Subtraction perturb the function across the majority of the domain rather than a few localized regions. The latter can potentially be written off as random noise more easily than the former.

The higher the value of C used in Equation 4.19 and of η^2 used in Equation 4.20, the more similar the attacks tend to behave, and the less sensitive the algorithm tends to be to varying the attacker’s hyperparameters. We observe from Figure 4.10i that the combination of both simple defense techniques on Subtraction Rnd and Subtraction Sq increase robustness.

Overall, we believe that this experiment, and our work in general, motivates the study of further defenses beyond the simple defenses proposed, particularly when targeting applications in which robustness is critical. More sophisticated defenses, for example those that are aware of the attack method or budget, can be explored in further work.

4.5.6 Further Experiments

In Appendix B, we present more details on the above findings, as well as exploring: (i) different kernel choices, (ii) attacking the elimination algorithm, (iii) online vs. offline kernel learning, and (iv) a proof-of-concept attack that automatically adapts its hyperparameters.

4.6 Conclusion

We have studied the problem of adversarial attacks on GP bandits, providing both a theoretical understanding of when certain attacks are guaranteed to succeed, as well as a detailed experimental study of the cost and success rate of our attacks. Possible future research directions include (i) further investigating more

“automated” attacks that can adapt their hyperparameters online, and (ii) theoretical lower bounds on the attack budget, which are currently unknown even in the simpler linear bandit setting, and (iii) exploring more advanced defenses, especially for applications where robustness is a critical factor.

Chapter 5

Black-box Adversarial Attacks on CNNs

Following our discussion from Section 2.3, we address challenges in black-box adversarial attacks, focusing on the black-box hard-label setting and proposing domain-knowledge-driven techniques for dimensionality reduction, while also introducing query-efficient hyperparameter selection methods.

Here, we focus the sequential search on perturbations, optimizing them sequentially for each image we aim to attack, akin to a typical BO setup. A key difference here is that GP is used to model, indirectly, the *latent perturbation* instead. Then, we apply dimensionality techniques to the *latent perturbation* and produces an *image perturbation* that can be applied to attack the CNN. BO has successfully been used with dimensionality techniques such as Nearest Neighbour Interpolation (NNI) and Low-frequency Fourier Transform (LF) for ℓ_∞ , and ℓ_2 untargeted attacks in a query limited setting (Shukla et al., 2021).

NNI is a simple image resizing technique that assigns each pixel in the output image the value of the nearest pixel in the input image, without any interpolation or smoothing (Mitchell, 1997). We illustrate in Figure 5.1 the structure of the image perturbation produced by NNI when applied to the latent perturbation for the ℓ_∞ attack as per (Shukla et al., 2021). We can clearly see the latent perturbation structure even in the image perturbation produced; *i.e.*, NNI’s latent perturbation $\lfloor rd \rfloor \times \lfloor rd \rfloor = 2 \times 2$ structure can be seen in the 32×32 -pixel image perturbation in Figure 5.1a.

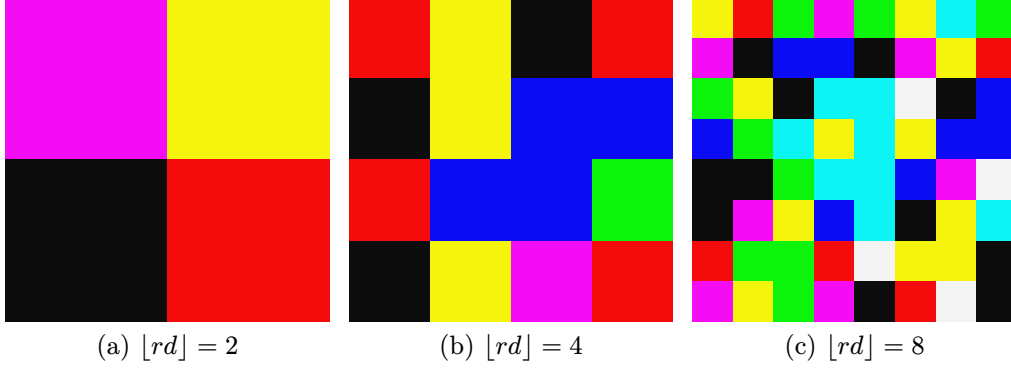


Figure 5.1. Image perturbation (32×32 -pixels) resulting from using NNI for the ℓ_∞ attack; Hyperparameter $\lfloor rd \rfloor$ determines the latent perturbation size.

0	1	2	3	4	3	2	1
5	6	7	8	9	33	32	31
10	11	12	13	14	30	29	28
15	16	17	18	19	27	26	25
20	21	22	23	24	23	22	21
15	25	26	27	19	18	17	16
10	28	29	30	14	13	12	11
5	31	32	33	9	8	7	6

Figure 5.2. Discrete Fourier transform of image data (Rao, Kim, & Hwang, 2010), showing the conjugate symmetry; Real values - white, conjugate pairs - light and dark grey. We refer to top left quadrant (0 to 24) as Q1, top right quadrant (1 to 23) as Q2, bottom left (5 to 24) as Q3 and bottom right as Q4.

Fourier transform is a linear transform that, when applied to a natural image, results in a representation in frequency space by sine and cosine basis vectors. The Fast Fourier Transform (FFT) is an algorithm that computes the discrete Fourier transform of a sequence, or its inverse (Cooley & Tukey, 1965). As image data is real-valued, as illustrated in Figure 5.2, only the resulting transform in the light grey areas (quadrants Q1,Q3) are unique, and the dark grey areas are the corresponding conjugate pairs. In our illustrations of FFT values, whenever the unshifted & uncentered¹ configuration is shown, only unique values are displayed, ignoring the conjugate values, unless otherwise stated.

For the ℓ_2 setting, (Shukla et al., 2021) used BO to find the latent perturbation in low-frequency Fourier space; the latent perturbation is then transformed using FFT to an image perturbation, referring to this as LF. We show an example latent perturbation (in Fourier space) along with its image perturbation in

¹The configuration can be shifted & centered such that $(0, 0)$ is in the center of the image.

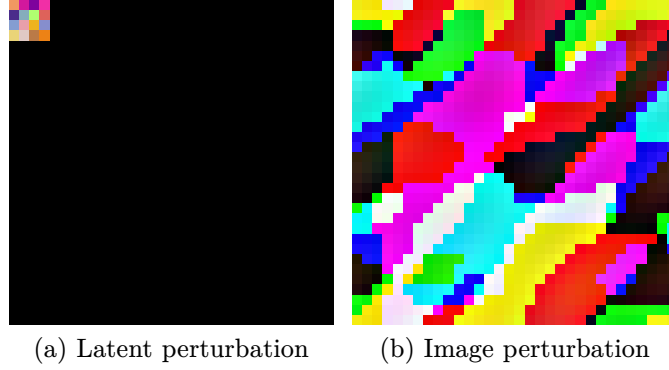


Figure 5.3. Latent perturbation and its corresponding Image perturbation resulting from applying LF for the ℓ_2 attack when hyperparameter $\lfloor rd \rfloor = 4$.

Figure 5.3; we see that LF only attacks a small, low-frequency portion of Fourier space. We note that increasing the $\lfloor rd \rfloor$ hyperparameter to attempt to cover all frequencies is not effective for ℓ_2 setting as it leads to applying the perturbation budget too *thinly* across frequencies.

While both NNI and LF methods effectively reduce the search space dimensionality and preserve norms during transformation, as noted in (Shukla et al., 2021), there is substantial potential for further exploration and enhancement of dimensionality reduction techniques. Inspired by successful application of domain knowledge to Boundary Attacks (Brunner et al., 2019), we propose alternative approaches to the dimensionality reduction using the same framework proposed in (Shukla et al., 2021) for both ℓ_∞ and ℓ_2 untargeted attacks.

5.1 Contributions

Our main contributions are as follows:

1. We introduce new transforms of the latent space for BO-based attacks built on domain knowledge of CNNs:
 - a. For ℓ_∞ attacks, we propose the Tiling Transform, which tiles latent perturbation to obtain image perturbation with tile sizes corresponding to feature map levels (low-level, *e.g.*, line detection vs high-level features) in the CNN being attacked.
 - b. For ℓ_2 attacks, we propose the Frequency-Specific Fourier Transform,

which allows choosing an attack frequency to exploit the CNN’s weakest frequency. We also present a variant focused on targeting a singular frequency.

2. We present techniques to select the hyperparameters while attack is progressing (*i.e.*, on the fly):
 - a. In the case of tiling-based attacks, we formulate a bandit-based technique to find the latent image size hyperparameter while performing the attack.
 - b. In the case of Fourier-based attacks, we propose using GP to model the success rate across the frequencies as the attack progresses and use BO to pick the next best frequency.
3. We demonstrate experimentally that our proposed attacks can be significantly more effective than the baseline methods introduced earlier, using fewer queries and achieving a higher attack success rate.

5.2 Setup

We consider the setup proposed in (Shukla et al., 2021). Let $F(\mathbf{x}) : \mathbb{R}^D \rightarrow C$ be a K -class image classifier, where $C = \{c_i\}_{i=1}^K$. In our setup, the image classifier F is a CNN that takes in the flattened, normalized Red, Green, and Blue (RGB) input $\mathbf{x} \in [0, 1]^D$, $c \in C$ is the corresponding ground truth label, and $F(\mathbf{x})$ is the label predicted by the CNN. We note that the flattened $\mathbf{x} \in [0, 1]^D$ input is equivalent to the RGB image $X \in [0, 255]^{d \times d \times l}$, with discrete values of size $d \times d$ with l color channels ($D = d \times d \times l$). In our work, we will refer to the image as $\mathbf{x}$ and convert it to the corresponding image X for illustration purposes.

Our objective is to find an image perturbation δ such that the CNN would misclassify (*i.e.*, $F(\mathbf{x}') \neq c$) the perturbed image $\mathbf{x}' = \mathbf{x} + \delta$. In our approach, we consider the hard-label, black-box, untargeted setting, where we *do not possess any knowledge* of the internal mechanisms of the specific CNN. Our formulation relies on domain knowledge which applies to general CNNs, but does not rely on information about the specific CNN (*e.g.*, architecture). When we make a query to the network using F , we will *only* receive the final output class $F(\mathbf{x}) \in C$.

We focus on the hard-label case since it can also easily be applied to soft-label settings, is more challenging, often more realistic, and was the focus of the main baselines that we build on.

We take the typical approach of considering a perturbation budget of ϵ under some norm ℓ_p , so that the image perturbation found is subtle and hard to detect in perturbed image. We consider commonly used norms in the literature for the perturbation budget, including both ℓ_2 and ℓ_∞ norms. We let N denote the total query budget available to the adversary; if an adversarial example is not found after N evaluations of the neural network F , the attack is considered to be unsuccessful. Our objective to find the image perturbation δ is as follows:

$$\begin{aligned} \delta = \arg \max_{\delta} f(\mathbf{x}, y, \delta) \quad \text{subject to } \|\delta\|_p < \epsilon \text{ and } \mathbf{x}' \in [0, 1]^D \\ \text{where } f(\mathbf{x}, y, \delta) = \begin{cases} 0 & \text{if } F(\mathbf{x}') \neq c \\ -1 & \text{otherwise} \end{cases}, \text{ and } \mathbf{x}' = \mathbf{x} + \delta \end{aligned} \quad (5.1)$$

We pose the hard-label black-box attack as a constrained optimization problem, to find the image perturbation δ where $f(\mathbf{x}, y, \delta)$ is maximal. We note that it is possible to modify the objective to a targeted setting, where the objective is to misclassify the input into a pre-determined label c_{target} instead of *any other* label. In such setting, we can simply replacing the condition $F(\mathbf{x}') \neq c$ with $F(\mathbf{x}') = c_{\text{target}}$ in Equation 5.1.

5.2.1 Adversarial Attack using Bayesian Optimization

The goal is to minimize the number of queries to the potentially costly neural network F , while trying to find image perturbation δ via finding the optima to f . In our setting, f is not known exactly (no knowledge of the neural network) and expensive to evaluate (*e.g.*, due to seeking few queries to avoid detection). We model the observations as coming from a GP with additive $\mathcal{N}(0, \sigma^2)$ noise. Typically for discrete functions, an approximate GP is used to model the observations, such as using a Gaussian prior but with a Bernoulli likelihood. However, we found that in practice, this is more complicated to implement (*e.g.*, intractable posteriors) without any significant benefit. A regular GP model was

Algorithm 5.1: BO-ATTACK($\mathbf{x}, y$)

```
1 Initialize  $\mathcal{D}_{N_{\text{init}}} \leftarrow \{(\delta'_1, v_1), \dots, (\delta'_{N_{\text{init}}}, v_{N_{\text{init}}})\}$ 
2 Update GP on  $\mathcal{D}_{N_{\text{init}}}$ 
3 for  $t = N_{\text{init}} + 1, \dots, N$  do
4   Optimize acq fn  $\delta'_t \leftarrow \arg \max_{\delta'} [a(\delta')]$ 
5   Map latent to image perturbation  $\delta_t \leftarrow T(\delta'_t)$ 
6   Scale  $\delta_t$  within perturbation budget, s.t.  $\|\delta_t\|_p \leq \epsilon$ 
7   Observe  $v_t \leftarrow f(\mathbf{x}, y, \delta_t)$ 
8   if  $v_t < 0$  then
9     Add  $\mathcal{D}_t \leftarrow \mathcal{D}_{t-1} \cup \{(\delta'_t, v_t)\}$  and update GP
10  else
11    return Successful image perturbation  $\delta_t$ 
12 return Unsuccessful
```

similarly used by in the most closely related existing work (Shukla et al., 2021). While using a GP to directly model binary-valued observations may seem unnatural, it is partly justified by the fact that GP methods are also provably effective under sub-Gaussian noise (Chowdhury & Gopalan, 2017b), which can be discrete-valued or continuous.

BO is particularly effective for problems with low dimensionality; although we can apply BO directly to obtain the image perturbation δ , standard techniques become infeasible in high-dimensional settings. In practice, BO’s speed and performance degrades quickly as dimensionality increases, as discussed in earlier chapters. We may apply a scalable BO method such as the additive methods, but dimensionality reduction techniques are still applied to reduce the size of the search space before application of the scalable BO (Ru et al., 2020; C.-C. Tu et al., 2019); see experiments in Section 3.4.5. In this chapter, we focus on proposing domain-knowledge-driven dimensionality techniques in the context of adversarial attacks on CNNs. Following (Shukla et al., 2021 Algorithm 1), BO is used to find the latent perturbation and dimensionality techniques are applied to get the image perturbation.

The BO framework, as shown in Algorithm 5.1, starts by having N_{init} initial perturbations sampled from some distribution (*e.g.*, standard normal or uniform). Then, we update the posterior distribution using all available data. The algorithm finds the next latent perturbation δ'_t by optimizing the acquisition

function over the posterior distribution. We use a well established approach by optimizing the Expected Improvement (EI) acquisition function $a(\cdot)$ using L-BFGS (J. Kim & Choi, 2020; Murphy, 2012). Next, the chosen latent perturbation δ'_t is mapped via transform $T(\cdot)$ from the low-dimensional latent space to an image perturbation δ_t , which is of the full input space. After scaling δ_t to meet the perturbation budget, we observe the value v_t by querying the CNN at the image perturbation point δ_t . This process repeats until a successful attack is achieved or when query budget is exhausted. We refer to this application of BO as BO-Adv.

We propose two different transforms based on domain knowledge of CNNs (described in detail later): Tiling for ℓ_∞ attack, and FSFT for ℓ_2 attack. Our proposed latent transformations preserve their respective norms. We note that although we tailor these transforms for the respective ℓ_p attacks, the transforms can also be used for the other (*e.g.*, Tiling for ℓ_2), but the resulting image perturbation must be scaled back to fit the perturbation budget ϵ (*e.g.*, $\|\delta\|_p = \epsilon$).

5.3 Tiling Transform (ℓ_∞ Attack)

We first introduce the Tiling method for the ℓ_∞ attack, exploiting the shift-invariant nature of simpler features (*i.e.*, edges and direction) encoded in the initial layers of CNNs. In CNNs, feature maps in the initial layers encode simpler features such as edges and direction, while deeper layers encode more complex features of the objects being classified (LeCun, 2016). The effectiveness of tiling a latent perturbation lies in attacking lower-level feature detection, causing incorrect signals to propagate in subsequent layers of the CNN.

Another motivation for the Tiling attack is that CNNs are constructed with shift-invariant components. At a high level, shift-invariance means that the classifier would not be affected by displacements of the input and continue to classify correctly. CNNs exhibit this desirable property (LeCun, Bengio, & Hinton, 2015) to some extent, thanks to the use and composition of shift-invariant components such as convolution filters (Bietti & Mairal, 2019). We attempt to exploit this property, so a successful attack in one section of the image is likely to be effective

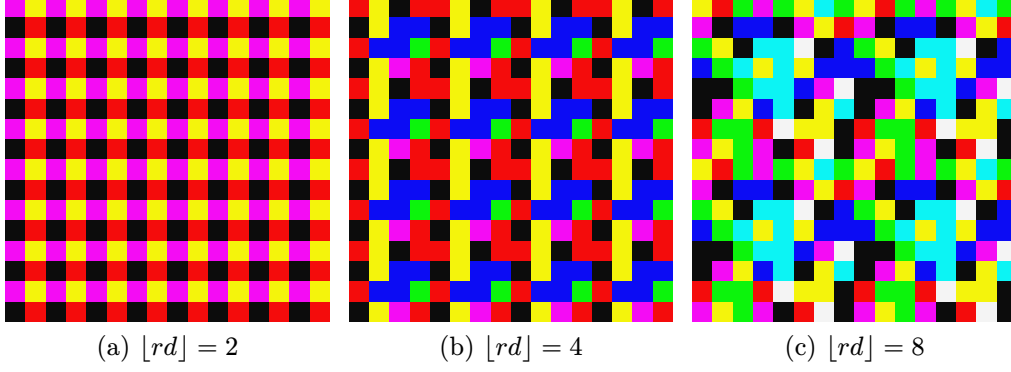


Figure 5.4. Example image perturbation produced by Tiling with various hyperparameters $\lfloor rd \rfloor$, corresponding to the level of feature map in CNN to attack.

Algorithm 5.2: TILING-TRANSFORM(δ'_t)

input : Latent perturbation $\delta'_t \in [0, 1]^{\lfloor rd \rfloor \times \lfloor rd \rfloor \times l}$
output: Image perturbation $\delta_t \in [0, 1]^{d \times d \times l}$

- 1 Let $K \leftarrow \lceil \frac{d}{\lfloor rd \rfloor} \rceil$ to be the number of tiles
- 2 Initialize tiled perturbation $\delta_t \leftarrow [\vec{0}]_{K \times K}$
- 3 **for** $x = 1, \dots, K$ **do**
- 4 **for** $y = 1, \dots, K$ **do**
- 5 $\delta_t[x][y] = \delta'_t$ // Where each $\delta_t[x][y]$ is an image sub-block.
- 6 Crop δ_t to size $d \times d \times l$
- 7 **return** δ_t

in other areas as well, especially at lower levels. Tiling allows the attacker to exploit multiple instances of the same vulnerability, increasing the chances of a successful attack. In comparison, NNI produces large constant-value regions that may not align well with typical CNN architectures for images, potentially reducing its effectiveness.

Tiling considers an attack on an image X_i with size $d \times d$ with l color channels, and a low-dimensional latent subspace $\lfloor rd \rfloor \times \lfloor rd \rfloor$ for every color channel. Similar to NNI, the latent subspace size $\lfloor rd \rfloor$ remains a hyperparameter. Here, it roughly related to the level of feature map to attack, with smaller values addressing lower-level features and larger values targeting higher-level features. Firstly, Algorithm 5.1 chooses latent perturbation δ'_t and pass it to this transformation as an input. We focus on the *base tile* perturbation, with each tile having a pixel size of $\lfloor rd \rfloor \times \lfloor rd \rfloor$, excluding color channels, as the perturbations for each color channel are applied following the pattern of this base tile. The *base tile*

is tiled $\lceil d/\lfloor rd \rfloor \rceil \times \lceil d/\lfloor rd \rfloor \rceil$ times in both dimensions. In the case where the tiled image noise is larger than $d \times d$, we crop to $d \times d$. Finally, we return the image perturbation δ_t , with size $d \times d \times l$. The algorithm is detailed in Algorithm 5.2 with image perturbations for various $\lfloor rd \rfloor$ values shown in Figure 5.4.

5.3.1 Projection on the ϵ -ball

We consider an (optional) additional transformation for ℓ_∞ attacks, by projecting the image perturbation to be exactly on the boundary of the ϵ -ball, *i.e.*, $\epsilon \times \text{sgn}(\delta)$. For example, considering a perturbation budget of $\epsilon = 5$, an image perturbation of $(\frac{1}{5} \frac{-1}{2})$ would be projected to $(\frac{5}{5} \frac{-5}{5})$. Projection onto the ϵ -ball is particularly effective in ℓ_∞ norm attacks due to the nature of this norm (maximum absolute) and the way it measures perturbations. By projecting, we ensure that every feature is perturbed to its maximum allowable extent, effectively ‘maximizes’ our ϵ budget, as $\|\epsilon \times \text{sgn}(\delta)\|_\infty = \epsilon$, yet ensuring that the perturbation remains within the specified perturbation budget ϵ . However, this technique only applies to ℓ_∞ , but not ℓ_2 attacks. The projection is done in Algorithm 5.2, just before image perturbation δ_t is returned. We note that this can also be similarly applied to NNI.

5.4 Frequency-Specific Fourier Transform (ℓ_2 Attack)

Here, we propose a frequency-specific Fourier transform for ℓ_2 attack, attacking the specific features that the CNN rely on to classify images. Data augmentation techniques, such as adversarial training and Gaussian data augmentation, may be used during training to improve CNN’s robustness. However, these techniques often result in trade-offs, where improvements in performance against certain types of corruptions come at the expense of reduced performance against others (Yin et al., 2019); their statistical study reveal these techniques enhance robustness against high-frequency domain corruptions, but may decrease robustness against low-frequency domain corruptions.

Low-frequency Fourier features correspond to the fundamental structure and overall content of an image, contribute to its general shape and layout of objects.

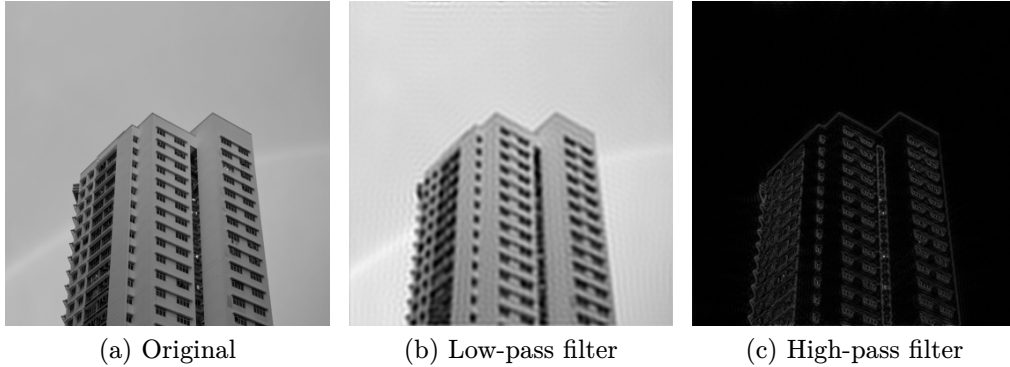


Figure 5.5. A multi-story apartment building with a rainbow arching over it; Applying the low/high pass filter to the image, with its respective high/low-frequency components removed.

On the other hand, high-frequency Fourier features emphasize the intricate details and finer nuances, contributing to the texture and specific characteristics of the image. We show an illustration in Figure 5.5b where the high-frequency features are removed, the resulting image is still perceptible, but blurry; whereas in Figure 5.5c, the low-frequency features are removed and the resulting image is imperceptible. This illustrates that the low-frequency features are important to human perception.

Low-frequency Fourier features correspond to an image’s basic structure and content, shaping its overall form and object layout, while high-frequency features highlight detailed aspects and textures. Despite the importance of low-frequency features in human perception, (Yin et al., 2019) shows that CNN models rely on both high and low-frequency features for effective classification. Models trained naturally are more susceptible to additive noise in higher frequencies. In contrast, models trained with adversarial training or Gaussian data augmentation exhibit enhanced robustness to higher frequencies, albeit with a compromise in lower frequency robustness. This indicates a notable trade-off during training that significantly influences the model’s robustness during inference. In addition, we observe that attacks at the lowest frequency are often ineffective, corresponding to LF needing a larger $[rd]$ for attacks to be more successful. In our black-box hard-label setting, we are not be able to access details on how the model was trained. Instead, we treat the selection of low vs. high-frequency as another hyperparameter f_0 to our algorithm; allowing the algorithm to target the CNN

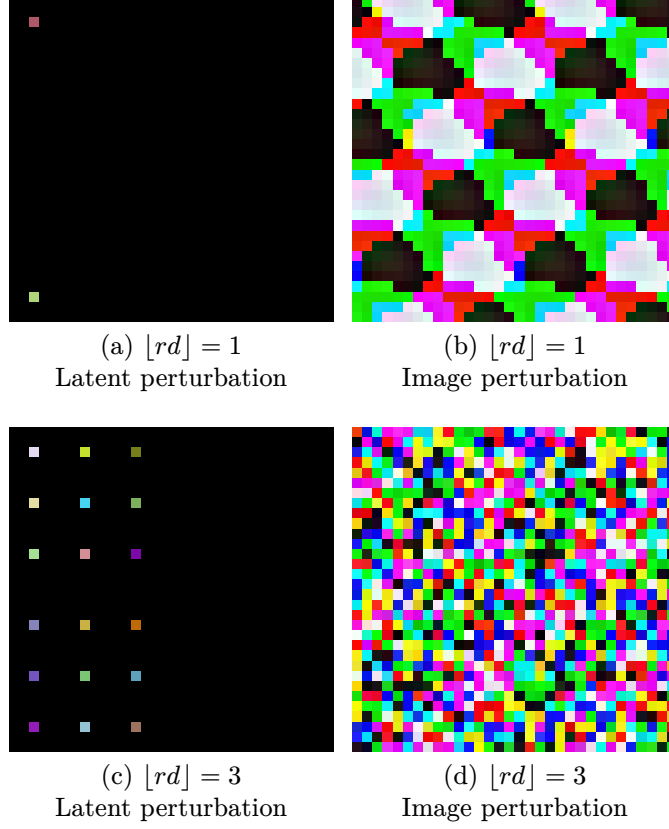


Figure 5.6. Example latent perturbation and its corresponding image perturbation produced by FSFT with $f_0 = (2, 2)$ and various $\lfloor rd \rfloor$.

at its *most vulnerable* frequency (or multiple such frequencies).

In Algorithm 5.3, we consider attack on an image X_i with size $d \times d$ with l color channels and a low-dimensional subspace $\lfloor rd \rfloor \times \lfloor rd \rfloor$ for every color channel and every quadrant (Q1, Q3). Let f_0 be another hyperparameter, the starting frequency relative to zero within each quadrant to start the attack, and $\lfloor rd \rfloor$ is a hyperparameter that control the size of the latent perturbation for every quadrant. To facilitate ease of implementation and experimentation, we also make the assumption of $f_x = f_y$ for simplicity, but this can be relaxed. We segment latent perturbation δ'_t across the quadrants, and we let δ'_q represent the latent perturbation for quadrant q . For every color channel and every quadrant, we evenly space the perturbation δ'_q over the rest of the quadrant, starting from frequency f_0 . In the case where δ'_q is too large to space over the rest of the Fourier space quarter, we will start at an earlier frequency such that f_0 will be part of the attack. We illustrate this process in Figure 5.6.

Algorithm 5.3: FSFT(δ'_t)

input : Latent perturbation $\delta'_t \in [0, 1]^{\lfloor rd \rfloor \times \lfloor rd \rfloor \times l \times 2}$
output: Image perturbation $\delta_t \in [0, 1]^{d \times d \times l}$
1 Initialize image perturbation $\delta_t \leftarrow [\vec{0}]_{d \times d \times l}$
2 **for** $i = 1, \dots, l$ **do** // i indexes channels
3 **for** $q = 1, 3$ **do** // q indexes quadrants
4 // $[:, :]$ indicates all pixels
5 Set quadrant q perturbation $\delta'_q \leftarrow \delta'_t[:, :][i][q]$
6 From f_0 , evenly space δ'_q over the quadrant q
7 Fourier transform δ'_q and set to δ_t
8 **return** δ_t

5.4.1 Simplified-Frequency-Specific Fourier Transform

With the aim of enhancing simplicity and practicality, we introduce a streamlined version of FSFT, referred to as the Simplified-Frequency-Specific Fourier Transform (SFSFT). In the SFSFT, we target one specific frequency, *i.e.*, $\lfloor rd \rfloor = 1$, $f_0 = (f_x, f_y)$ within all quadrants (Q1, Q3) for attack, eliminating the need to evenly space the perturbation across the quadrant. Here, f_x and f_y can be different (*e.g.*, $f_0 = (2, 3)$) and we fix the coordinate system across Q1, Q3. In addition, any non-unique frequencies are filled in with their respective conjugates. Notably, within this framework, the latent perturbation δ'_t maintains a small size of $l \times 2$. Figure 5.7 shows that if the best frequency is chosen, attack on a single frequency is highly effective.

5.5 Tuning Hyperparameters During Attack

Our primary goal thus far is to ensure query efficiency and successful attacks using the chosen hyperparameters, with hyperparameters typically optimized *separately*. In the previous sections, our interest was in a scenario involving attacking a CNN classifying an image $\mathbf{x}_i$ with a total query budget of N , perturbation budget ϵ , and given hyperparameter θ - *e.g.*, latent image size $\lfloor rd \rfloor$ for Tiling. Here, we describe a distinct yet related objective, namely, finding an image perturbation δ_t while also *optimizing for the best hyperparameter* θ . In more detail, we consider identifying the optimal attack hyperparameter θ^* within set of possible hyperparameters $\Theta = \{\theta_1, \dots, \theta_k\}$ given M images to at-

tack $\mathbf{X} = \{\mathbf{x}_1, \dots, \mathbf{x}_M\}$. The optimal hyperparameter θ^* is the hyperparameter that achieves the maximum success rate averaged over these images (See Section 5.6 for definition). We describe two methods of tuning the hyperparameter while performing the attack for Tiling and SFSFT respectively.

5.5.1 Bandit Hyperparameter Search

Multi-armed bandit (MAB) techniques have been successfully used to perform hyperparameter optimization, *e.g.*, (Bouneffouf, Rish, & Aggarwal, 2020), and we apply such techniques to identify the best hyperparameter while the attack progresses.

The bandit agent can optimize the hyperparameter for *any* underlying attack method – *e.g.*, Algorithm 5.1 or an adversary choosing the latent perturbation randomly (*i.e.*, Random Adversary – Rng-Adv). All transforms that solely depend on the discretized hyperparameter (*e.g.*, $\lfloor rd \rfloor$) are applicable, *i.e.*, we are able to optimize Tiling’s hyperparameter, noting that it can also be applied to NNI and LF. Our setup is as follows: Each attack using a particular hyperparameter θ_i is treated as an independent arm (i -th arm) pull on a k -armed Bernoulli Bandit. We model the success/failure of the adversarial attack with hyperparameter θ_i as a Bernoulli variable and we learn this as we attack the various images. The strategy applied to select a particular arm is known as the bandit algorithm, we consider various commonly used algorithms in the literature:

- Thomson Sampling (TS-Bandit) - Treats the success rate of an adversarial attack as a probabilistic model, utilizing samples from a Beta distribution to guide decision-making.
- Upper Confidence Bound (UCB-Bandit) - Achieves a balance between exploration (high variance) and exploitation (high means) through the use of an exploration factor c .
- ϵ -greedy (ϵ -Greedy-Bandit) - Chooses a random action with ϵ probability, otherwise a greedy action with $1 - \epsilon$ probability.

Since these algorithms are very well known, we refer the reader to (Slivkins, 2019) for more details.

Algorithm 5.4: BANDIT($\Theta, \mathbf{X}$)

input: Set of possible hyperparameters Θ
Images $\mathbf{X}$ to attack

- 1 Initialize bandit $\mathbf{B}_0$
- 2 **for** $j = 1, \dots, M$ **do**
- 3 Sample an arm $\theta_j \in \Theta$ using $\mathbf{B}_{j-1}$
- 4 Observe $r_j \leftarrow \text{BAYES-ATTACK}(\mathbf{x}_j \mid \theta_j)$
- 5 Given observation r_j , update $\mathbf{B}_j$

The process is as follows: For each image $\mathbf{x}_j$, the process begins by sequentially selecting an arm θ_j using a bandit agent based on the bandit. This arm selection is accomplished through the bandit agent; for Bayesian methods like TS-Bandit, we sample from a Beta distribution. Subsequently, the attacker attacks the image $\mathbf{x}_j$ using the chosen hyperparameter θ_j with Algorithm 5.1, *i.e.*, $\text{BAYES-ATTACK}(\mathbf{x}_j \mid \theta_j)$. The outcome of the attack, serves as the reward for updating bandit agent. For TS-Bandit, this entails updating distribution parameters (*e.g.*, a and b of the Beta distribution) given the new observation. For other methods, this entails updating of their respective values. This iterative process allows for the optimization of hyperparameters during the adversarial attack, aiming to improve the overall success rate of the attack strategy; High-level pseudo-code is given in in Algorithm 5.4.

5.5.2 Bayesian Optimization Hyperparameter Search

In the case of FSFT, bandit techniques are not appropriate due to the substantial number of arms. For example, even in the simplified SFSFT, this number of arms depends on the resolution of the image. When dealing with a large number of arms, the task of exploring all possibilities becomes infeasible for the technique described above, making it challenging to identify a good arm (Y. Wang, Audibert, & Munos, 2008). We focus on SFSFT, where the task is to select the best hyperparameter f_0 during the attack.

We plot the success rate for each frequency f_0 used in attacks on both MNIST and CIFAR-10 datasets. The results are shown in Figure 5.7. We observe that the success rate is relatively smooth across the variables f_x, f_y , allowing us to use GP to model the success rate across the frequencies as the attack progresses.

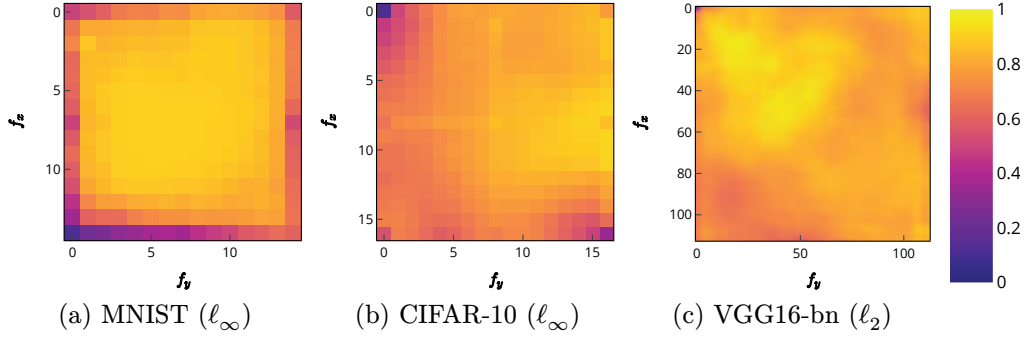


Figure 5.7. Heat map shows SFSFT’s success rate across frequencies $f_0 = (f_x, f_y)$, with lighter (yellow) shades representing higher success rates and darker (purple) shades indicating lower rates.

The process is similar to BO, using it to sequentially pick the next best frequency. We employ established methods, modeling the success rate across the (f_x, f_y) frequencies with the Matérn $1/2$ kernel. For the acquisition function, we utilize the L-BFGS algorithm to efficiently optimize the commonly applied EI. In this manner, we are able to leverage on BO’s sample efficiency to sequentially optimize f_0 . It is important to clarify that the BO mentioned here differs from the BO used in BO-Adv – the BO mentioned here is used to model the hyperparameter, while the BO mentioned in BO-Adv is used to model the latent perturbation.

5.6 Experiments

Our experiments² encompass both attacks constrained by ℓ_∞ and ℓ_2 norms, focusing on hard-label, black-box, and untargeted attacks. We employ standard datasets, including MNIST (Lecun, Bottou, Bengio, & Haffner, 1998), CIFAR-10 (Krizhevsky, 2009), and ImageNet (Deng et al., 2009). We seek to ensure a consistent comparison with established hard-label techniques, employing the same CNN architectures and parameters as described in prior works (Cheng et al., 2019, 2020; Shukla et al., 2021) for MNIST and CIFAR-10; we target pre-trained models for ImageNet, which include ResNet50 (K. He, Zhang, Ren, & Sun, 2016), Inception-v3 (Szegedy, Vanhoucke, Ioffe, Shlens, & Wojna, 2016), and VGG16-bn (Simonyan & Zisserman, 2015) CNN models. More experiments

²Source code available at <https://github.com/eric-vader/Adversarial>.

can be found in the Appendix C.

Primarily, we compare our methods to NNI and LF for ℓ_∞ and ℓ_2 norms respectively, noting that the experiments in (Shukla et al., 2021) already compared NNI and LF against a variety of other methods and showed generally superior performance. We also compare our methods against Sign-Opt and Sq-Attack. Both Sign-Opt and Sq-Attack are methods that focus on query efficiency, providing the capability to specify a predefined query budget (Andriushchenko, Croce, Flammarion, & Hein, 2020b; Cheng et al., 2020). We note that all methods adhere to a hard-label setting, except for Sq-Attack, which has an advantage, benefiting from access to soft-label information.

Alongside the Bayesian Optimization agent BO-Adv, we also conducted experiments with a random adversary (Rng-Adv), which selects latent perturbations randomly. In Rng-Adv, perturbations are chosen using the same random sampling method as the initial samples.

In the context of bandits, we adapted widely available bandit implementations to suit our specific application and introduced two baselines for comparative analysis. The first baseline is the random bandit (Rng-Bandit), which randomly selects an arm according to a uniform distribution. The second baseline is the optimal bandit (Optimal-Bandit), which always selects the arm with the highest possible reward (*i.e.*, maximum success rate), representing the optimal hyperparameter θ^* .

For the TS-Bandit bandit agent, the beta distribution is initialized with parameters a and b both set to 1, where for other methods, all arms are initialized with a success rate of 0. The UCB-Bandit strategy requires the user to set the exploration parameter, and we use a fixed value of $c = 1$. Lastly, for ϵ -Greedy-Bandit, we set at $\epsilon = 0.1$, which determines the balance between exploration and exploitation during the bandit optimization process.

5.6.1 Experimental Setup

To assess the performance, we randomly sampled a $M = 1000$ images from the test set of CIFAR-10 and MNIST, and from the validation set of ImageNet. We exclude images that are already misclassified by the target network, and

only focus on attacking images initially classified with the correct label. We normalized these images to the $[0, 1]$ range. For all methods, we consider the size of the low-dimensional subspace $\lfloor rd \rfloor \times \lfloor rd \rfloor$ within the range $\lfloor rd \rfloor \in [1, 20]$. We display the methods' performance across the entire range where possible. For methods with extra hyperparameters (*e.g.*, f_0 in FSFT and p_0 in Sq-Attack), where possible, we use standard techniques like Grid Search, to find the best hyperparameters. The other settings and hyperparameters are summarized in Sections C.1, C.2.

Kernel

We choose the Matérn- ν kernel, which encodes prior beliefs about f , is characterized by the form in Equation 1.4. The smoothness parameter ν plays a pivotal role in determining the degree of smoothness exhibited by the functions, with lower values yielding rougher functions and higher values resulting in smoother ones. We use the Matérn kernel with $\nu = 2.5$ to model f in Algorithm 5.1, which corresponds to the smoothness of the learned function being twice differentiable (Shahriari et al., 2016; Snoek et al., 2012). We used Matérn kernel with $\nu = 0.5$ to model the success rate across the (f_x, f_y) frequencies (Section 5.5.2). Consistent with standard approaches in literature, we update the hyperparameters after every observation and use L-BFGS to optimize the widely used EI acquisition function. More details discussed in Chapter 1.

5.6.2 Metrics

Our main goal is to assess our methods' performance and efficiency, particularly under limited query budgets.

Success Rate

We measure the performance of our attack by computing Success-Rate up to time t , which indicates the proportion of successful attacks given a query budget:

$$\text{Success-Rate}(t) = \frac{|\{\mathbf{x}_i \in \mathbf{X}_t \mid F(\mathbf{x}'_i) \neq c_i\}|}{|\mathbf{X}_t|}, \quad (5.2)$$

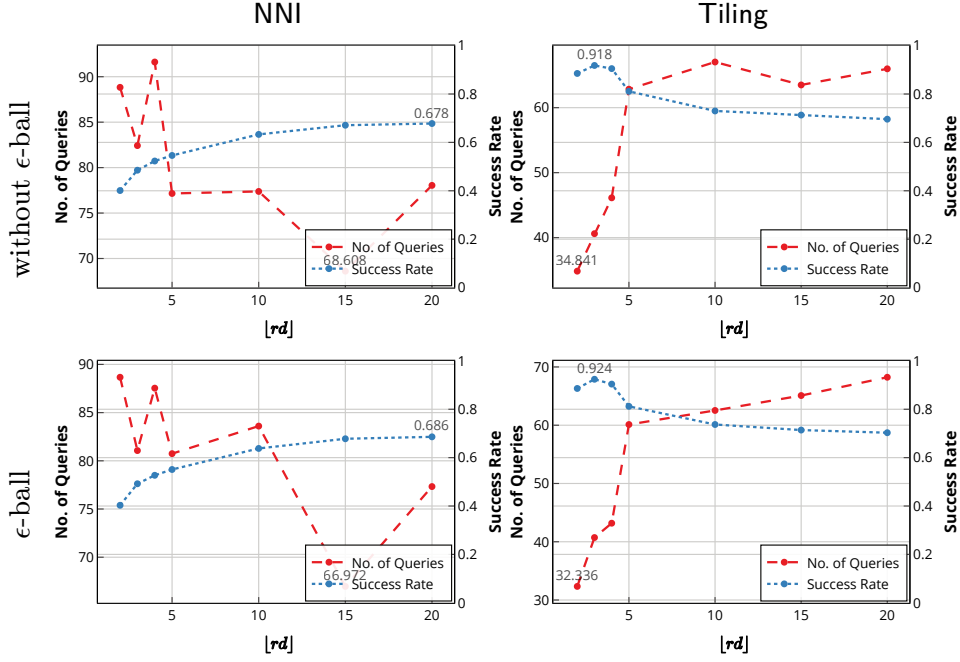


Figure 5.8. Untargeted ℓ_∞ BO-Adv Attack on CIFAR-10, comparing NNI vs Tiling and with vs without ϵ -ball Projection.

where $\mathbf{X}_t = \{\mathbf{x}_1, \dots, \mathbf{x}_t\}$ is the set of randomly sampled images from the dataset up until time t , and $\mathbf{x}'_i = \delta_i + \mathbf{x}_i$ is the perturbed image at iteration i . For untargeted attacks, an attack is deemed successful if the CNN misclassifies the perturbed image $\mathbf{x}'_i$, by assigning it any label other than its true label c_i .

Number of Queries

We measure the efficiency of our method by recording, for every successfully attacked image, the number of queries used to achieve the attack. We note that the number of queries is bounded by the query budget N .

5.6.3 Ablation Experiments for Tiling

To demonstrate the individual contributions of the two improvements in the ℓ_∞ setting, we toggled each improvement separately – ϵ -ball Projection and Tiling; when ϵ -ball Projection is turned off (w/o ϵ -ball), where there is no optimization. When Tiling is turned off, we revert back to using NNI. We do this to gain better understanding of Tiling and ϵ -ball, and their contributions to improving performance and efficiency. From Figure C.4, ϵ -ball Projection improves both NNI and Tiling as it makes full use of the perturbation budget. Similarly, Tiling not

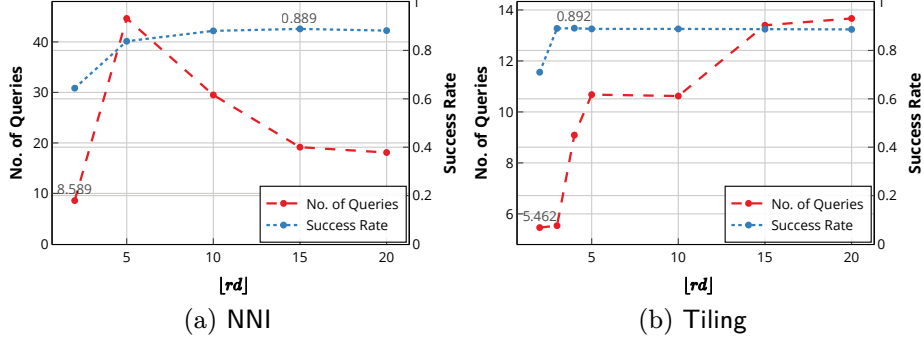


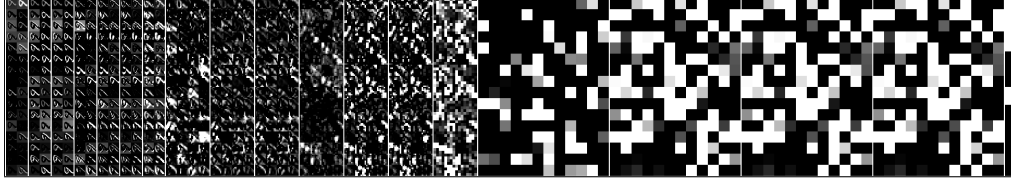
Figure 5.9. Untargeted ℓ_∞ BO-Adv Attack on MNIST.

only improves the attack but also reduces the number of queries required. Tiling, when applied with ϵ -ball Projection, demonstrates superior results; subsequently, we refer to this.

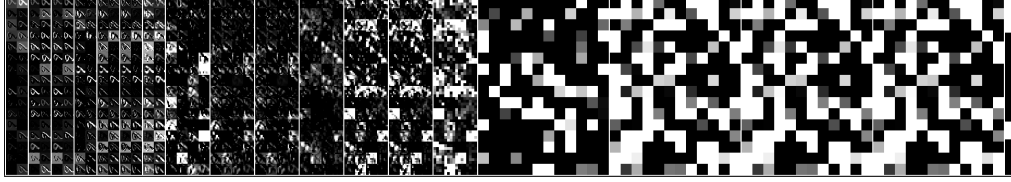
5.6.4 Untargeted ℓ_∞ Attack Experiments

From Figures C.4, 5.9, we see that our Tiling outperforms NNI for both MNIST and CIFAR-10, achieving better success rates, using fewer queries at lower latent dimensions. Tiling outperforms NNI at every latent dimension $[rd]$, achieving higher success rate while using fewer queries. Even considering the best results across different latent dimensions $[rd]$, Tiling is still superior; for CIFAR-10, the highest success rate is achieved at $[rd] = 20$ for NNI, whereas for Tiling the highest success rate is higher and is achieved at a lower $[rd] = 3$ while using drastically fewer queries.

Moreover, we observe that Tiling performs better with lower-dimensionality, a smaller $[rd]$ would mean that the tiles are smaller, attacking the lower feature maps would trickle the attack deeper to higher feature maps. In addition, perturbations with lower $[rd]$ are more susceptible to shift-invariant effects; a smaller tile would have more opportunities to activate same vulnerability across the image. Figure 5.10 contrasts activations across CNN layers in attacked and non-attacked scenarios. Notably, in the attack case, activations from around the 7th layer (counting from the left, input layer) differ significantly from those in the non-attacked state. These altered activations propagate to deeper layers, disrupting the detection of more abstract features and ultimately leading to



(a) Without perturbation – Digit ‘9’ becomes increasingly abstract as the signals propagate through the network.



(b) Incorrect classification after applying perturbation from Tiling, targeting lower-level feature detection.

Figure 5.10. Intermediate feature maps across CNN layers for digit ‘9’ in MNIST; from left (input layers with low-level features) to right (higher-level features), ending rightmost with outputs for C classes.

erroneous output.

5.6.5 Untargeted ℓ_2 Attacks Experiments

Figure 5.11 shows a clear trend of FSFT outperforming LF in ImageNet, echoing (Yin et al., 2019)’s emphasis on the relevance of both *high* and *low* frequency features. By allowing the attack to target arbitrary areas in the frequency space, FSFT achieves a higher success rates but also needing fewer queries and at an exceptionally low-dimensionality of $\lfloor rd \rfloor = 1$ in the latent space. This significantly bolsters the efficiency and effectiveness of the adversarial attack, affording FSFT the ability to scale the attack effectively to larger images; this effectiveness also validates SFSFT in targeting single specific frequencies.

5.6.6 Bandit Hyperparameter Search Experiments

Here, we allow the bandit to dynamically select the hyperparameter as we attacks a dataset of $M = 8000$ images. For instance, the bandit may initiate an attack on the first image with $\lfloor rd \rfloor = 1$ and on the subsequent image with $\lfloor rd \rfloor = 12$, progressively learning from the feedback and optimizing its hyperparameter selections. Repeating the experiment 25 times with varied image sets allows for a reliable evaluation of our approach in diverse scenarios, ensuring robust testing

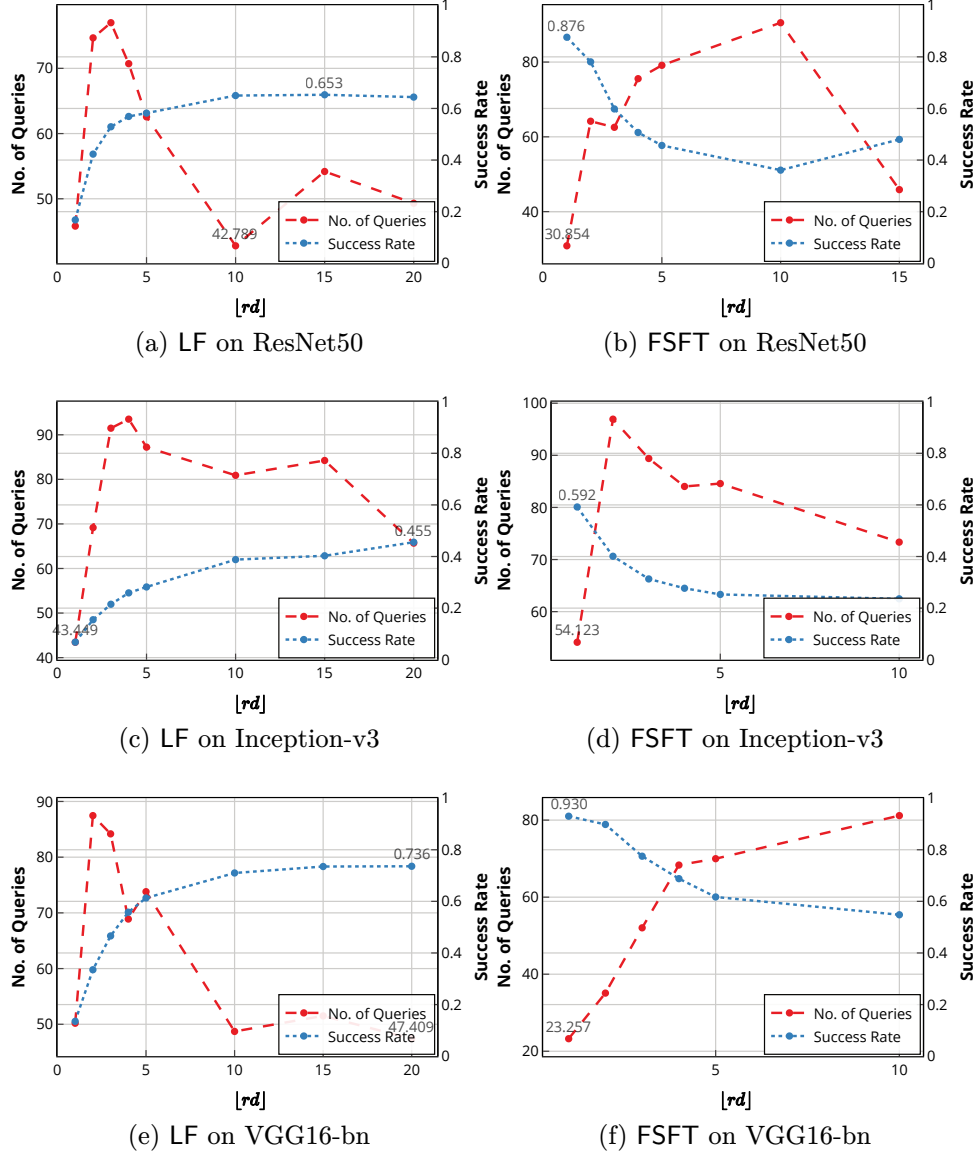


Figure 5.11. Untargeted ℓ_2 BO-Adv Attack on ImageNet CNNs.

of our bandit-based hyperparameter selection.

According to the results shown in Figure 5.12b, TS-Bandit achieves a notable success rate of 90.97% while using only 40.8 adversarial queries per image. In comparison, the outcomes obtained using **Optimal-Bandit**, which utilizes the optimal arm $[rd] = 3$, resulted in an average of 38.33 adversarial queries per image, achieving only slightly higher success rate of 91.5%. This comparison illustrates that our bandit-based approach effectively identifies the optimal arm $[rd] = 3$ during the adversarial attacks, leading to improved performance over time without the necessity of separate hyperparameter optimization.

Overall, both TS-Bandit and UCB-Bandit exhibit strong performance, achiev-

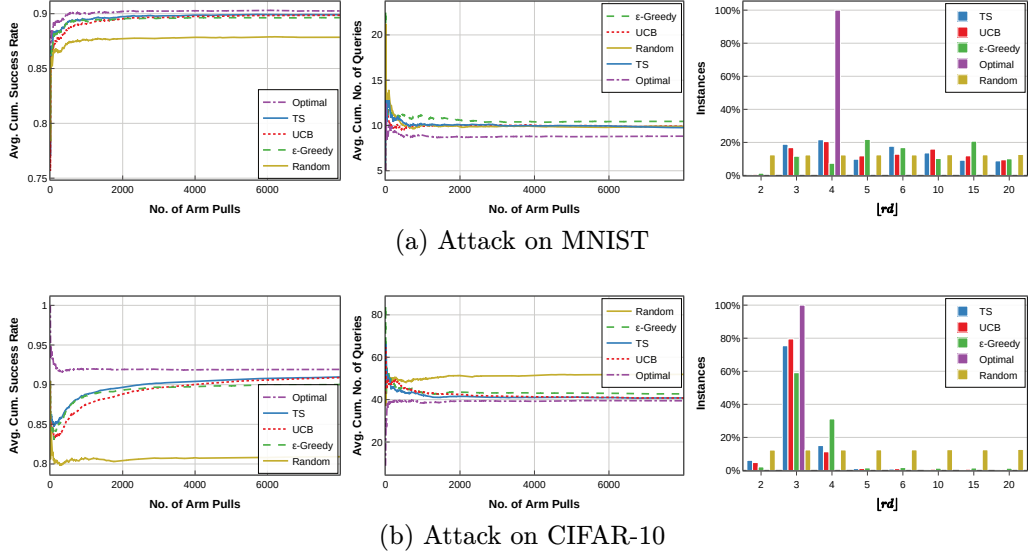


Figure 5.12. Bandit hyperparameter search for the best arm (*i.e.*, $[rd]$) with the best reward for untargeted ℓ_∞ Rng-Adv Attacks using Tiling.

ing high success rates while minimizing the usage of queries. Specifically, for the CIFAR-10 dataset, UCB-Bandit displays a marginal advantage over TS-Bandit in selecting the optimal θ^* in the long run, as depicted in Figure 5.12b. However, TS-Bandit consistently outperforms UCB-Bandit in the short term, having superior success rates and a lower query count. Notably, UCB-Bandit requires setting of the exploration hyperparameter which balances between exploration and exploitation. In contrast, TS-Bandit inherently maintains this balance by virtue of its random sampling strategy. Although TS-Bandit has the flexibility to choose any arm, as more rewards are observed, it increasingly favors the optimal arm. Both TS-Bandit and UCB-Bandit demonstrate robustness when faced with scenarios lacking a clear best arm, as demonstrated in the MNIST experiment (See Figure 5.12a), where multiple arms perform nearly as well as the optimal one. While ϵ -Greedy-Bandit fares well with a low number of arm pulls, its performance wanes over time. In our specific context, TS-Bandit emerges as the most effective approach, requiring minimal hyperparameter tuning beyond the set of arms Θ .

Overall, our experiment demonstrates that a bandit can optimize the hyperparameters of adversary executing sustained and query-efficient black-box adversarial attacks on a CNN across images without the need for *any* prior knowledge, such as the latent dimension $[rd]$, while operating within the confines of only

hard-label information. Our setting aligns naturally with our broader goal to be query-efficient and not relying on any prior information. Our approach employs bandit-based optimization to dynamically enhance our attacks in real-time, achieving performance levels comparable to those achievable with knowledge of the optimal hyperparameter in the long run.

5.6.7 BO Hyperparameter Search Experiments

We leverage BO to dynamically determine the frequency hyperparameter f_0 during adversarial attacks on a dataset comprising $M = 4000$ images, with a query budget of $N = 500$. This approach enables us to demonstrate the efficacy of our technique under constrained conditions, while maintaining consistent experimental parameters elsewhere; note that these attacks are on CNNs from scratch without any prior knowledge or access to the model.

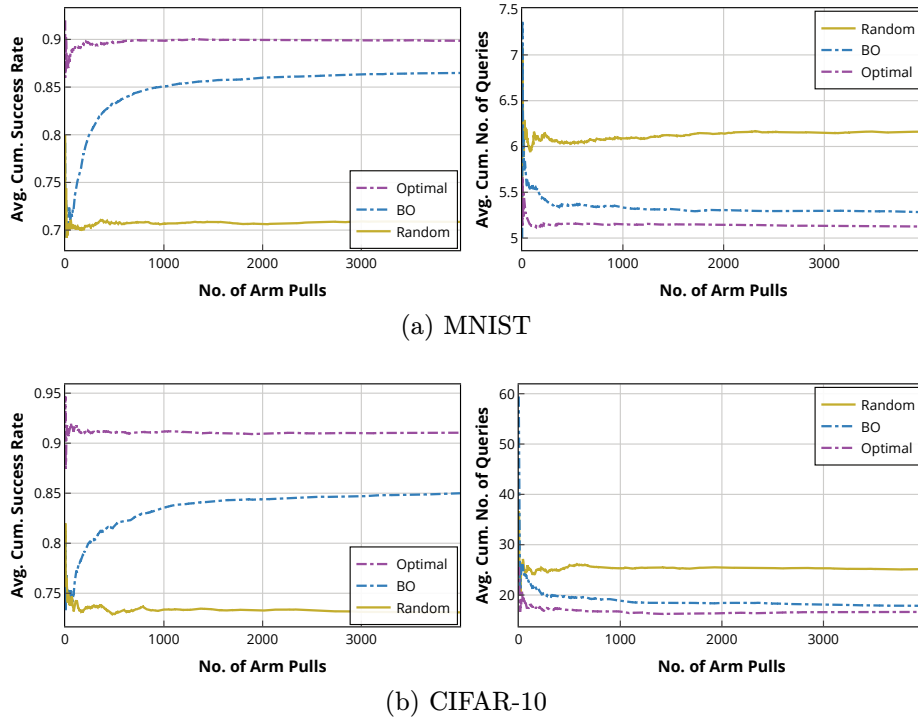


Figure 5.13. BO hyperparameter search for the best frequency, f_0 , with the best performance for untargeted ℓ_∞ Rng-Adv Attacks using SFSFT.

The effectiveness of our proposed technique is evident from Figure 5.13. The average cumulative success rate and number of queries attained through BO progressively converge towards an optimal outcome, even in the context of a diminished query budget of $N = 500$ and a dataset comprising $M = 4000$ images.

Using a larger query budget and dataset, as done in our bandit experiments, could potentially boost the performance even further.

5.6.8 Comparisons with Other Methods

While our attacks are tailored for specific ℓ_∞ or ℓ_2 settings, we show that Tiling, SFSFT, NNI, and LF can be adapted to other ℓ_p settings through suitable perturbation projection. Our analysis focuses on query budgets up to 1000, assessing the methods’ efficacy in low-query regimes. Figure 5.14 reveals that our methods (Tiling or SFSFT) either match or surpass others in the hard-label setting, while also requiring notably fewer queries and operating in smaller latent spaces (*e.g.*, 4 dimensions for SFSFT on MNIST). Our methods outperform Sq-Attack in lower budget scenarios, despite its use of more queries and soft-label information, achieving similar results.

Analysis of Table 5.1 reveals that SFSFT and Tiling function in the smallest and second smallest latent spaces, respectively, compared to other methods. We note that Sq-Attack considers different subset of pixels in every iteration, we consider the largest subset of pixels perturbed as its latent perturbation size.

Setting	Method	Latent Perturbation Size		
		MNIST (ℓ_∞)	CIFAR-10 (ℓ_∞)	VGG16-bn (ℓ_2)
hard-label	Tiling	9	27	12
hard-label	SFSFT	4	6	12
hard-label	NNI	225	1200	1200
hard-label	LF	200	1350	2400
hard-label	Sign-Opt	784	3072	150528
hard-label	Sq-Attack	484	324	7569

Table 5.1. Attack Methods for MNIST, CIFAR-10, and VGG16-bn: Overview of settings and latent perturbation size.

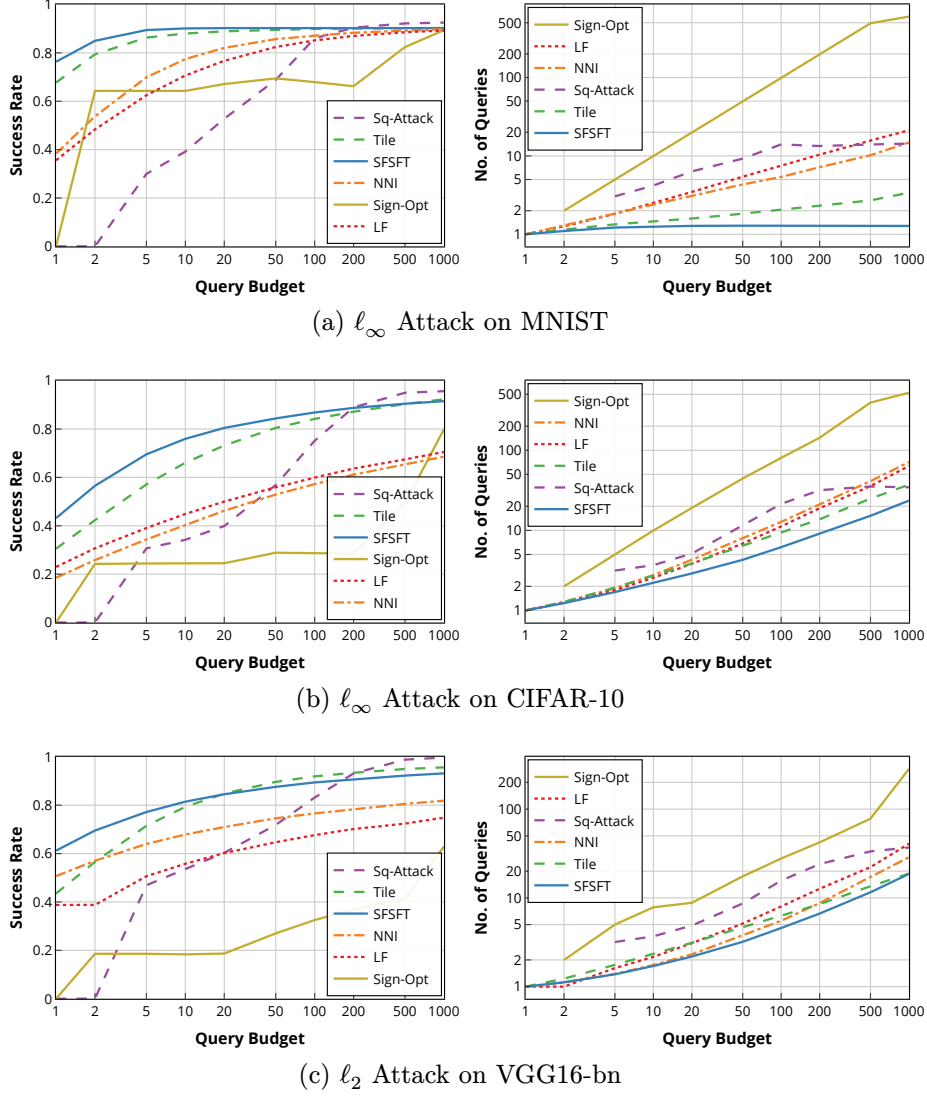


Figure 5.14. Comparison of various methods on various query budgets.

5.7 Conclusion

We have developed adversarial attack techniques for CNNs, incorporating domain knowledge through the Tiling Transform (Tiling) for ℓ_∞ , and Frequency-Specific Fourier Transform (FSFT) for ℓ_2 attacks. We introduced bandit-based approach (Tiling) and applied BO (SFSFT) to finding the best hyperparameter during attack. Our experiments validate the effectiveness of our proposed methods, which outperform other hard-label techniques by achieving higher success rates with fewer queries.

Chapter 6

Conclusion and Future Work

This thesis focuses on global optimization and addresses key challenges inherent in real-world applications. Many real-world problems entail *expensive* to evaluate and *black-box* objective functions, *high dimensionality*, and the demand for *robust* solutions. Bayesian Optimization (BO) offers a potential solution and is shown to be effective where the objective function is *black-box* and *expensive* to evaluate, yet scaling it to *higher dimensions* poses difficulties due to limited data and increased computational demands. With the broader emphasis in Machine Learning on enhancing robustness for dependable real-world deployment, our work enhanced BO’s performance and application in *high-dimensional* settings and improve *robustness*, fostering broader applicability and adoption across diverse domains.

We first proposed a method for *high-dimensional* GP optimization with generalized additive models. We introduced a tree-structured decomposition that balances expressivity with computational efficiency and ease of model learning. Our method efficiently learns the structure using Gibbs Sampling and edge mutation, making it suitable for resource-limited settings in line with BO’s primary motivation. We also presented a zooming-based message passing approach that benefits BO with generalized additive models in continuous domains, with or without tree structures. We demonstrated that our method is competitive on synthetic functions and real datasets, with significantly reduced computation compared to more complex graph structures. Future research directions encompass exploring varied decomposition techniques, investigating alternative learn-

ing methods for these decompositions, and applying the resultant dependency graphs to diverse tasks.

Since publication, there have been several related works in this area. One such work is the orthogonal additive kernel (OAK), proposed by (Lu, Boukouvalas, & Hensman, 2022). This kernel imposes an orthogonality constraint on the additive functions, and the author found that the first and second order orthogonal additive kernel components can account for $> 99\%$ variance in f . Higher order components can absorb effects from lower order components, resulting in a needlessly complex model. This result is similar to the idea that tree-structured decomposition is expressive enough. Another related work is the random decomposition upper-confidence bound algorithm (RDUCB), proposed by (Ziomek & Bou-Ammar, 2023). This algorithm employs data-independent decomposition sampling rules, contrasting with our data-driven approach. The author demonstrated that a random tree-based decomposition sampler strikes a favorable balance between maximal information gain, and the risk of being exploited by an adversary.

We then studied adversarial attacks on GP bandits, focusing on the GP-UCB algorithm and a related elimination algorithm. We provided a theoretical study that states conditions under which an attack can succeed and used this to design various attacks with varying levels of aggression and assumptions on prior knowledge. Our experiments showed that GP-UCB can fail significantly even with a relatively low-budget adversarial attack. Through our publication, which examines the perspective of an attacker, we aim to spur further efforts to bolster BO’s *robustness*. Future research directions, includes further investigating more ‘automated’ attacks that can adapt their hyperparameters online and theoretical lower bounds on the attack budget.

Finally, we proposed an application of BO to perform adversarial attacks on CNNs. We studied *high-dimensional* adversarial attacks using BO on CNNs, proposing two approaches to dimensionality reduction for untargeted attacks in a query-limited setting that efficiently attack the CNN. We introduced the Tiling Transform for untargeted ℓ_∞ attacks, leveraging the shift-invariant nature of features in the initial CNN layers. For untargeted ℓ_2 attacks, we proposed

Frequency-Specific Fourier Transform, enabling the selection of starting frequencies as hyperparameters. In line with our primary goal, we demonstrate the use of bandit techniques for hyperparameter tuning during the attack, as well as the iterative utilization of BO to select optimal frequencies for Frequency-Specific Fourier Transform as the attack progresses. Our goal is to inspire further research aimed at enhancing the *robustness* of CNNs against *high-dimensional* adversarial attacks. Future research directions involve tackling general attacks beyond ℓ_∞, ℓ_2 and, investigating other dimensionality techniques. Another avenue for future work is to optimize both query-efficient, black-box adversarial attacks and the hyperparameter f_0 for SFSFT simultaneously (*i.e.*, modelling f_0 and the perturbation together). In our work, while we considered optimizing the hyperparameters alongside the perturbations, for SFSFT, there exists an opportunity to jointly optimize both f_0 and the latent perturbation.

Our primary aim was to amplify the efficiency of BO in *high-dimensional* contexts and increase its *robustness*, thereby broadening its applicability across various fields. In our three aforementioned contributions, we devised an approach to enhance GP optimization in *high-dimensional* settings, investigated *robustness* concerns through proposing practical adversarial attacks on GP bandits, and introduced an application of BO for performing *high-dimensional* adversarial attacks (*robustness*) on CNNs. In conclusion, we have explored and presented various **Bayesian Optimization techniques for high-dimensional and adversarial settings**. Tackling challenges stemming from these contexts is crucial for the effective application of BO in practical scenarios, and numerous avenues for further research remain open.

Bibliography

- Akkoyunlu, E. A. (1973). The enumeration of maximal cliques of large graphs. *SIAM J. Comput.*, 2, 1–6.
- Alzantot, M., Sharma, Y., Chakraborty, S., Zhang, H., Hsieh, C.-J., & Srivastava, M. B. (2019). GenAttack: practical black-box attacks with gradient-free optimization. *Proceedings of the Genetic and Evolutionary Computation Conference*, 1111–1119. Prague, Czech Republic: GECCO.
- Anaconda. (2016). Anaconda Software Distribution. Anaconda Inc.
- Andriushchenko, M., Croce, F., Flammarion, N., & Hein, M. (2020a). Square attack: A query-efficient black-box adversarial attack via random search. *16th European Conference on Computer Vision, 12368*, 484–501. Glasgow, UK: ECCV.
- Andriushchenko, M., Croce, F., Flammarion, N., & Hein, M. (2020b). Square attack: A query-efficient black-box adversarial attack via random search. *Computer Vision - ECCV 2020 - 16th European Conference*. Springer.
- Arnborg, S., Corneil, D. G., & Proskurowski, A. (1987). Complexity of finding embeddings in a k -tree. *Siam Journal on Algebraic and Discrete Methods*, 8, 277–284.
- Athalye, A., Carlini, N., & Wagner, D. A. (2018). Obfuscated Gradients Give a False Sense of Security: Circumventing Defenses to Adversarial Examples. *Proceedings of the 35th International Conference on Machine Learning*, 80, 274–283. Stockholmsmässan, Stockholm, Sweden: ICML.
- Athalye, A., Engstrom, L., Ilyas, A., & Kwok, K. (2018). Synthesizing Robust Adversarial Examples. *Proceedings of the 35th International Conference on Machine Learning*, 80, 284–293. Stockholmsmässan, Stockholm, Sweden: ICML.

- Auer, P. (2002). Using Confidence Bounds for Exploitation-Exploration Trade-offs. *Journal of Machine Learning Research*, 3, 397–422.
- Ayadi, A., Ghorbel, O., Obeid, A. M., & Abid, M. (2017). Outlier detection approaches for wireless sensor networks: A survey. *Comput. Networks*, 129, 319–333.
- Balandat, M., Karrer, B., Jiang, D. R., Daulton, S., Letham, B., Wilson, A. G., & Bakshy, E. (2020). BoTorch: A framework for efficient monte-carlo bayesian optimization. *Advances in Neural Information Processing Systems* 33. virtual: NeurIPS.
- Beland, J. J., & Nair, P. B. (2017). Bayesian optimization under uncertainty. *NIPS BayesOpt Workshop*, 2.
- Bergstra, J., Bardenet, R., Bengio, Y., & Kégl, B. (2011). Algorithms for Hyper-Parameter Optimization. *Advances in Neural Information Processing Systems* 24, 2546–2554. Granada, Spain: NeurIPS.
- Berkelaar, M., Eikland, K., & Notebaert, P. (2004). *lp_solve 5.5, Open source (Mixed-Integer) Linear Programming system*. Software.
- Bertsimas, D., Nohadani, O., & Teo, K. M. (2010). Nonconvex Robust Optimization for Problems with Constraints. *INFORMS J. Comput.*, 22, 44–58.
- Bietti, A., & Mairal, J. (2019). Group invariance, stability to deformations, and complexity of deep convolutional representations. *Journal of Machine Learning Research*, 20, 1–49.
- Biggio, B., Corona, I., Maiorca, D., Nelson, B., Srndic, N., Laskov, P., ... Roli, F. (2013). *Evasion Attacks against Machine Learning at Test Time*. 8190, 387–402.
- Biggio, B., Nelson, B., & Laskov, P. (2012). Poisoning Attacks against Support Vector Machines. *Proceedings of the 29th International Conference on Machine Learning*. Edinburgh, Scotland, UK: ICML.
- Bogunovic, I., Krause, A., & Scarlett, J. (2020). Corruption-Tolerant Gaussian Process Bandit Optimization. *The 23rd International Conference on Artificial Intelligence and Statistics*, 108, 1071–1081. Palermo, Sicily, Italy: AISTATS.
- Bogunovic, I., Losalka, A., Krause, A., & Scarlett, J. (2021). Stochastic Linear

- Bandits Robust to Adversarial Attacks. *The 24th International Conference on Artificial Intelligence and Statistics*, 130, 991–999. Virtual Event: AIS-TATS.
- Bogunovic, I., Scarlett, J., Jegelka, S., & Cevher, V. (2018). Adversarially Robust Optimization with Gaussian Processes. *Advances in Neural Information Processing Systems 31*, 5765–5775. Montréal, Canada: NeurIPS.
- Bouneffouf, D., Rish, I., & Aggarwal, C. C. (2020). Survey on applications of multi-armed and contextual bandits. *IEEE Congress on Evolutionary Computation*, 1–8. Glasgow, United Kingdom: CEC.
- Brendel, W., Rauber, J., & Bethge, M. (2018). Decision-Based Adversarial Attacks: Reliable Attacks Against Black-Box Machine Learning Models. *6th International Conference on Learning Representations*. Vancouver, British Columbia, Canada: ICLR.
- Broyden, C. G. (1970). The convergence of a class of double-rank minimization algorithms 1. General considerations. *IMA Journal of Applied Mathematics*, 6, 76–90.
- Brunner, T., Diehl, F., Truong-Le, M., & Knoll, A. C. (2019). Guessing Smart: Biased Sampling for Efficient Black-Box Adversarial Attacks. *2019 IEEE/CVF International Conference on Computer Vision*, 4957–4965. Seoul, Korea (South): ICCV.
- Buitinck, L., Louppe, G., Blondel, M., Pedregosa, F., Mueller, A., Grisel, O., ... Varoquaux, G. (2013). API design for machine learning software: Experiences from the scikit-learn project. *CoRR*, *abs/1309.0238*. Retrieved from <http://arxiv.org/abs/1309.0238>
- Bull, A. D. (2011). Convergence rates of efficient global optimization algorithms. *Journal of Machine Learning Research*, 12, 2879–2904.
- Cai, X., Gomes, S., & Scarlett, J. (2021). Lenient Regret and Good-Action Identification in Gaussian Process Bandits. *Proceedings of the 38th International Conference on Machine Learning*, 139, 1183–1192. Virtual Event: ICML.
- Cai, X., & Scarlett, J. (2021). On Lower Bounds for Standard and Robust Gaussian Process Bandit Optimization. *Proceedings of the 38th International Conference on Machine Learning*, 139, 1216–1226. Virtual Event: ICML.

- Cano, A., & Moral, S. (1994). Heuristic Algorithms for the Triangulation of Graphs. *Proceedings of the 5th International Conference on Processing and Management of Uncertainty in Knowledge-Based Systems, 945*, 98–107. Paris, France: IPMU.
- Carlini, N., & Wagner, D. (2017). Towards evaluating the robustness of neural networks. *2017 IEEE Symposium on Security and Privacy (SP)*, 39–57.
- Chen, B., Castro, R. M., & Krause, A. (2012). Joint Optimization and Variable Selection of High-dimensional Gaussian Processes. *Proceedings of the 29th International Conference on Machine Learning*. Edinburgh, Scotland, UK: ICML.
- Chen, P.-Y., Zhang, H., Sharma, Y., Yi, J., & Hsieh, C.-J. (2017). ZOO: Zeroth order optimization based black-box attacks to deep neural networks without training substitute models. *Proceedings of the 10th ACM Workshop on Artificial Intelligence and Security*, 15–26. Dallas, TX, USA: CCS.
- Chen, Y., Huang, A., Wang, Z., Antonoglou, I., Schrittwieser, J., Silver, D., & Freitas, N. de. (2018). Bayesian Optimization in AlphaGo. *CoRR*, *abs/1812.06855*. Retrieved from <http://arxiv.org/abs/1812.06855>
- Cheng, M., Le, T., Chen, P.-Y., Zhang, H., Yi, J., & Hsieh, C.-J. (2019). Query-efficient hard-label black-box attack: An optimization-based approach. *7th International Conference on Learning Representations*. New Orleans, LA, USA: ICLR.
- Cheng, M., Singh, S., Chen, P. H., Chen, P.-Y., Liu, S., & Hsieh, C.-J. (2020). Sign-OPT: A query-efficient hard-label adversarial attack. *8th International Conference on Learning Representations*. Addis Ababa, Ethiopia: ICLR.
- Chowdhury, S. R., & Gopalan, A. (2017a). On Kernelized Multi-armed Bandits. *Proceedings of the 34th International Conference on Machine Learning, 70*, 844–853. Sydney, NSW, Australia: ICML.
- Chowdhury, S. R., & Gopalan, A. (2017b). On kernelized multi-armed bandits. *Proceedings of the 34th International Conference on Machine Learning, 70*, 844–853. Sydney, NSW, Australia: ICML.
- Contal, E., Buffoni, D., Robicquet, A., & Vayatis, N. (2013). Parallel Gaussian Process Optimization with Upper Confidence Bound and Pure Exploration.

- Machine Learning and Knowledge Discovery in Databases*, 8188, 225–240. Prague, Czech Republic: PKDD.
- Cooley, J. W., & Tukey, J. W. (1965). An algorithm for the machine calculation of complex fourier series. *Mathematics of Computation*, 19, 297–301.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms, 3rd Edition*. MIT Press.
- Cui, J., Yang, B., & Hu, X. (2019). Deep Bayesian Optimization on Attributed Graphs. *The Thirty-Third AAAI Conference on Artificial Intelligence*, 1377–1384. Honolulu, Hawaii, USA: AAAI Press.
- Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., & Fei-Fei, L. (2009). ImageNet: A large-scale hierarchical image database. *Proceedings of the 2009 IEEE Conference on Computer Vision and Pattern Recognition*, 248–255. Miami, Florida, USA: CVPR.
- Djolonga, J., Krause, A., & Cevher, V. (2013). High-Dimensional Gaussian Process Bandits. *Advances in Neural Information Processing Systems 26*, 1025–1033. Lake Tahoe, Nevada, United States: NeurIPS.
- Duraj, A., & Szczepaniak, P. S. (2021). Outlier Detection in Data Streams - A Comparative Study of Selected Methods. *Proceedings of the 25th International Conference*, 192, 2769–2778. Szczecin, Poland: KES.
- Duvenaud, D. (2014). *Automatic model construction with Gaussian processes* (PhD thesis). University of Cambridge.
- Eggenberger, K., Feurer, M., Hutter, F., Bergstra, J., Snoek, J., Hoos, H., et al.others. (2013). Towards an empirical foundation for assessing bayesian optimization of hyperparameters. *NIPS Workshop on Bayesian Optimization in Theory and Practice*, 10.
- Eriksson, D. (2022, February 18). High-dimensional bayesian optimization.
- Eriksson, D., Pearce, M., Gardner, J. R., Turner, R., & Poloczek, M. (2019). Scalable Global Optimization via Local Bayesian Optimization. *Advances in Neural Information Processing Systems 32*, 5497–5508. Vancouver, British Columbia, Canada: NeurIPS.
- Eykholt, K., Evtimov, I., Fernandes, E., Li, B., Rahmati, A., Xiao, C., ... Song, D. (2018). Robust Physical-World Attacks on Deep Learning Visual Classi-

- fication. *Proceedings of the 2018 IEEE Conference on Computer Vision and Pattern Recognition*, 1625–1634. Salt Lake City, UT, USA: CVPR.
- Fletcher, R. (1970). A new approach to variable metric algorithms. *Comput. J.*, *13*, 317–322.
- Galstyan, A., & Cohen, P. R. (2007). Empirical Comparison of “Hard” and “Soft” Label Propagation for Relational Classification. *Proceedings of the 17th International Conference on Inductive Logic Programming*, 4894, 98–111. Corvallis, OR, USA: ILP.
- Garcelon, E., Rozière, B., Meunier, L., Tarbouriech, J., Teytaud, O., Lazaric, A., & Pirotta, M. (2020). Adversarial Attacks on Linear Contextual Bandits. *Advances in Neural Information Processing Systems 33*. virtual: NeurIPS.
- Genton, M. G. (2001). Classes of Kernels for Machine Learning: A Statistics Perspective. *Journal of Machine Learning Research*, *2*, 299–312.
- Gleixner, A. M., Hendel, G., Gamrath, G., Achterberg, T., Bastubbe, M., Berthold, T., ... Shinano, Y. (2021). MIPLIB 2017: Data-driven compilation of the 6th mixed-integer programming library. *Math. Program. Comput.*, *13*, 443–490.
- Goldfarb, D. (1970). A family of variable-metric methods derived by variational means. *Mathematics of Computation*, *24*, 23–26.
- Goodfellow, I. J., Shlens, J., & Szegedy, C. (2015). Explaining and Harnessing Adversarial Examples. *3rd International Conference on Learning Representations*. San Diego, CA, USA: ICLR.
- GPy. (since 2012). *GPy: A gaussian process framework in python*. <http://github.com/SheffieldML/GPy>.
- Grosse, K., Papernot, N., Manoharan, P., Backes, M., & McDaniel, P. D. (2017). Adversarial Examples for Malware Detection. *22nd European Symposium on Research in Computer Security*, 10493, 62–79. Oslo, Norway: ESORICS.
- Guo, C., Gardner, J. R., You, Y., Wilson, A. G., & Weinberger, K. Q. (2019). Simple Black-box Adversarial Attacks. *97*, 2484–2493.
- Guo, C., Rana, M., Cissé, M., & Maaten, L. van der. (2018). Countering Adversarial Images using Input Transformations. *6th International Conference on Learning Representations*. Vancouver, British Columbia, Canada: ICLR.

- Györfi, L., Kohler, M., Krzyzak, A., & Walk, H. (2002). *A Distribution-Free Theory of Nonparametric Regression*. Springer.
- Harris, C. R., Millman, K. J., Walt, S. J. van der, Gommers, R., Virtanen, P., Cournapeau, D., ... Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, 585, 357–362.
- He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. *Proceedings of the 2016 IEEE Conference on Computer Vision and Pattern Recognition*, 770–778. Las Vegas, NV, USA: CVPR.
- He, Y., Meng, G., Chen, K., Hu, X., & He, J. (2019). Towards privacy and security of deep learning systems: A survey. *CoRR*, *abs/1911.12562*. Retrieved from <http://arxiv.org/abs/1911.12562>
- Hoang, T. N., Hoang, Q. M., Ouyang, R., & Low, K. H. (2018). Decentralized High-Dimensional Bayesian Optimization With Factor Graphs. *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence*, 3231–3238. New Orleans, Louisiana, USA: AAAI Press.
- Hutter, F., Hoos, H. H., & Leyton-Brown, K. (2010). Automated Configuration of Mixed Integer Programming Solvers. *Proceedings of the 7th International Conference of Integration of AI and OR Techniques in Constraint Programming for Combinatorial Optimization Problems*, 6140, 186–202. Bologna, Italy: CPAIOR.
- Hutter, F., Hoos, H. H., & Leyton-Brown, K. (2014). An Efficient Approach for Assessing Hyperparameter Importance. *Proceedings of the 31th International Conference on Machine Learning*, 32, 754–762. Beijing, China: ICML.
- Ilyas, A., Engstrom, L., Athalye, A., & Lin, J. (2018). Black-box Adversarial Attacks with Limited Queries and Information. *Proceedings of the 35th International Conference on Machine Learning*, 80, 2142–2151. Stockholmsmässan, Stockholm, Sweden: ICML.
- Ilyas, A., Engstrom, L., & Madry, A. (2019). Prior Convictions: Black-box Adversarial Attacks with Bandits and Priors. *7th International Conference on Learning Representations*. New Orleans, LA, USA: ICLR.
- Jenatton, R., Archambeau, C., González, J., & Seeger, M. W. (2017). Bayesian Optimization with Tree-structured Dependencies. *Proceedings of the 34th In-*

- ternational Conference on Machine Learning*, 70, 1655–1664. Sydney, NSW, Australia: ICML.
- Jmila, H., & Khedher, M. I. (2022). Adversarial machine learning for network intrusion detection: A comparative study. *Comput. Networks*, 214, 109073.
- Jun, K.-S., Li, L., Ma, Y., & Zhu, X. (Jerry). (2018). Adversarial Attacks on Stochastic Bandits. *Advances in Neural Information Processing Systems 31*, 3644–3653. Montréal, Canada: NeurIPS.
- Juuti, M., Szyller, S., Marchal, S., & Asokan, N. (2019). PRADA: Protecting against DNN model stealing attacks. *2019 IEEE European Symposium on Security and Privacy (EuroS&P)*, 512–527.
- Kandasamy, K., Schneider, J. G., & Póczos, B. (2015). High Dimensional Bayesian Optimisation and Bandits via Additive Models. *Proceedings of the 32nd International Conference on Machine Learning*, 37, 295–304. Lille, France: ICML.
- Kharkovskii, D. (2020). *Automated Machine Learning: New Advances on Bayesian Optimization* (PhD thesis). Department of Computer Science, School of Computing, National University of Singapore.
- Kim, H. (2020). Torchattacks: A PyTorch repository for adversarial attacks. *arXiv Preprint arXiv:2010.01950*.
- Kim, J., & Choi, S. (2020). On local optimizers of acquisition functions in bayesian optimization. *Proceedings of the European Conference on Machine Learning and Knowledge Discovery in Databases*, 12458, 675–690. Ghent, Belgium: PKDD.
- Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. *3rd International Conference on Learning Representations*. San Diego, CA, USA: ICLR.
- Kirschner, J., Mutny, M., Hiller, N., Ischebeck, R., & Krause, A. (2019). Adaptive and Safe Bayesian Optimization in High Dimensions via One-Dimensional Subspaces. *Proceedings of the 36th International Conference on Machine Learning*, 97, 3429–3438. Long Beach, California, USA: ICML.
- Klein, A., & Hutter, F. (2019). Tabular Benchmarks for Joint Architecture and Hyperparameter Optimization. *CoRR*, abs/1905.04970. Retrieved from

<http://arxiv.org/abs/1905.04970>

- Krizhevsky, A. (2009). *Learning Multiple Layers of Features from Tiny Images* (Technical Report No. TR-2009-06). University of Toronto.
- Kurakin, A., Goodfellow, I. J., & Bengio, S. (2017). Adversarial examples in the physical world. *5th International Conference on Learning Representations*. Toulon, France: ICLR.
- Kushner, Harold J. (1962). A versatile stochastic model of a function of unknown and time varying form. *Journal of Mathematical Analysis and Applications*, 5, 150–167.
- Kushner, H. J. (1964). A New Method of Locating the Maximum Point of an Arbitrary Multippeak Curve in the Presence of Noise. *Journal of Basic Engineering*, 86, 97–106.
- LeCun, Y. (2016). Deep learning and the future of ai. CERN Colloquium.
- LeCun, Y., Bengio, Y., & Hinton, G. E. (2015). Deep learning. *Nat.*, 521, 436–444.
- Lecun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86, 2278–2324.
- Lee, Y., Basith, S., & Choi, S. (2018). Recent Advances in Structure-Based Drug Design Targeting Class A G Protein-Coupled Receptors Utilizing Crystal Structures and Computational Simulations. *Journal of Medicinal Chemistry*, 61, 1–46.
- Li, C., Gupta, S., Rana, S., Nguyen, V., Venkatesh, S., & Shilton, A. (2017). High Dimensional Bayesian Optimization using Dropout. *Proceedings of the Twenty-Sixth International Joint Conference on Artificial Intelligence*, 2096–2102. Melbourne, Australia: IJCAI.
- Li, C.-L., Kandasamy, K., Póczos, B., & Schneider, J. G. (2016). High Dimensional Bayesian Optimization via Restricted Projection Pursuit Models. *Proceedings of the 19th International Conference on Artificial Intelligence and Statistics*, 51, 884–892. Cadiz, Spain: AISTATS.
- Li, Z., & Scarlett, J. (2022). Gaussian process bandit optimization with few batches. *Proceedings of the 25th International Conference on Artificial Intelligence and Statistics*, 151, 92–107. AISTATS.

- Liu, D. C., & Nocedal, J. (1989). On the limited memory BFGS method for large scale optimization. *Mathematical Programming*, 45, 503–528.
- Liu, Y., Chen, X., Liu, C., & Song, D. (2017). Delving into Transferable Adversarial Examples and Black-box Attacks. *5th International Conference on Learning Representations*. Toulon, France: ICLR.
- Lu, X., Boukouvalas, A., & Hensman, J. (2022). Additive Gaussian processes revisited. In K. Chaudhuri, S. Jegelka, L. Song, C. Szepesvari, G. Niu, & S. Sabato (Eds.), *Proceedings of the 39th international conference on machine learning* (pp. 14358–14383). ICML.
- Lu, X., Gonzalez, J., Dai, Z., & Lawrence, N. D. (2018). Structured Variationally Auto-encoded Optimization. *Proceedings of the 35th International Conference on Machine Learning*, 80, 3273–3281. Stockholmsmässan, Stockholm, Sweden: ICML.
- Lykouris, T., Mirrokni, V. S., & Leme, R. P. (2018). Stochastic bandits robust to adversarial corruptions. *Proceedings of the 50th Annual ACM SIGACT Symposium on Theory of Computing*, 114–122. Los Angeles, CA, USA: STOC.
- Ma, X., & Blaschko, M. B. (2020). Additive Tree-Structured Covariance Function for Conditional Parameter Spaces in Bayesian Optimization. *The 23rd International Conference on Artificial Intelligence and Statistics*, 108, 1015–1025. Palermo, Sicily, Italy: AISTATS.
- Martinez-Cantin, R., Tee, K., & McCourt, M. (2018). Practical bayesian optimization in the presence of outliers. *Proceedings of the Twenty-First International Conference on Artificial Intelligence and Statistics*, 84, 1722–1731. AISTATS.
- Martins, J. R., & Ning, A. (2021). *Engineering design optimization*. Cambridge University Press.
- Micchelli, C. A., Xu, Y., & Zhang, H. (2006). Universal Kernels. *Journal of Machine Learning Research*, 7, 2651–2667.
- Mitchell, T. M. (1997). *Machine learning, international edition*. McGraw-Hill.
- Mondal, A. K., Chowdhury, S. P., Jayendran, A., Asnani, H., Singla, P., & P., P. A. (2020). MaskAAE: Latent space optimization for Adversarial Auto-Encoders. *Proceedings of the Thirty-Sixth Conference on Uncertainty in*

- Artificial Intelligence*, 124, 689–698. virtual online: UAI.
- Moon, S., An, G., & Song, H. O. (2019). Parsimonious Black-Box Adversarial Attacks via Efficient Combinatorial Optimization. *Proceedings of the 36th International Conference on Machine Learning*, 97, 4636–4645. Long Beach, California, USA: ICML.
- Moriconi, R., Kumar, K. S. S., & Deisenroth, M. P. (2019). High-Dimensional Bayesian Optimization with Manifold Gaussian Processes. *CoRR*, abs/1902.10675. Retrieved from <http://arxiv.org/abs/1902.10675>
- Murphy, K. P. (2012). *Machine learning - a probabilistic perspective*. MIT Press.
- Mutny, M., & Krause, A. (2018). Efficient High Dimensional Bayesian Optimization with Additivity and Quadrature Fourier Features. *Advances in Neural Information Processing Systems 31*, 9019–9030. Montréal, Canada: NeurIPS.
- Nash, S. G. (2000). A survey of truncated-newton methods. *Journal of Computational and Applied Mathematics*, 124, 45–59.
- Naveršnik, K., & Rojnik, K. (2012). Handling Input Correlations in Pharmacoeconomic Models. *Value in Health*, 15, 540–549.
- Nelder, J. A., & Mead, R. (1965). A Simplex Method for Function Minimization. *Comput. J.*, 7, 308–313.
- Nicolae, M.-I., Sinn, M., Tran, M. N., Buesser, B., Rawat, A., Wistuba, M., ... Edwards, B. (2018). Adversarial robustness toolbox v1.2.0. *CoRR*, 1807.01069. Retrieved from <https://arxiv.org/pdf/1807.01069>
- Nicolas Kruchten. (2015). *Visualizing Family Trees*. <http://nicolas.kruchten.com/content/2015/08/family-trees/>.
- Nogueira, J., Martinez-Cantin, R., Bernardino, A., & Jamone, L. (2016). Unscented Bayesian optimization for safe robot grasping. *2016 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 1967–1972. Daejeon, South Korea: IROS.
- Nyitrai, T., & Virág, M. (2019). The effects of handling outliers on the performance of bankruptcy prediction models. *Socio-Economic Planning Sciences*, 67, 34–42.
- Oh, C., Gavves, E., & Welling, M. (2018). BOCK : Bayesian optimization with cylindrical kernels. *Proceedings of the 35th International Conference on*

- Machine Learning*, 80, 3865–3874. Stockholmsmässan, Stockholm, Sweden: ICML.
- Papernot, N., McDaniel, P. D., Goodfellow, I. J., Jha, S., Celik, Z. B., & Swami, A. (2017). Practical Black-Box Attacks against Machine Learning. *Proceedings of the 2017 ACM on Asia Conference on Computer and Communications Security*, 506–519. Abu Dhabi, United Arab Emirates: CCS.
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., ... Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. In *Advances in neural information processing systems 32* (pp. 8024–8035). NeurIPS.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... Duchesnay, Édouard. (2011). Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Rao, K. R., Kim, D. N., & Hwang, J. J. (2010). *Fast fourier transform: Algorithms and applications* (Vol. 32). Springer.
- Rasmussen, C. E., & Williams, C. K. I. (2006). *Gaussian processes for machine learning*. MIT Press.
- Ren, K., Zheng, T., Qin, Z., & Liu, X. (2020). Adversarial Attacks and Defenses in Deep Learning. *Engineering*, 6, 346–360.
- Ribeiro, M., Grolinger, K., & Capretz, M. A. M. (2015). MLaaS: Machine learning as a service. *2015 IEEE 14th International Conference on Machine Learning and Applications (ICMLA)*, 896–902.
- Rolland, P., Scarlett, J., Bogunovic, I., & Cevher, V. (2018). High-dimensional bayesian optimization via additive models with overlapping groups. *Proceedings of the Twenty-First International Conference on Artificial Intelligence and Statistics*, 84, 298–307. AISTATS.
- Ru, B., Cobb, A. D., Blaas, A., & Gal, Y. (2020). BayesOpt Adversarial Attack. *8th International Conference on Learning Representations*. Addis Ababa, Ethiopia: ICLR.
- Russakovsky, O., Deng, J., Su, H., Krause, J., Satheesh, S., Ma, S., ... Fei-Fei, L. (2015). ImageNet Large Scale Visual Recognition Challenge. *International Journal of Computer Vision*, 115, 211–252.

- Russell, S., & Norvig, P. (2020). *Artificial intelligence: A modern approach (4th edition)*. Pearson.
- Sedgewick, R., & Wayne, K. (2016). *Algorithms (Fourth edition deluxe)*. Addison-Wesley.
- Sergeyev, Y. D., & Kvasov, D. E. (2017). *Deterministic global optimization: An introduction to the diagonal approach*. Springer.
- Shahriari, B., Swersky, K., Wang, Z., Adams, R. P., & Freitas, N. de. (2016). Taking the human out of the loop: A review of bayesian optimization. *Proceedings of the IEEE*, 104, 148–175.
- Shanno, D. F. (1970). Conditioning of quasi-newton methods for function minimization. *Mathematics of Computation*, 24, 647–656.
- Sharif, M., Bhagavatula, S., Bauer, L., & Reiter, M. K. (2016). Accessorize to a Crime: Real and Stealthy Attacks on State-of-the-Art Face Recognition. *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, 1528–1540. Vienna, Austria: SIGSAC.
- Sharma, A., Bian, Y., Munz, P., & Narayan, A. (2022). Adversarial patch attacks and defences in vision-based tasks: A survey. *CoRR*, abs/2206.08304. <https://doi.org/10.48550/arXiv.2206.08304>
- Shukla, S. N., Sahu, A. K., Willmott, D., & Kolter, J. Z. (2021). Simple and Efficient Hard Label Black-box Adversarial Attacks in Low Query Budget Regimes. *The 27th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 1461–1469. Singapore: KDD.
- Sieberer, S., McWilliam, S., & Popov, A. A. (2019). Nonlinear electrostatic effects in MEMS ring-based rate sensors under shock excitation. *International Journal of Mechanical Sciences*, 157-158, 485–497.
- Silver, D., Huang, A., Maddison, C. J., Guez, A., Sifre, L., Driessche, G. van den, ... Hassabis, D. (2016). Mastering the game of Go with deep neural networks and tree search. *Nat.*, 529, 484–489.
- Simon-Gabriel, C.-J., Ollivier, Y., Schölkopf, B., Bottou, L., & Lopez-Paz, D. (2018). Adversarial Vulnerability of Neural Networks Increases With Input Dimension. *CoRR*, abs/1802.01421. Retrieved from <http://arxiv.org/abs/1802.01421>

- Simonyan, K., & Zisserman, A. (2015). Very deep convolutional networks for large-scale image recognition. *3rd International Conference on Learning Representations*. San Diego, CA, USA: ICLR.
- Slivkins, A. (2019). Introduction to multi-armed bandits. *Foundations and Trends® in Machine Learning*. <https://doi.org/10.1561/22000000068>
- Snoek, J., Larochelle, H., & Adams, R. P. (2012). Practical Bayesian Optimization of Machine Learning Algorithms. *Advances in Neural Information Processing Systems 25*, 2960–2968. Lake Tahoe, Nevada, United States: NeurIPS.
- Snoek, J., Rippel, O., Swersky, K., Kiros, R., Satish, N., Sundaram, N., ... Adams, R. P. (2015). Scalable Bayesian Optimization Using Deep Neural Networks. *Proceedings of the 32nd International Conference on Machine Learning*, 37, 2171–2180. Lille, France: ICML.
- Sony Corporation. (2021). *Sony Alpha 1 (A1) Mirrorless Camera*. <https://www.sony.com/electronics/interchangeable-lens-cameras/ilce-1>.
- Spruyt, V. (2014). The Curse of Dimensionality in classification. *Computer Vision for Dummies*, 21, 35–40.
- Srinivas, N., Krause, A., Kakade, S. M., & Seeger, M. W. (2010). Gaussian Process Optimization in the Bandit Setting: No Regret and Experimental Design. *Proceedings of the 27th International Conference on Machine Learning*, 1015–1022. Haifa, Israel: ICML.
- Sundaram, S., & Gharesifard, B. (2019). Distributed optimization under adversarial nodes. *IEEE Transactions on Automatic Control*, 64, 1063–1076.
- Suya, F., Tian, Y., Evans, D., & Papotti, P. (2017). Query-limited Black-box Attacks to Classifiers. *CoRR*, *abs/1712.08713*. Retrieved from <http://arxiv.org/abs/1712.08713>
- Szegedy, C., Vanhoucke, V., Ioffe, S., Shlens, J., & Wojna, Z. (2016). Rethinking the inception architecture for computer vision. *Proceedings of the 2016 IEEE Conference on Computer Vision and Pattern Recognition*, 2818–2826. Las Vegas, NV, USA: CVPR.
- Szegedy, C., Zaremba, W., Sutskever, I., Bruna, J., Erhan, D., Goodfellow, I. J., & Fergus, R. (2014). Intriguing properties of neural networks. *2nd*

- International Conference on Learning Representations*. Banff, AB, Canada: ICLR.
- Tramèr, F., Zhang, F., Juels, A., Reiter, M. K., & Ristenpart, T. (2016). Stealing machine learning models via prediction APIs. *25th USENIX Security Symposium, USENIX Security 16, Austin, TX, USA, August 10-12, 2016*, 601–618. USENIX Association.
- Tu, C.-C., Ting, P.-S., Chen, P.-Y., Liu, S., Zhang, H., Yi, J., ... Cheng, S.-M. (2019). AutoZOOM: Autoencoder-Based Zeroth Order Optimization Method for Attacking Black-Box Neural Networks. *The Thirty-Third AAAI Conference on Artificial Intelligence*, 742–749. Honolulu, Hawaii, USA: AAAI Press.
- Tu, J., Li, H., Yan, X., Ren, M., Chen, Y., Liang, M., ... Urtasun, R. (2021). Exploring adversarial robustness of multi-sensor perception systems in self driving. *Conference on Robot Learning, 164*, 1013–1024. London, UK: CoRL.
- Vakili, S., Khezeli, K., & Picheny, V. (2021). On Information Gain and Regret Bounds in Gaussian Process Bandits. *The 24th International Conference on Artificial Intelligence and Statistics, 130*, 82–90. Virtual Event: AISTATS.
- Vo, V. Q., Abbasnejad, E., & Ranasinghe, D. (2022). Query Efficient Decision Based Sparse Attacks Against Black-Box Deep Learning Models. *The Tenth International Conference on Learning Representations*. Virtual Event: ICLR.
- Wainwright, Martin J., Fosson, S. M., & Ravazzi, C. (2015). Graphical models and message-passing algorithms: Some introductory lectures. In *Mathematical foundations of complex networked information systems* (pp. 51–108). Cham: Springer International Publishing.
- Wang, Y., Audibert, J.-Y., & Munos, R. (2008). Algorithms for infinitely many-armed bandits. *Advances in Neural Information Processing Systems 21*, 1729–1736. Vancouver, British Columbia, Canada: Curran Associates, Inc.
- Wang, Zi, Gehring, C., Kohli, P., & Jegelka, S. (2018). Batched large-scale bayesian optimization in high-dimensional spaces. *Proceedings of the Twenty-First International Conference on Artificial Intelligence and Statistics, 84*, 745–754. AISTATS.
- Wang, Zi, & Jegelka, S. (2017). Max-value Entropy Search for Efficient Bayesian Optimization. *Proceedings of the 34th International Conference on Machine*

- Learning*, 70, 3627–3635. Sydney, NSW, Australia: ICML.
- Wang, Zi, Li, C., Jegelka, S., & Kohli, P. (2017). Batched High-dimensional Bayesian Optimization via Structural Kernel Learning. *Proceedings of the 34th International Conference on Machine Learning*, 70, 3656–3664. Sydney, NSW, Australia: ICML.
- Wang, Ziyu, Zoghi, M., Hutter, F., Matheson, D., & Freitas, N. de. (2013). Bayesian Optimization in High Dimensions via Random Embeddings. *Proceedings of the 23rd International Joint Conference on Artificial Intelligence*, 1778–1784. Beijing, China: IJCAI.
- Whitehouse, J., Wu, Z. S., & Ramdas, A. (2023). Improved self-normalized concentration in hilbert spaces: Sublinear regret for GP-UCB. *CoRR*, abs/2307.07539. <https://doi.org/10.48550/arXiv.2307.07539>
- Xiang, Y., Gubian, S., Suomela, B., & Hoeng, J. (2013). Generalized Simulated Annealing for Global Optimization: The GenSA Package. *R J.*, 5, 13.
- Yin, D., Lopes, R. G., Shlens, J., Cubuk, E. D., & Gilmer, J. (2019). A Fourier Perspective on Model Robustness in Computer Vision. *Advances in Neural Information Processing Systems* 32, 13255–13265. Vancouver, British Columbia, Canada: NeurIPS.
- Ying, C., Klein, A., Christiansen, E., Real, E., Murphy, K., & Hutter, F. (2019). NAS-Bench-101: Towards Reproducible Neural Architecture Search. *Proceedings of the 36th International Conference on Machine Learning*, 97, 7105–7114. Long Beach, California, USA: ICML.
- Zaharia, M., Chen, A., Davidson, A., Ghodsi, A., Hong, S. A., Konwinski, A., ... Zumar, C. (2018). Accelerating the Machine Learning Lifecycle with MLflow. *Bulletin of the IEEE Computer Society Technical Committee on Data Engineering*, 41, 39–45.
- Zhang, M., Li, H., & Su, S. W. (2019). High Dimensional Bayesian Optimization via Supervised Dimension Reduction. *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence*, 4292–4298. Macao, China: IJCAI.
- Zhao, P., Liu, S., Chen, P.-Y., Hoang, N., Xu, K., Kailkhura, B., & Lin, X. (2019). On the Design of Black-Box Adversarial Examples by Lever-

aging Gradient-Free Optimization and Operator Splitting Method. *2019 IEEE/CVF International Conference on Computer Vision*, 121–130. Seoul, Korea (South): ICCV.

Ziomek, J., & Bou-Ammar, H. (2023). Are Random Decompositions all we need in High Dimensional Bayesian Optimisation? *CoRR*, *abs/2301.12844*. <https://doi.org/10.48550/arXiv.2301.12844>

Appendix A

Appendix on Additive GP-UCB using Tree Structures

A.1 Implementation Details

We implemented all algorithms in Python 3.8.3. The Python environments are managed using Conda, and experiments are managed using MLflow (Zaharia et al., 2018). The specific hardware used is not crucial, as we ensure consistent runs across different hardware via the use of Conda and MLflow.

Both Graph Overlap and Graph No-Overlap are re-implemented using the original implementations and the details from their publications (Kandasamy et al., 2015; Rolland et al., 2018).

For LineBO algorithms, we use the original author’s implementation¹. The REMBO and InterleavedREMBO implementations are also based on the implementation found in LineBO’s repository. We additionally consider the heuristic NelderMead algorithm (Nelder & Mead, 1965), which is provided by SciPy, as in LineBO’s repository. Implementation of the algorithms used in the BO-Based Adversarial Attack experiments are from the original author’s repository².

¹Implementation at <https://github.com/jkirschner42/LineBO>.

²Implementation at https://github.com/rubinxin/BayesOpt_Attack.

Method	Unique parameter values
DescentLineBO	As per hartmann6.yaml ³
RandomLineBO	
CoordinateLineBO	
REMBO	Embedded Dimension = $\sqrt{D}$
InterleavedREMBO	Interleaved Runs = 4
NelderMead	Contraction Factor = 0.3
	Initial Step-size = 0.1
	Restart Threshold = 0.001
Graph No-Overlap	Max Clique Size = D
Graph Overlap	N/A
ADDGP-BO	As per defaults in repository
GP-BO	

Table A.1. State-of-the-art methods’ parameters values.

A.2 Experimental Details

The existing algorithms that we use in our experiments, along with their parameters, are summarized in Table A.1. We note that for parameters unique to LineBO, we have used the recommended parameters as stated in (Kirschner et al., 2019), specifically the parameters found in their Hartmann6 experiments³.

A summary of the functions that were used in our experiments is given in Table A.2. We ran the functions in the Section A.3 over both continuous and discrete domains, investigating Tree’s performance. The rest of the functions were run over continuous domains.

A.2.1 Effect of Hyperparameters

In this section, we explore how different hyperparameters impact the learning process of its dependency structure thereby affecting model performance. Our discussion will not only demonstrate the effects of various hyperparameters, but also offer practical guidance for selecting appropriate hyperparameters to optimize model efficiency and effectiveness.

³From repository of (Kirschner et al., 2019) – <https://github.com/jkirschner42/LineBO/blob/master/config/hartmann6.yaml>.

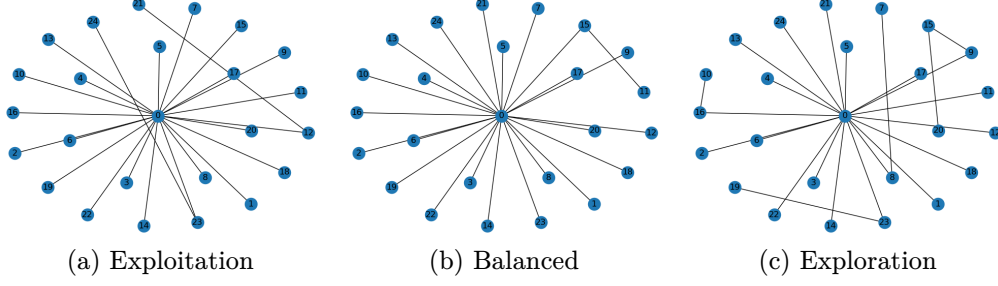


Figure A.1. Tree’s learned dependency structure at Iteration 33 of a run.

Acquisition Function

This section delves into how the exploration-exploitation trade-off in the UCB acquisition function affects the graph learning performance of Tree. We employ the Star-25 (Continuous) function, illustrated in Figure 3.4a, as the test scenario to conduct our analysis. Adhering to our experimental framework described in Section 3.4.1, we concentrate exclusively on adjusting the balance between $\mu(x)$ and $\sigma(x)$ as specified in Equation 1.7. This modification aims to evaluate the influence of this trade-off on the performance of Tree in learning graph structures.

- *Exploitation*: We set $\beta(t) = 0$, effectively disabling the uncertain reward term $\sigma(x)$, prioritizes exploiting points with high mean values.
- *Balanced*: Here, $\beta(t) = 0.5 \log(2t)$, is as per Section 3.4.1.
- *Exploration*: We use $a_{\text{UCB}} = \sigma(x)$, thereby disabling the expected reward term $\mu(x)$, focuses on exploring regions with high variance.

Despite both *Exploitation* and *Exploration* represent extreme cases, the results depicted in Figure A.3b demonstrate Tree’s effectiveness. In the *Exploration* scenario, Tree reconstructs the ground-truth dependency structure, as demonstrated in Figure A.2c. Conversely, under *Exploitation* conditions, Tree manages to learn a structure that closely approximates the actual one, which is detailed in Figure A.2a.

While Tree demonstrates its capability to accurately recover the ground-truth dependency structure, even in scenarios with extreme conditions, it is important to strike a balance between exploration and exploitation to achieve long-term performance. As shown in Figure A.3, the *Exploitation* approach tends to prioritize the best-known structure, which may result in sub-optimal learning

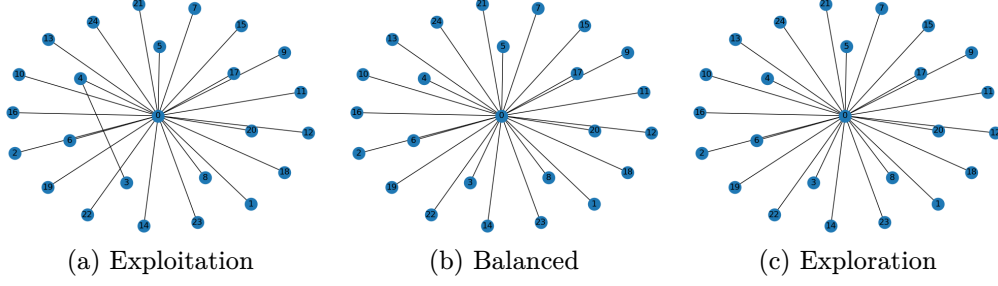


Figure A.2. Tree’s learned dependency structure at Iteration 50 of a run.

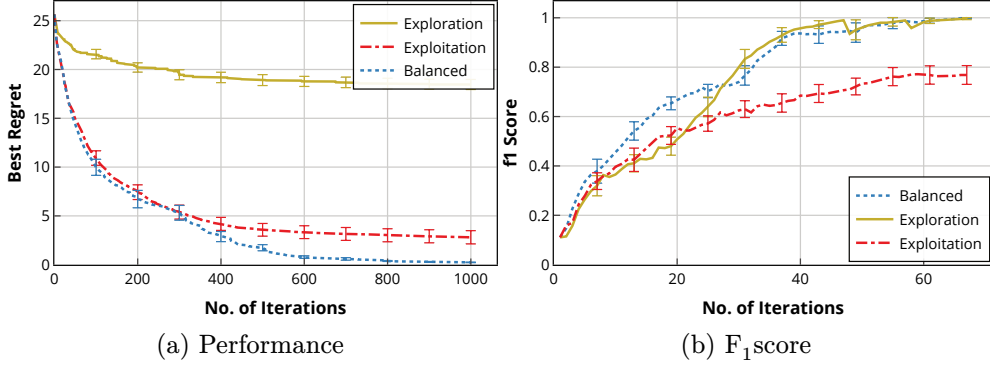


Figure A.3. Comparing Exploitation, Balanced and Exploration scenarios for Star-25 (Continuous).

outcomes in the long-term. Conversely, *Exploration* emphasizes investigating various structures, leading to slower convergence in performance, yet it often successfully identifies the ground-truth. This dichotomy between exploration and exploitation is a pivotal aspect of machine learning, influencing the selection of an apt acquisition function for *Tree*. The key lies in choosing an acquisition function that adeptly balances these two facets. In the context of UCB, we achieve this (*Balanced*) by modulating $\beta(t)$ to enable more exploration initially and gradually shift towards exploitation, ensuring a more effective learning trajectory.

Structure Sampling

In this section, we conduct an in-depth examination of how the hyperparameters N_{samples} and C jointly influence *Tree*’s performance. These parameters are crucial as they determine the total number of structure samples taken throughout the course of the run. To assess *Tree*’s sensitivity to their combined effects, We use the Star-25 (Continuous) function and execute a series of 9 experiments,

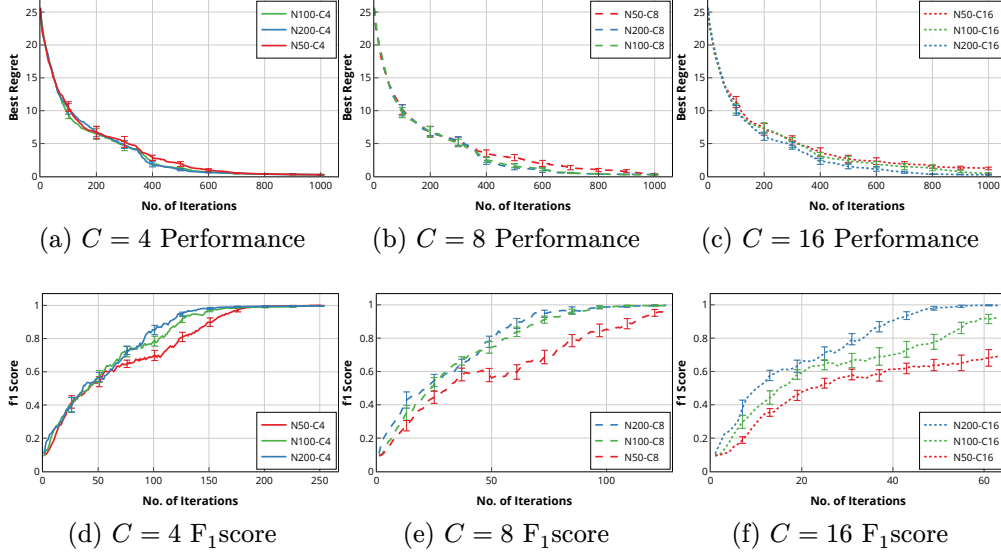


Figure A.4. Comparison of N_{samples} values [50, 100, 200] within C [4, 8, 16] for the Star-25 (Continuous) function. Each combination is denoted as ‘N’ and ‘C’, representing the specific pairing of N_{samples} and C parameters respectively.

$N_{\text{samples}} \in [50, 100, 200]$ and $C \in [4, 8, 16]$. A summary highlighting the total number of structure samples for each configuration, is provided in Table A.3. All other experimental settings remain consistent with those outlined in Section 3.4.1.

Observations from Figure A.4 reveal that, within each C value, the larger N_{samples} yields the best graph learning performance, although the benefits diminish incrementally. For instance, in the cases of $C = 4$ and $C = 8$, we observe similar levels of performance and graph learning effectiveness for $N_{\text{samples}} = 100$ and $N_{\text{samples}} = 200$. This suggests that $N_{\text{samples}} = 100$ is sufficient, while $N_{\text{samples}} = 50$ falls short for these specific C values. The selection of C is primarily driven by computational constraints, as sampling the graph every iteration may not be feasible. Once C is determined, it is crucial to provide Tree with an adequate number of N_{samples} to explore a diverse range of structures during the tree learning process.

Zoom Strategy

In this analysis, we focus on the hyperparameters N_{gridsize} and N_{lvl} , which affect the ZOOM-STRATEGY, as depicted in Figure 3.3. Employing the Star-25 (Continuous) function while maintaining settings in Section 3.4.1, we maintain

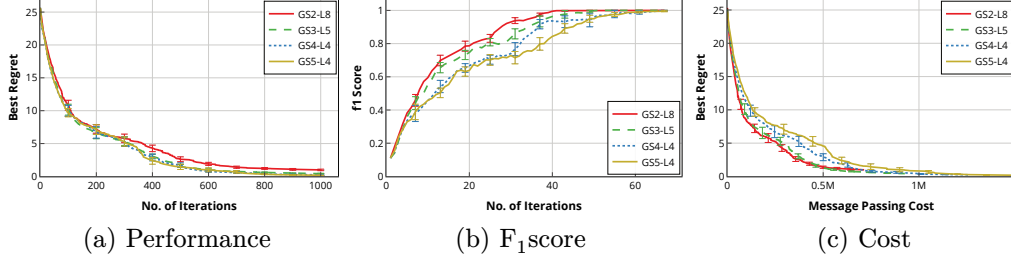


Figure A.5. Comparison of N_{gridsize} and N_{lv1} values for the Star-25 (Continuous) function. Each combination is denoted as ‘GS’ and ‘L’, representing the specific pairing of N_{gridsize} and N_{lv1} parameters respectively.

a constant resolution for the function domain while varying N_{gridsize} in the range of $[2, 5]$. N_{lv1} is not predetermined, we adjust it to the required number of levels to ensure the given domain is explored.

All four experiments successfully learned the ground-truth dependency structure, exhibiting comparable performance as shown in Figure A.5. It’s observed that increasing N_{gridsize} leads to marginally improved long-term performance, albeit at the cost of significantly more message passing. Notably, in tree structures, the message passing cost increases quadratically with N_{gridsize} . Therefore, N_{gridsize} serves as a crucial hyperparameter that balances the trade-off between the accuracy of optimization (performance) and computational time needed.

Type	Name	Dim.	No. of Exps.
Add. GP Fns.	Grid-3×3	9	250
	Star-10	10	250
	Partition-12	12	250
	Star-25	25	250
	Ancestry-132	132	125
Add. GP Fns. (Scalability)	Grid-2×2	4	125
	Grid-3×3	9	125
	Grid-4×4	16	125
	Grid-5×5	25	100
	Grid-6×6	36	100
	Grid-7×7	49	50
	Grid-8×8	64	50
	Grid-9×9	81	50
	Grid-10×10	100	25
	Grid-11×11	121	25
	Grid-12×12	144	25
	Grid-13×13	169	25
	Grid-14×14	196	25
	Grid-15×15	225	25
Non-GP Fns.	Camelback2	2	150
	Hartmann6	6	150
	Camelback2+10Aux	12	150
	Hartmann6+14Aux	20	150
	Rosenbrock20	20	150
	Stybtang250	250	150
NAS	naval	9	100
	parkinsons		100
	protein		100
	slice		100
Lpsolve	misc05inf	74	150
	mtest4ma		150
	qiu		150
BA	MNIST	196	1800

Table A.2. Summary of both Synthetic (top) and Real (bottom) functions used in our experiments; by type, dimensionality.

	$C = 4$	$C = 8$	$C = 16$
$N_{\text{samples}} = 50$	12500	6250	3125
$N_{\text{samples}} = 100$	25000	12500	6250
$N_{\text{samples}} = 200$	50000	25000	12500

Table A.3. Summary of the various structure sampling hyperparameters along with the corresponding total number of structure samples $N_{\text{iter}} \times N_{\text{samples}}/C$.

A.3 Additive GP Functions Experiments

In addition to the observations made in the main document, **Tree**’s efficiency is also demonstrated for both discrete and continuous functions – see Figures A.7, A.8. We see that **Tree** is able to learn a tree structure that performs well while being cost-efficient, even when the hypothesis that **Tree** considers is not realizable in both Grid- 3×3 and Partition-12. In the case of Partition-12, **Tree**’s graph learning is not realizable, whereas both **Graph No-Overlap** and **Graph Overlap** are realizable. The results for Partition-12 lends further support to the discussion in Section 3.4.3 on Grid- 3×3 , **Tree** remains competitive in terms of the optimization performance despite poorer graph learning. Most surprisingly, **Tree** is sometimes able to outperform **Oracle**; see Figure A.7g and Figure A.7j. To understand this, recall from Section 3.4.1 that we limited the number of individual acquisition function evaluations to 1000. For tree structures, we are dealing with functions of at most two variables, whereas this limitation in evaluations has the greatest impact on graphs whose triangulated versions have larger cliques. In the present example, the true graph provided for **Oracle** contains cliques of size at least 3 after triangulation. These results suggest that even when the hypothesis is not realizable, the performance penalty for **Tree** is minor. That is, the lack of model richness is compensated for by faster model selection and the ability to optimize the acquisition function accurately.

In addition, we ran experiments on Ancestry-132, a 132-dimensional family tree of (Nicolas Kruchten, 2015), shown in Figure A.6. The graph was obtained from the original author’s repository⁴ and converted to an equivalent representation for use in our codebase. We used the graph structure in the same way as any of the synthetic dependency graphs structures, which we specified in Section 3.4.3. We note that for higher dimensions, F_1 score in Figure A.9b may potentially improve by increasing the number of graphs N_{samples} sampled in Algorithm 3.4; we have adopted the same choice across all experiments for consistency.

⁴From repository of (Nicolas Kruchten, 2015) – <https://raw.githubusercontent.com/nicolaskruchten/genealogy/master/family.json>

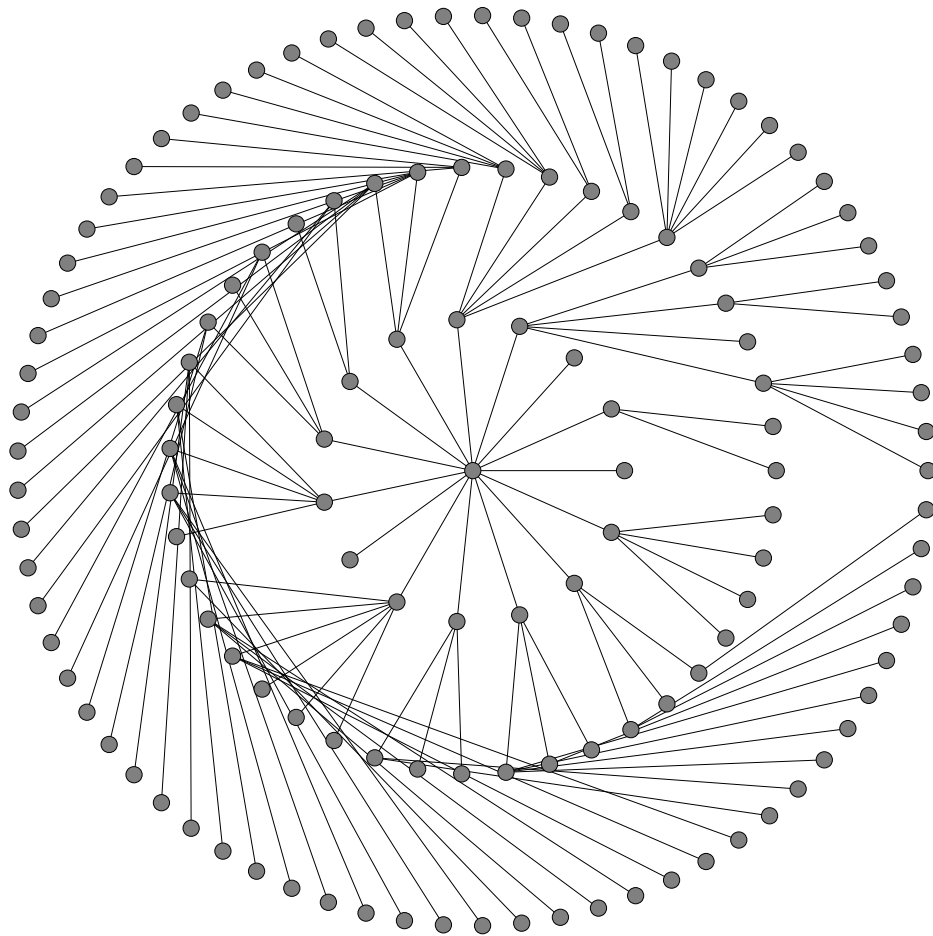
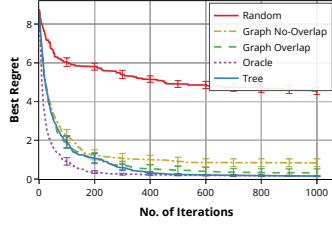
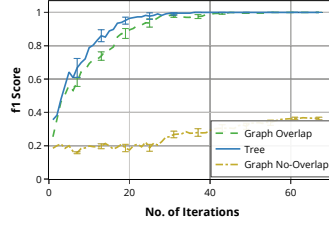


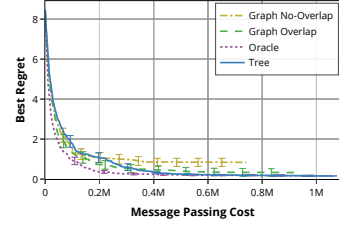
Figure A.6. Ancestry-132 Dependency Graph Structure.



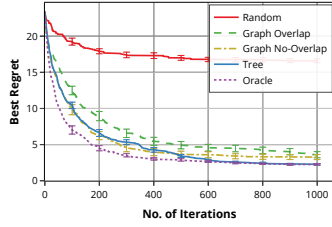
(a) Star-10 (Discrete) Performance



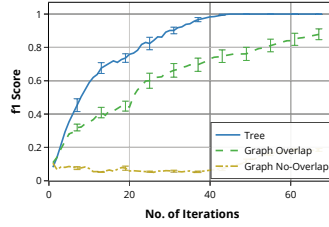
(b) Star-10 (Discrete) F_1 score



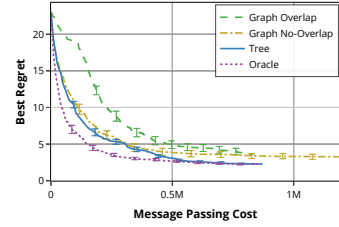
(c) Star-10 (Discrete) Cost



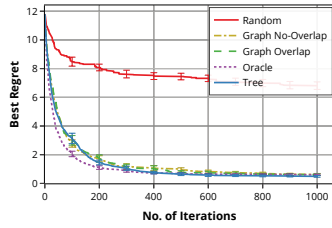
(d) Star-25 (Discrete) Performance



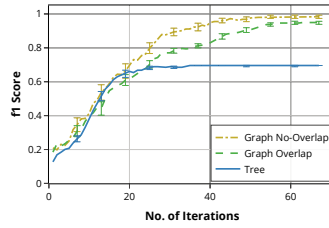
(e) Star-25 (Discrete) F_1 score



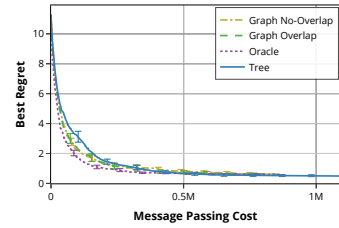
(f) Star-25 (Discrete) Cost



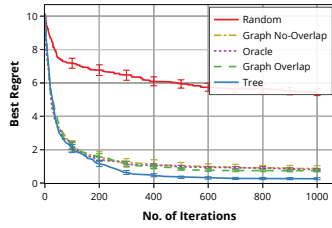
(g) Partition-12 (Discrete) Performance



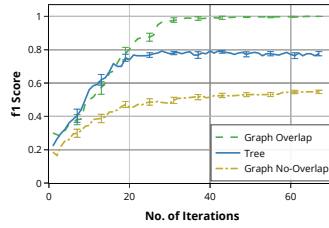
(h) Partition-12 (Discrete) F_1 score



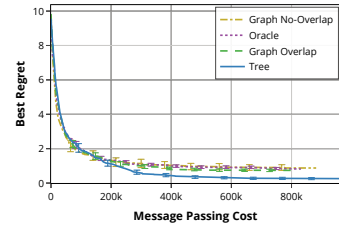
(i) Partition-12 (Discrete) Cost



(j) Grid-3×3 (Discrete) Performance



(k) Grid-3×3 (Discrete) F_1 score



(l) Grid-3×3 (Discrete) Cost

Figure A.7. Results for synthetic additive functions on discrete domains.

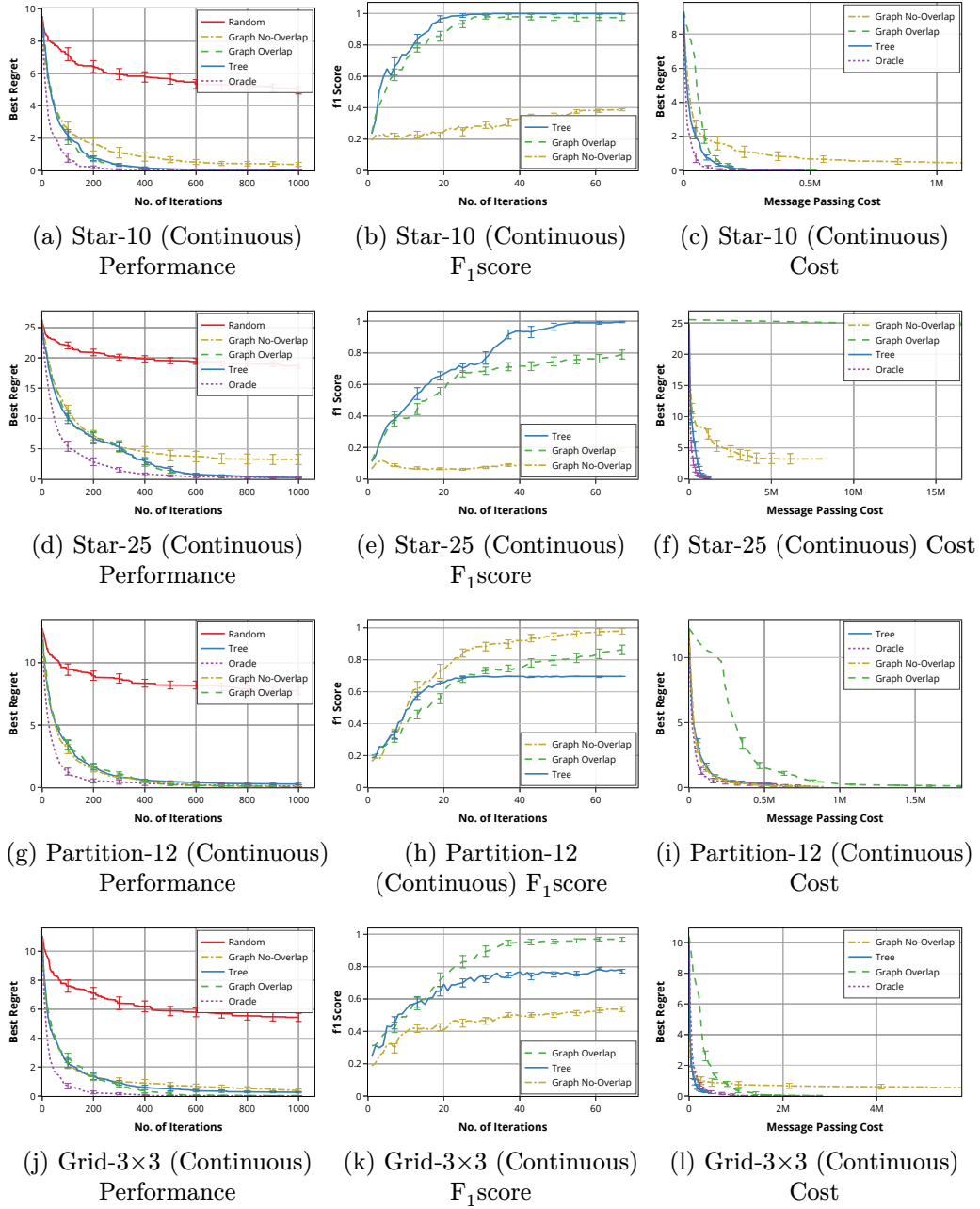


Figure A.8. Results of synthetic additive functions on continuous domains.

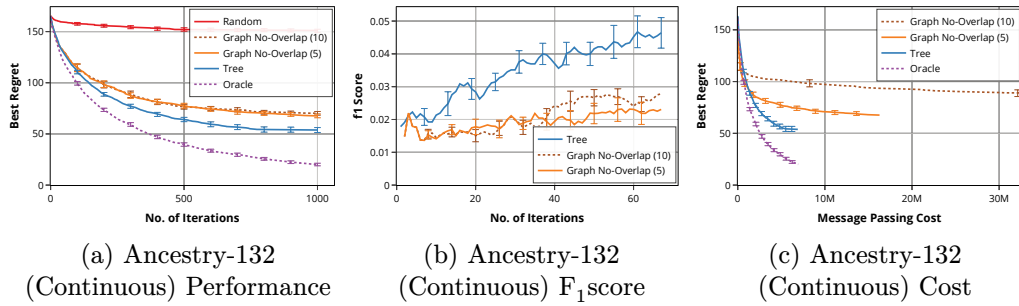


Figure A.9. Results for the Ancestry-132 graph.

A.4 Non-GP Functions Experiments

A.4.1 Synthetic Functions

Camelback2 is a commonly used BO benchmark function with two dimensions that has two global minima with the same value of -1.0316 (Eggenberger et al., 2013). Hartmann6 (6D) is defined as

$$f(x) = -\sum_{i=1}^4 \alpha_i \exp\left(-\sum_{j=1}^6 A_{ij} (x_j - P_{ij})^2\right), \quad (\text{A.1})$$

and has seven minima – six local and one global (Eggenberger et al., 2013). We use implementations from HPOlib2, letting α, A and P be as defined in (Eggenberger et al., 2013). Rosenbrock20 (20D) is a synthetic function that have a single global optima at $x = (1.0, \dots, 1.0)$ with value of 0. Rosenbrock20 is challenging to optimize, as the values returned by the function have a wide range even when using a small domain of $[0, 1]$.

The Styblinski-Tang function, Stybtang250 (250D):

$$f(x) = \frac{1}{2} \sum_{i=1}^{250} (x_i^4 - 16x_i^2 + 5x_i), \quad (\text{A.2})$$

has a global minimum of approximately -39.16599×250 at $x = (-2.9, \dots, -2.9)$. We restrict the domain of the function to $[-4, 4]^{250}$. Similar to Rosenbrock20, the values returned by Stybtang250 has a wide range.

Appendix B

Appendix on Adversarial Attacks on Bayesian Optimization

B.1 Implementation Details

Our implementation is based on Python 3.8, using Conda (Anaconda, 2016) and MLflow (Zaharia et al., 2018) to manage environments and experiments across multiple machines. Our implementation uses several standard machine learning and scientific packages, such as GPy (GPy, since 2012), NumPy (Harris et al., 2020) and others. The various libraries and the specific versions used can be found in our supplied code, in the Conda environment file `attack_bo.yml`.

Implementations of the synthetic functions are from HPOLib2 (Eggersperger et al., 2013). We use the original authors’ implementation (Zi Wang & Jegelka, 2017) for the robot pushing objective functions (Robot3D and Robot4D).

B.2 Experimental Details

For objective functions with 1 or 2 dimensions, we run the experiments with 10 initial points and 100 iterations. Due to the added difficulty in higher dimensions, we use 50 initial points and 250 iterations for the other experiments.

We adopt the Matérn- $5/2$ kernel, and initialize the kernel parameters with

values learned from data sampled from f prior to the optimization:

- For 1D and 2D functions, we randomly select 100 points from a multi-dimensional grid where each dimension is evenly spaced;
- For higher-dimensional functions, we use a total of 1000 points using a mix of strategies. We sample 500 points using the same grid strategy as for the 1D and 2D functions, as well as an additional 500 random points sampled from a uniform distribution and are not confined to a grid.

The kernel parameters are then fixed, and not updated throughout the attack. This amounts to an ‘idealized’ setting for the player in which the kernel is well-known even in advance. In Section B.6, we additionally present results for the case that the kernel parameters are learned online.

As mentioned in the main body, we focus primarily on attacking a standard form of GP-UCB with $\beta_t = 0.5 \log(2t)$. However, in Section 4.5.5, we additionally consider a variant of GP-UCB with enlarged confidence bounds designed for improved robustness (Bogunovic et al., 2020).

	Name	Dim.	#Exps.
GP-UCB Attack	Synthetic1D	1	1510
	Forrester1D	1	1510
	Levy1D	1	1510
	Levy-Hard1D	1	1510
	Bohachevsky2D	2	1510
	Bohachevsky-Hard2D	2	1510
	Branin2D	2	1510
	Camelback2D	2	1510
	Hartmann6D	6	910
	Robot3D	3	910
	Robot4D	4	910
Kernels	Branin2D-RBF	2	1510
	Branin2D-Matérn $3/2$	2	1510
	Camelback2D-RBF	2	1510
	Camelback2D-Matérn $3/2$	2	1510
MaxVar	MaxVar-Forrester1D	1	1510
	MaxVar-Camelback2D	2	1510
	MaxVar-Robot3D	3	910
Online	Synthetic1D-Online	1	1510
	Forrester1D-Online	1	1510
	Bohachevsky2D-Online	2	1510
Defense	Levy1D ($C = 0.5, \eta^2 = 0.01$)	1	1510
	Levy1D ($C = 0.5, \eta^2 = 0.1$)	1	1510
	Levy1D ($C = 0.5, \eta^2 = 1$)	1	1510
	Levy1D ($C = 2, \eta^2 = 0.01$)	1	1510
	Levy1D ($C = 2, \eta^2 = 0.1$)	1	1510
	Levy1D ($C = 2, \eta^2 = 1$)	1	1510
	Levy1D ($C = 8, \eta^2 = 0.01$)	1	1510
	Levy1D ($C = 8, \eta^2 = 0.1$)	1	1510
	Levy1D ($C = 8, \eta^2 = 1$)	1	1510
Dynamic	Synthetic1D-Dynamic	1	500
	Forrester1D-Dynamic	1	500
	Levy1D-Dynamic	1	500

Table B.1. Summary of various experiments and functions used, sorted by type and dimensionality.

B.3 GP-UCB Attack Experiments

B.3.1 Hyperparameters Discussion

We consider 30 hyperparameter configurations (e.g., choices of Δ or $h_{\max}$) for each attack, choosing the relevant minimum and maximum values so that we sufficiently cover the entire spectrum of the behavior, e.g., configurations with low to high success rates. From Figures B.2, B.5, we can observe that higher costs may not associate with a higher success rate; this is also discussed in the main body.

In general, increasing a hyperparameter value is representative of increasing the aggressiveness of the attack. Here, we see examples of poorly chosen parameters (e.g., smallest hyperparameter value) having high cost yet attacking unsuccessfully. For example, from Figure B.3a and illustrated in Figure B.1 (top), we see that the poorly chosen hyperparameter $\Delta = 0$ for Clipping incurs a high cost with a poor success rate. In this case, the attack is *not aggressive enough* (i.e., ‘under-aggressive’), and the GP-UCB algorithm is not incentivized to explore $\mathcal{R}_{\text{target}}$. Over many iterations, GP-UCB repeatedly explores $\mathcal{R}_{\text{target}}$, and continues to determine that $\mathcal{R}_{\text{target}}$ is no better than $(\tilde{\mathbf{x}}^*, f(\tilde{\mathbf{x}}^*))$. Hence, the attack can incur a high cost due to GP-UCB continuing to choose points outside $\mathcal{R}_{\text{target}}$ across the entire time horizon.

At the other end of the spectrum, increasing the aggressiveness of the attack can sometimes incur significantly more cost without any insignificant increase in success rate. For instance, we observe in Figure B.3a (see also Figure B.1) that the hyperparameter $\Delta = 42.5$ (bottom) incurs more cost than the most efficient $\Delta_{\text{efficient}} = 17.8$ (middle) but without a significant increase in success rate. Thus, the attack is overly aggressive; more cost than necessary is being used to incentivize the algorithm to choose actions in $\mathcal{R}_{\text{target}}$. In this particular case (Figure B.1), increasing Δ more than $\Delta_{\text{efficient}}$ increases the cost, as it unnecessarily, “swallow” more of the local maxima on the right of $\mathcal{R}_{\text{target}}$. On the other hand, sometimes increasing the aggressiveness of the attack can have a minimal impact on the cost-success trade-off. For instance, we see this behavior in Figure B.2j, and Figure B.3h, where increasing aggression improves the success

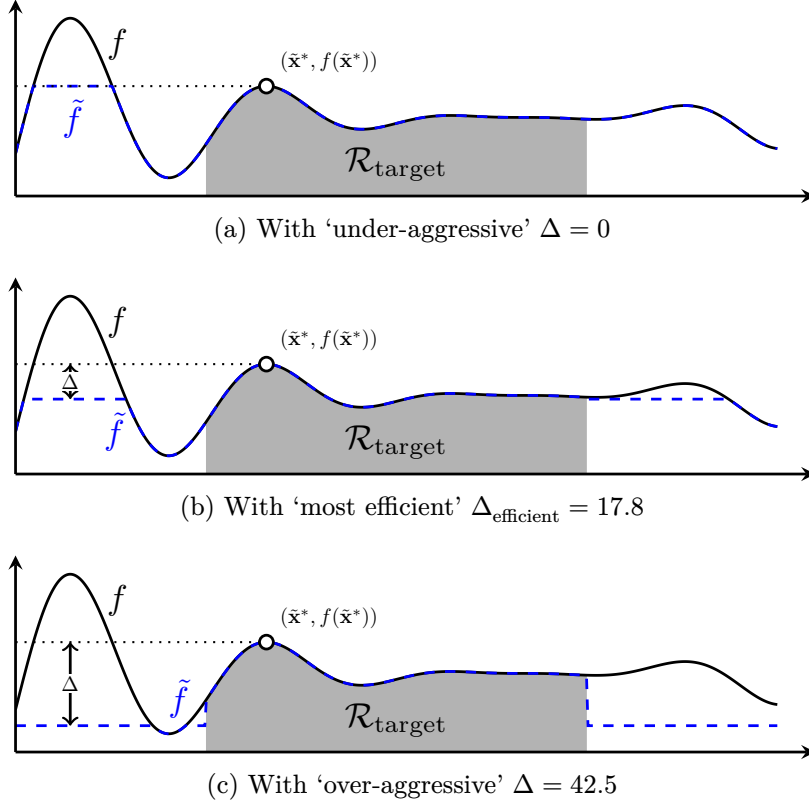


Figure B.1. Clipping attack on Synthetic1D; see Equation 4.18 for the definition of ‘most efficient’.

rate.

Considering that successfully forcing actions in $\mathcal{R}_{\text{target}}$ is the attacker’s primary goal and that $\theta_{\text{efficient}}$ is difficult to find, in general, it may be preferable to be ‘over-aggressive’ than ‘under-aggressive’ when considering hyperparameters.

B.3.2 Comparison of attacks

In Figure B.6, we again see Clipping attacking efficiently and successfully, as discussed in Section 4.5. Similarly, we continue to observe that Subtraction Rnd and Subtraction Sq are often the next best attacks following Clipping. In Figures B.4, B.5, Subtraction Rnd has a slight advantage over Subtraction Sq, though the two are similar. Due to Subtraction Rnd’s smooth $h(x)$, the attack behavior varies less with respect to its hyperparameter h_{max} , and the attacks generally succeed with slightly less cost compared with Subtraction Sq.

The unique behavior of the various attacks in Bohachevsky2D is similar to the behavior observed for Bohachevsky-Hard2D. The efficiency of the Aggres-

sive Subtraction attack in Bohachevsky-Hard2D continues to be similar to Bohachevsky2D, despite having a smaller $\mathcal{R}_{\text{target}}$ in Bohachevsky-Hard2D. Both Bohachevsky2D and Bohachevsky-Hard2D serve as illustrations of the difficult case discussed in Section 4.3.3.

B.4 Kernel Experiments

We have focused on the Matérn $5/2$ kernel, as it is widely-adopted and provides useful properties that we can exploit in our theoretical analysis. Here we additionally demonstrate our attacks on two other popular kernel functions: Radial Basis Function (RBF) and Matérn $3/2$. Apart from the kernel choice, the setup is the same as that of Section B.3.

From Figure B.8, we observe that the RBF kernel is easier to attack as it is smoother than Matérn $5/2$; the RBF kernel corresponds to a Matérn kernel where $\nu \rightarrow \infty$, leading to infinitely many derivatives. Theorem 4.1 indicates that smoother functions (smaller γ_t) are easier to attack. Similarly, Matérn $3/2$ is harder to attack, which is consistent with it having a larger γ_t . Intuitively, using a less smooth kernel encourages more exploration, which makes it more difficult to force the algorithm into the target region.

B.5 MaxVar+Elimination Attack Experiments

Here we move from attacking GP-UCB to attacking the MaxVar+Elimination algorithm, described in Section 4.3.1. With the exception of the selection rule itself, other settings in each of the MaxVar attack experiments are identical to the corresponding GP-UCB attack experiments.

We observe from Figure B.9 that the behavior of both MaxVar-Camelback2D and MaxVar-Robot3D are similar to the GP-UCB counterparts. Notwithstanding the non-standard ‘gap’ as observed in the figure, MaxVar-Forrester1D behaved similarly when compared to Forrester1D. One exception is the ‘gap’ in Figure B.9a, which may be due to the ‘non-smooth’ selection criteria of the t -th point, explicitly filtering candidate points that are not as high as the highest LCB.

B.6 Online-Learned Kernel Experiments

In GP-UCB Attack, we initialized the kernel parameters with values learned from data sampled from f prior to the optimization. Here, we consider an alternative setting in which the kernel parameters are learned online. The kernel parameters here are updated using the widely-used technique – maximum likelihood optimized with the Limited-memory Broyden-Fletcher-Goldfarb-Shanno (L-BFGS) algorithm. Our experiments use the same settings as the corresponding GP-UCB attack experiments, with the exception of the kernel parameters.

Comparing the online vs. offline approaches in Figure B.10, we see that there is generally no major difference in the behavior of these two approaches; being online vs. offline does not appear to significantly impact robustness. A possible exception is that some *specific runs* (i.e., random seeds) appear to have an unusually low success rate in Synthetic1D-Online when online kernel learning is used, particularly with Subtraction Rnd. A possible explanation for this is that the attack can sometimes perturb the function to the extent that the maximum-likelihood kernel parameters are highly misleading, causing the algorithm to behave in an unpredictable manner, and choose fewer points from $\mathcal{R}_{\text{target}}$.

B.7 Dynamic Experiments

We have focused on the fixed-hyperparameter setting, where the hyperparameters are fixed prior to the attack (e.g., Δ is pre-determined and fixed in the Aggressive Subtraction Attack). This setting serves as an important stepping stone to more general settings, and also aligns with our theory.

On the other hand, from our experiments and discussion, it is evident that the choice of the attack’s hyperparameters can be a very important factor in the success of the attack. If f is known to the attacker, then in principle various configurations could be simulated to find the best one. However, pre-specifying the hyperparameters may be much more difficult when limited prior knowledge is available.

Here, we provide an initial investigation into whether the hyperparameters can be chosen automatically, *i.e.*, dynamically adjusted online during the attack.

We consider a simple strategy to adjust the hyperparameter: The attack gets less aggressive whenever $\mathcal{R}_{\text{target}}$ is sampled consecutively K times, and more aggressive otherwise. Thus, the hyperparameter (e.g., representing Δ) is updated as follows:

$$\theta_t = \begin{cases} \theta_{t-1} - F \cdot \theta_{t-1} & \text{sampled consecutively,} \\ \theta_{t-1} + F \cdot \theta_{t-1} & \text{otherwise,} \end{cases} \quad (\text{B.1})$$

where F is the size of the hyperparameter update, as a fraction of the current value.

Apart from this change, we consider the same setup as that of Section B.3. We apply the strategy to **Aggressive Subtraction** on Synthetic1D, Forrester1D and Levy1D; based on minimal manual tuning, we set $F = 0.1$ and $K = 3$ across the three functions without further tweaking. We compare the fixed vs. dynamic attacks for various hyperparameters, where setting a hyperparameter in the dynamic setting means fixing its *initial* value only.

From Figure B.11, this strategy works well on these functions, significantly improving the performance (i.e., trade-off of success rate and cost) for the configurations that perform poorly in the case of being fixed. This experiment serves as a useful proof of concept, but further research is needed towards establishing fully automated attacks method for general scenarios. We believe that our work provides a good starting point towards this goal.

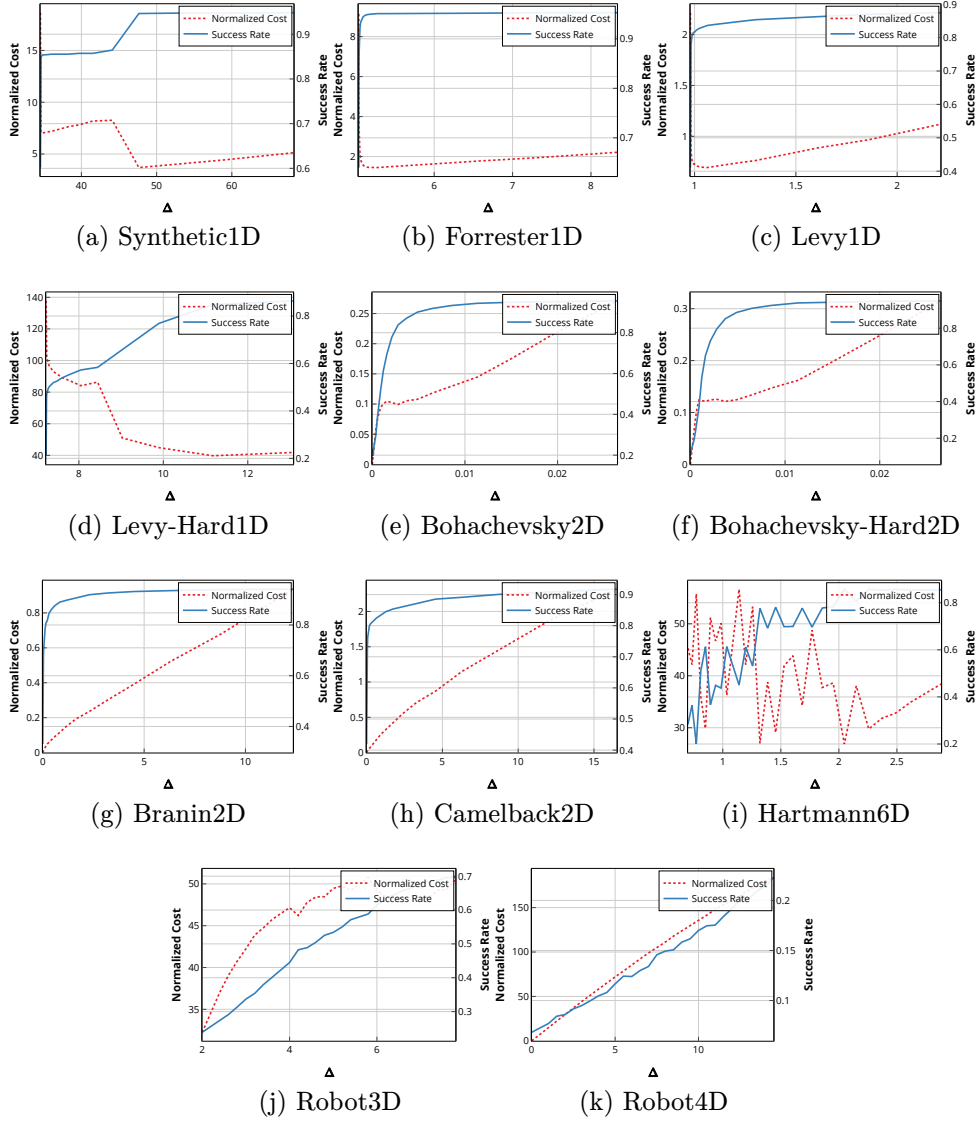


Figure B.2. Effect of Aggressive Subtraction's hyperparameter Δ on the success rate and cost incurred.

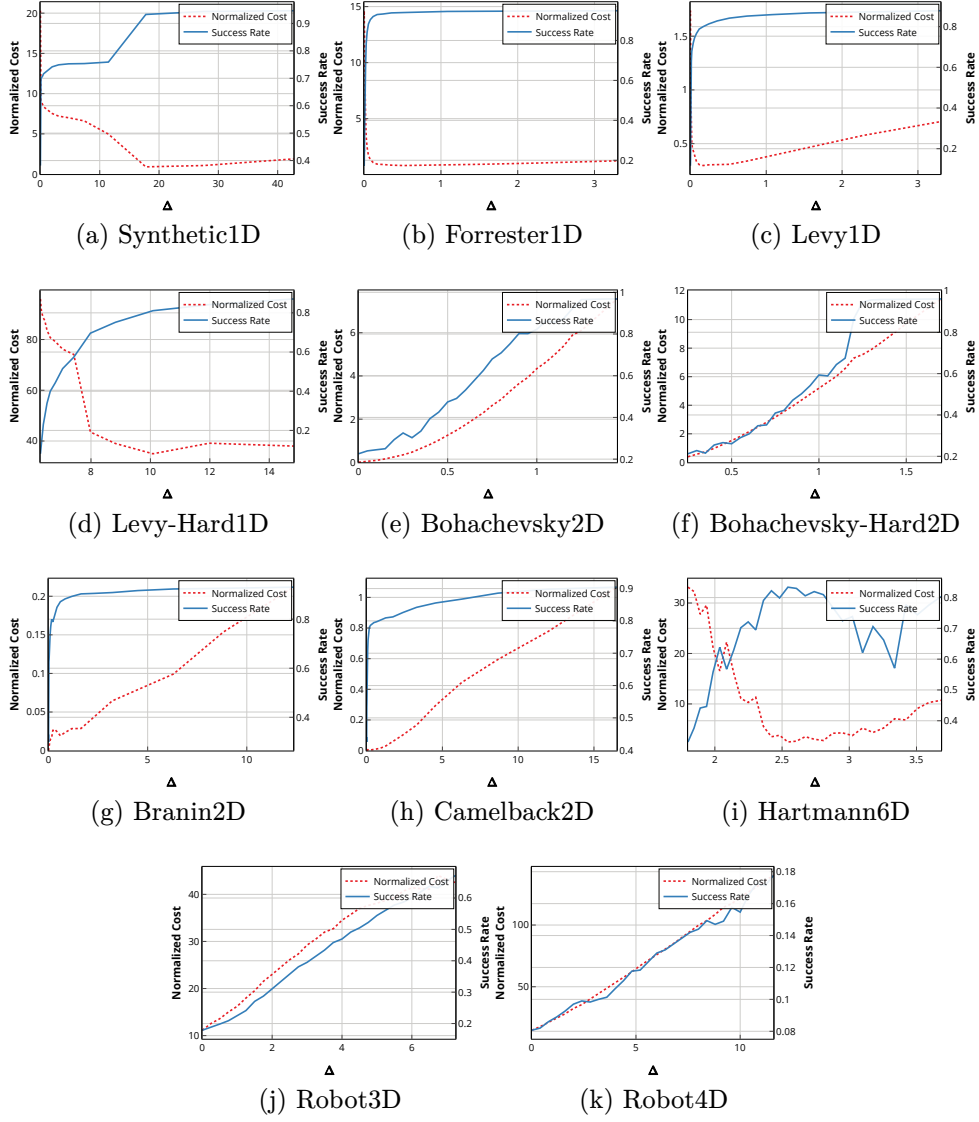


Figure B.3. Effect of Clipping's hyperparameter Δ on the success rate and cost incurred.

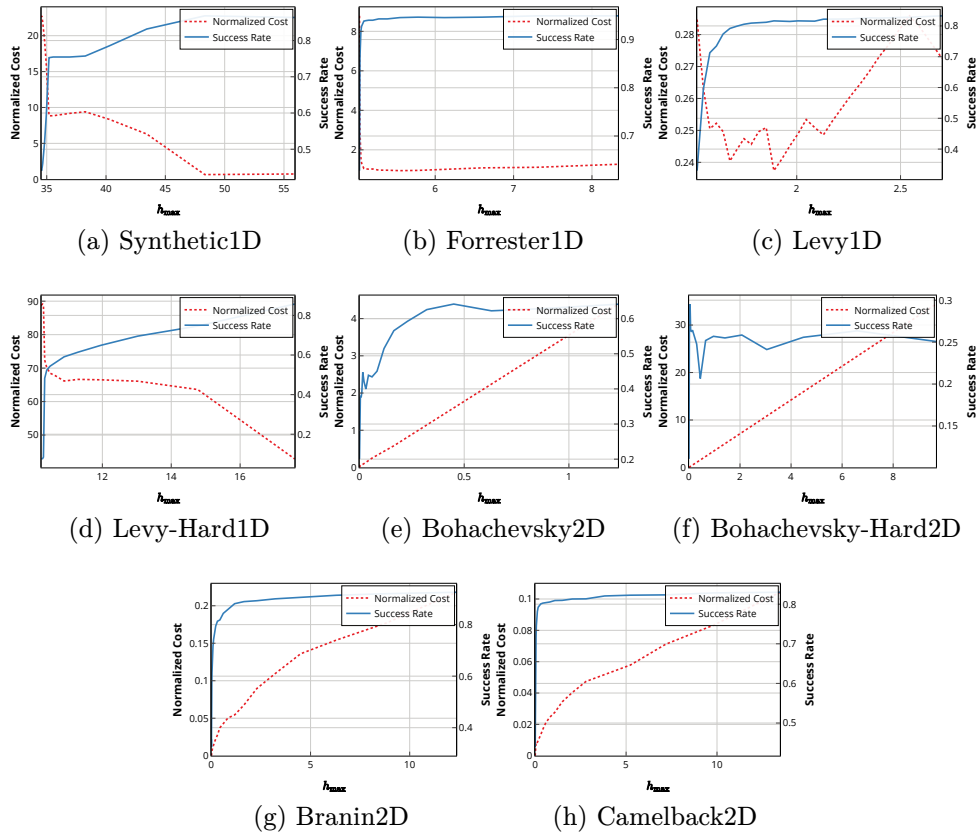


Figure B.4. Effect of Subtraction Rnd's hyperparameter $h_{\max}$ on the success rate and cost incurred.

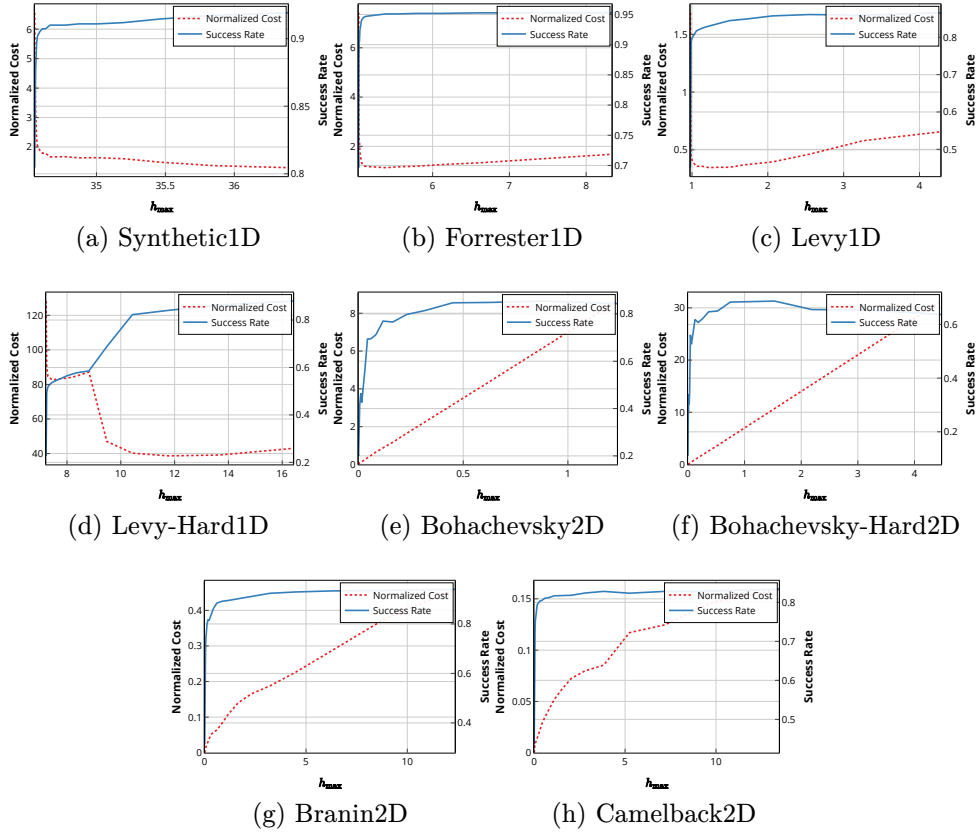


Figure B.5. Effect of Subtraction Sq's hyperparameter $h_{\max}$ on the success rate and cost incurred.

+ No Attack ♦ Random × Subtraction Rnd ▲ Subtraction Sq ■ Clipping ● Aggressive Subtraction

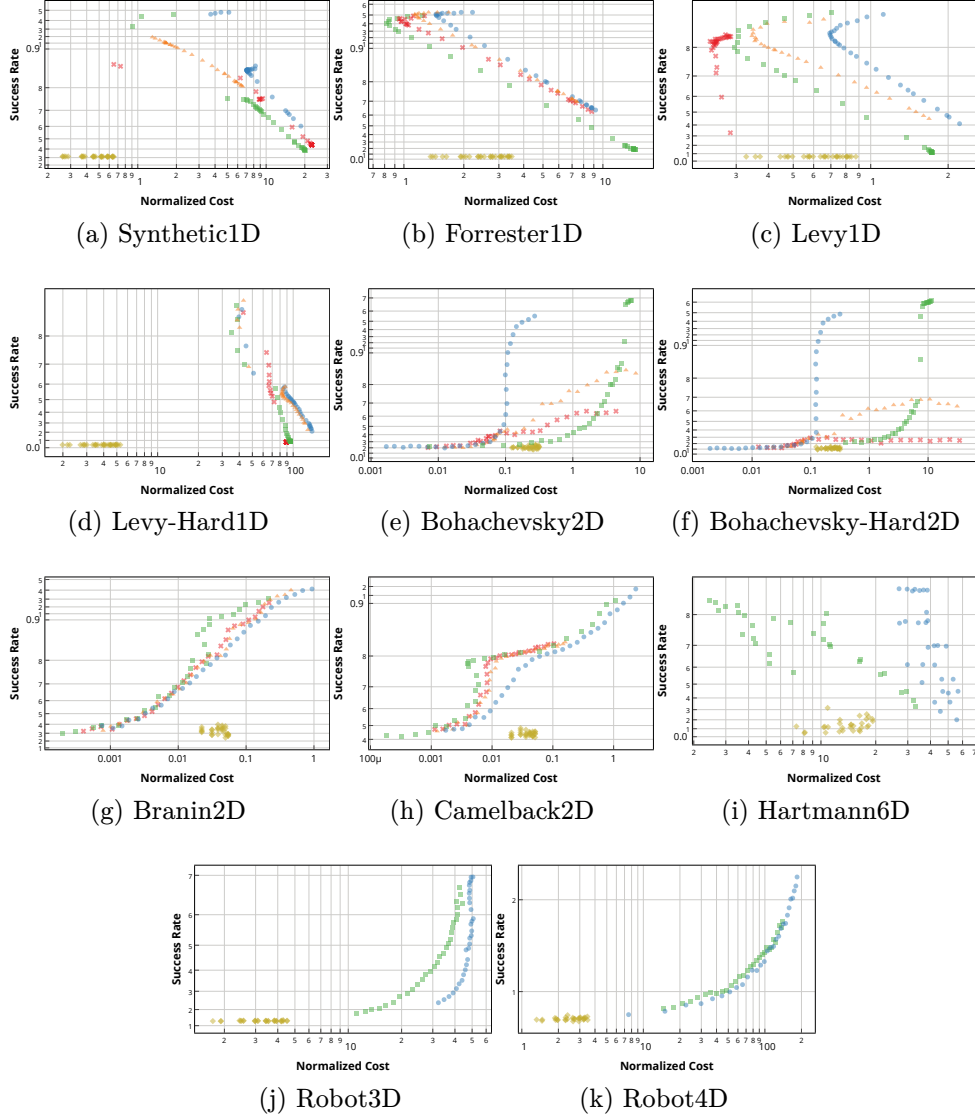


Figure B.6. Success rate vs. cost averaged over random seeds, each point is the performance of a particular hyperparameter.

+ No Attack ♦ Random × Subtraction Rnd ▲ Subtraction Sq ■ Clipping ● Aggressive Subtraction

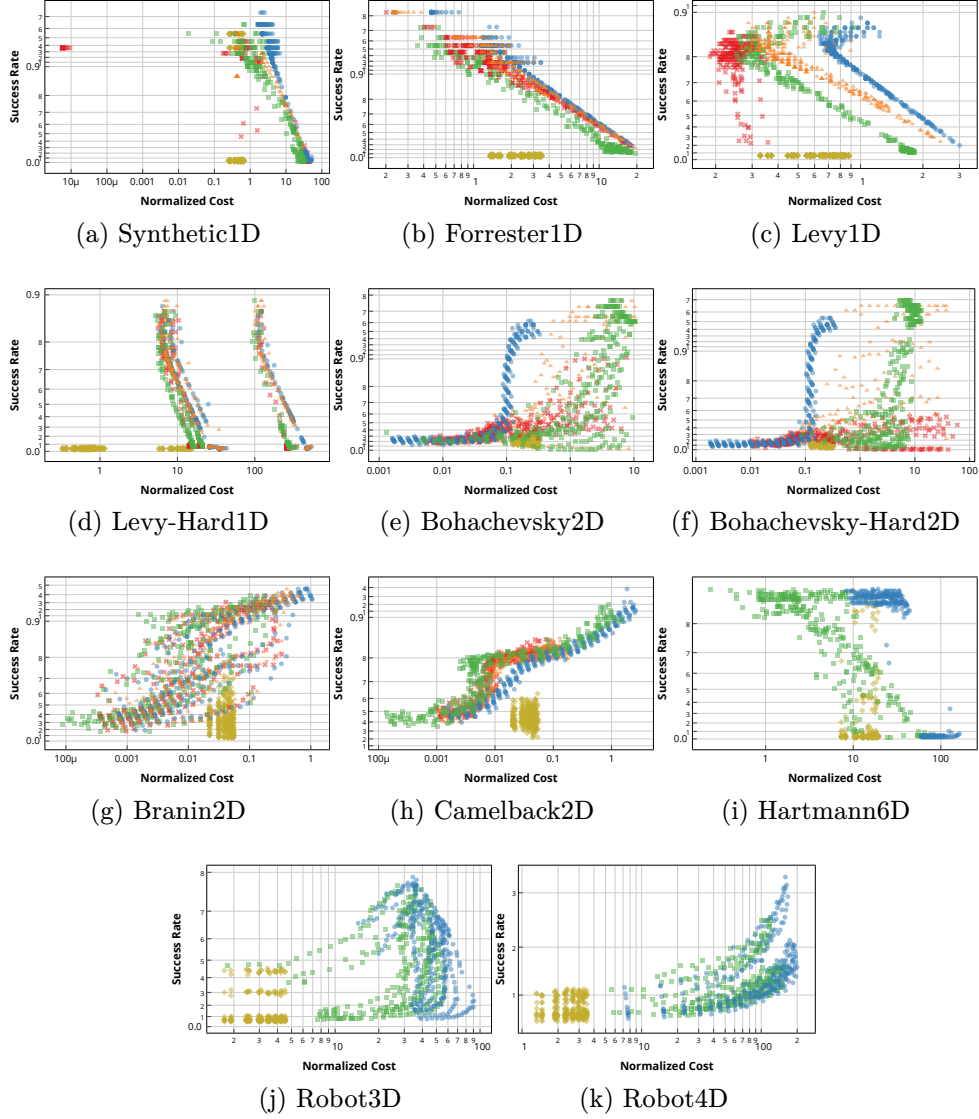


Figure B.7. Success rate vs. cost with every random seed shown individually, i.e., each point is a single run.

+ No Attack ♦ Random × Subtraction Rnd ▲ Subtraction Sq ■ Clipping ● Aggressive Subtraction

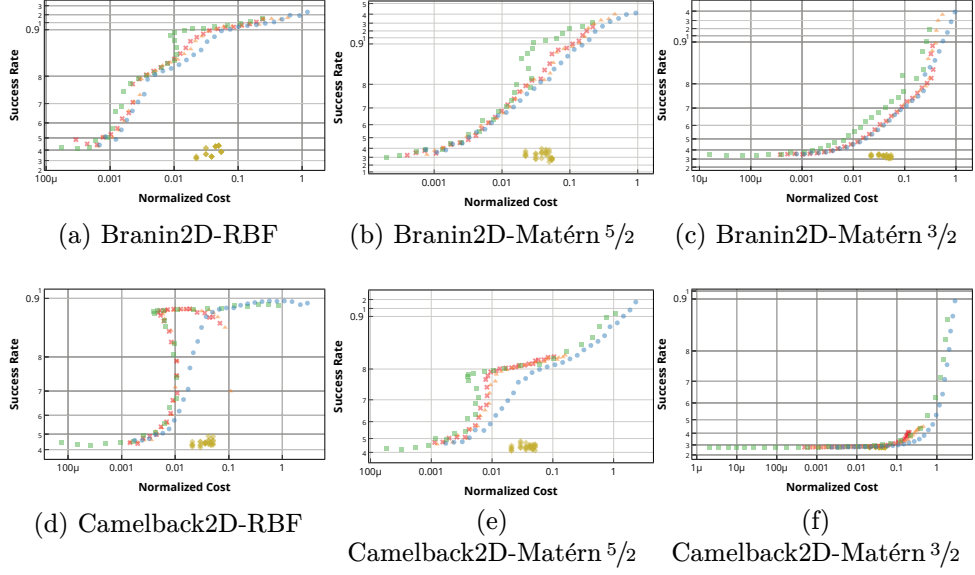


Figure B.8. Experiments comparing different kernels: Radial Basis Function (RBF), Matérn $5/2$, and Matérn $3/2$.

+ No Attack ♦ Random × Subtraction Rnd ▲ Subtraction Sq ■ Clipping ● Aggressive Subtraction

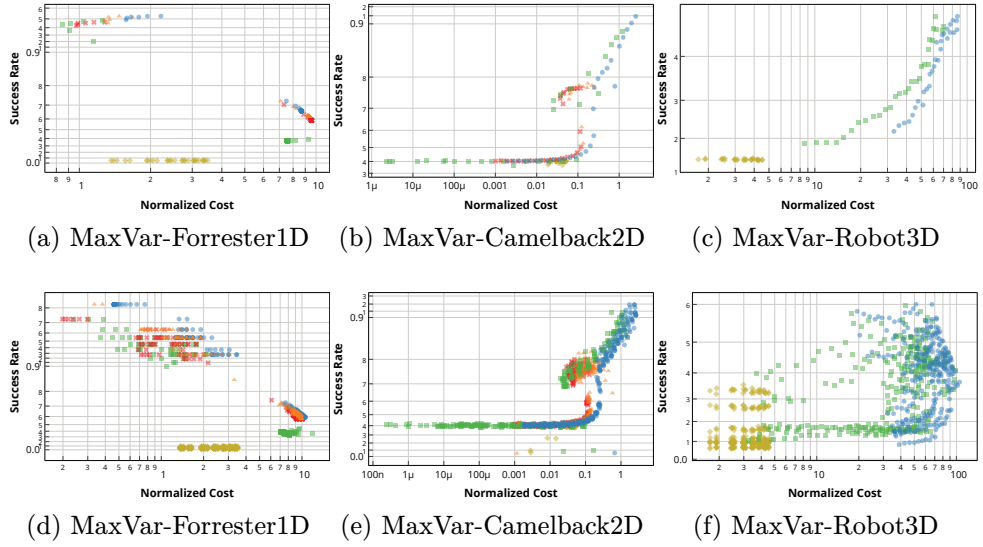


Figure B.9. We attack the MaxVar + Elimination algorithm in 3 experiments.

+ No Attack ♦ Random × Subtraction Rnd ▲ Subtraction Sq ■ Clipping ● Aggressive Subtraction

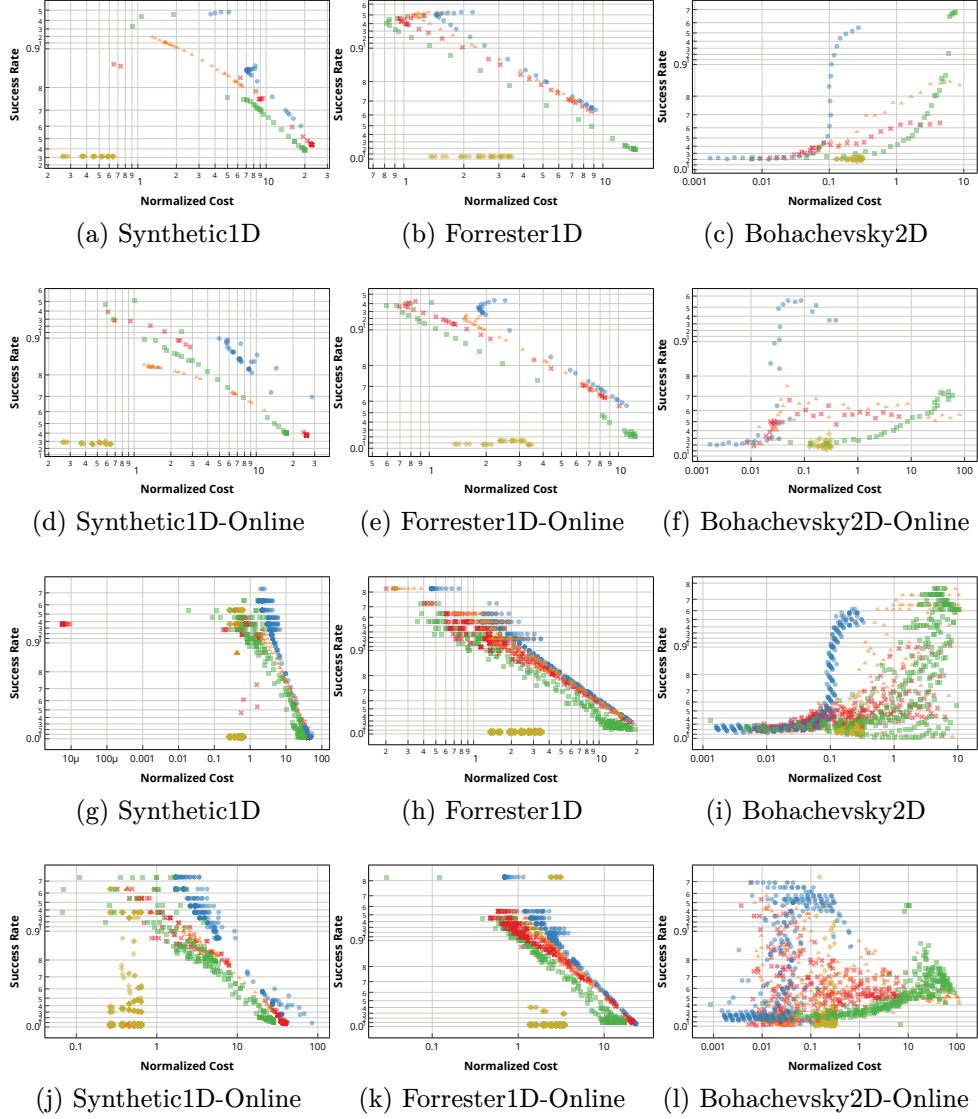


Figure B.10. Experiments comparing the kernel parameters being learned online (second and forth rows) vs. learned from data sampled from f prior to the optimization (first and third rows). The top two rows average over random seeds, and the bottom two rows show every individual run.

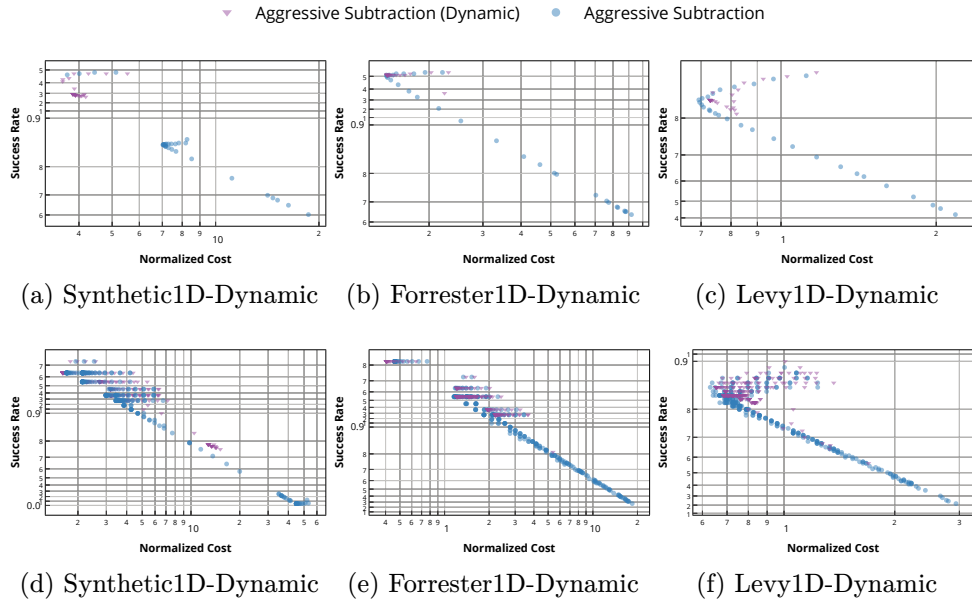


Figure B.11. Experiments applying a simple dynamic hyperparameter strategy for the Aggressive Subtraction attack. The top row averages over random seeds, and the bottom row shows every individual run.

Appendix C

Appendix on Black-box Adversarial Attacks on CNNs

C.1 Implementation Details

Our implementation is built upon Python 3.8.15 and utilizes Conda (Anaconda, 2016) along with MLflow (Zaharia et al., 2018) for environment management and experiment tracking across multiple machines. We rely on popular machine learning and scientific packages, including PyTorch (Paszke et al., 2019), BoTorch (Balandat et al., 2020), and NumPy (Harris et al., 2020). Detailed information about the libraries and versions used is provided in our supplied code through the Conda environment file ‘Adversarial.yml’.

The implementations of NNI and LF were from the original authors¹ (Shukla et al., 2021). For Sign-Opt, we used the implementation from (Nicolae et al., 2018), and for Sq-Attack, the one from (H. Kim, 2020). The MNIST (Lecun et al., 1998), CIFAR-10 (Krizhevsky, 2009), and ImageNet (Deng et al., 2009) datasets, along with their respective CNNs, were also obtained from the original authors (Cheng et al., 2019, 2020; K. He et al., 2016; Shukla et al., 2021; Simonyan & Zisserman, 2015; Szegedy et al., 2016).

¹Available at https://github.com/satyanshukla/bayes_attack.

C.2 Experimental Details

In our experimental setup, we evaluate various attacks to assess their performance and effectiveness. We consider two types of attacks: ℓ_∞ attacks and ℓ_2 attacks. For ℓ_∞ attacks, we begin with initial samples drawn from a standard normal distribution. We used a specific perturbation budget (following commonly used values in literature) for different datasets such as MNIST ($\epsilon = 0.3$), and CIFAR ($\epsilon = 0.05$) and ImageNet ($\epsilon = 0.05$). Similarly, for ℓ_2 attacks, initial samples are drawn from a uniform distribution in the range $[-1, 1]$, and a perturbation budget ($\epsilon = 20.0$) is applied for ImageNet. It’s important to mention that while various random initialization methods are viable for both ℓ_∞ and ℓ_2 settings, our choice aligns with the initialization approach used in (Shukla et al., 2021).

The images selected for attack consist of a randomly chosen subset of $M = 1000$ samples. These images are drawn from the test set of MNIST and CIFAR-10, while for Imagenet, the validation set is used. The total queries budget for BO is set to $N = 1000$, which includes an initial set of samples (with $N_{\text{init}} = 5$).

To explore the impact of different hyperparameters (latent space size $\lfloor rd \rfloor$), we run methods NNI and Tiling with $\lfloor rd \rfloor \in \{2, 3, 4, 5, 10, 15, 20\}$. For LF and FSFT we also include $\lfloor rd \rfloor = 1$, *i.e.*, $\lfloor rd \rfloor \in \{1, 2, 3, 4, 5, 10, 15, 20\}$, FSFT is the only method where $\lfloor rd \rfloor = 1$ is meaningful – attacking a single frequency. The different hyperparameters allow us to investigate the influence of $\lfloor rd \rfloor$ on the effectiveness of our attacks. For methods requiring additional hyperparameters, we primarily utilize Grid Search to determine optimal hyperparameters, starting from the recommendations provided in their respective studies.

C.3 Ablation Experiments for Tiling

For both Rng-Adv and BO-Adv agents, applying the ϵ -ball Projection enhances both NNI and Tiling by fully utilizing the perturbation budget, resulting in higher success rates and fewer queries. We observe in Figures C.1, C.2, C.3, and Figure C.2, that when compared with NNI, Tiling not only boosts the attack effectiveness while using less queries, but also requires smaller latent space $\lfloor rd \rfloor$.

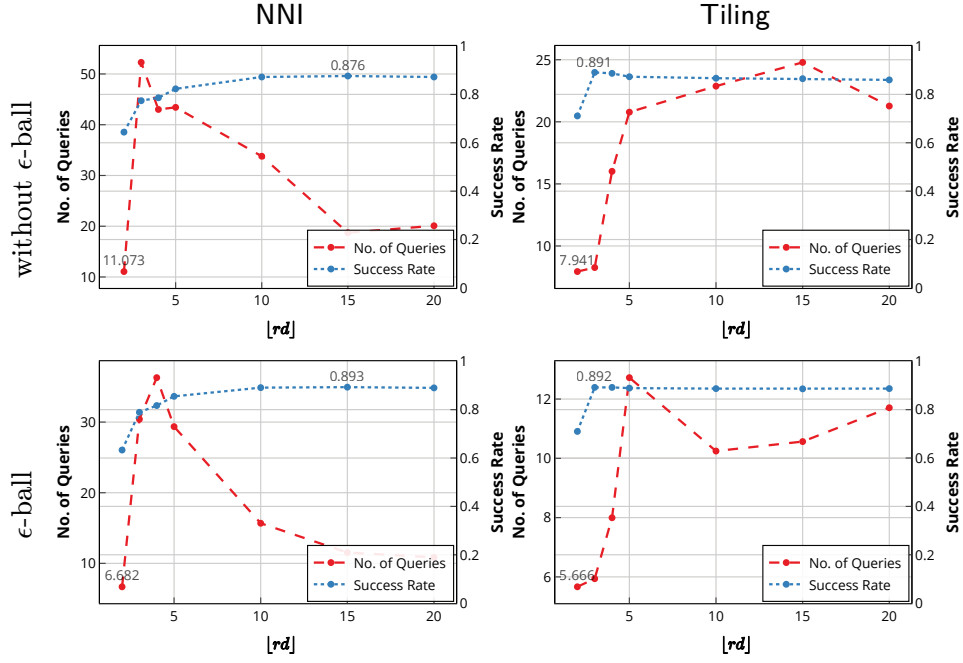


Figure C.1. Untargeted ℓ_∞ Rng-Adv Attack on MNIST, comparing NNI vs Tiling and with vs without ϵ -ball Projection.

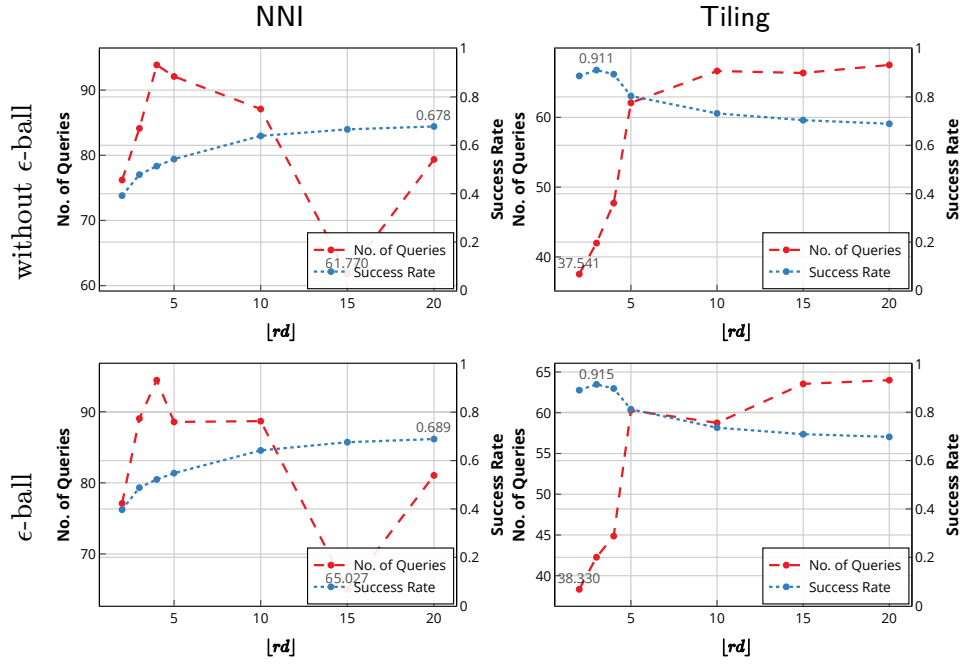


Figure C.2. Untargeted ℓ_∞ Rng-Adv Attack on CIFAR-10, comparing NNI vs Tiling and with vs without ϵ -ball Projection.

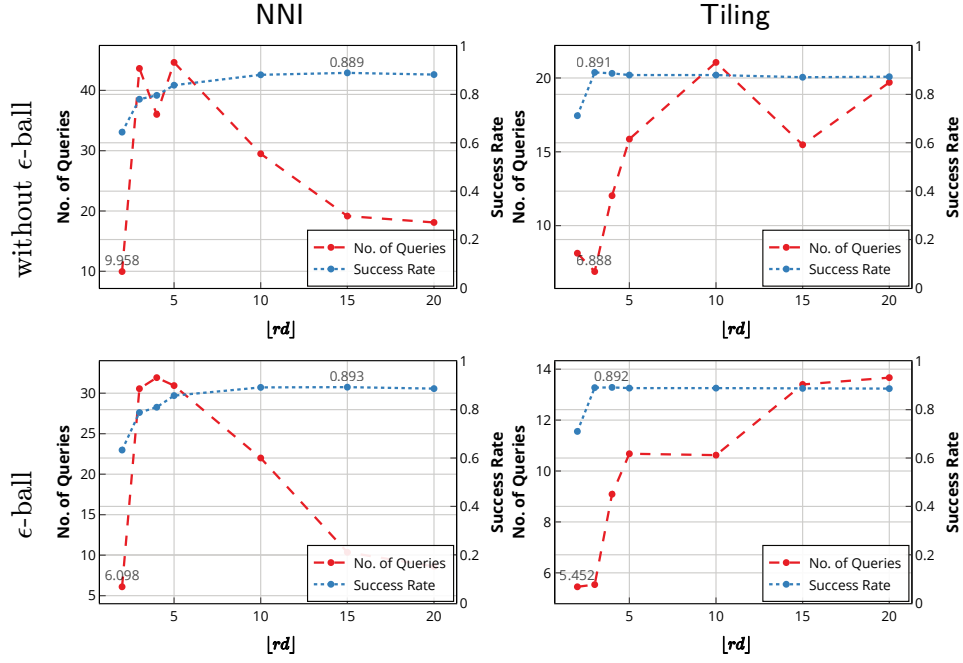


Figure C.3. Untargeted ℓ_∞ BO-Adv Attack on MNIST, comparing NNI vs Tiling and with vs without ϵ -ball Projection.

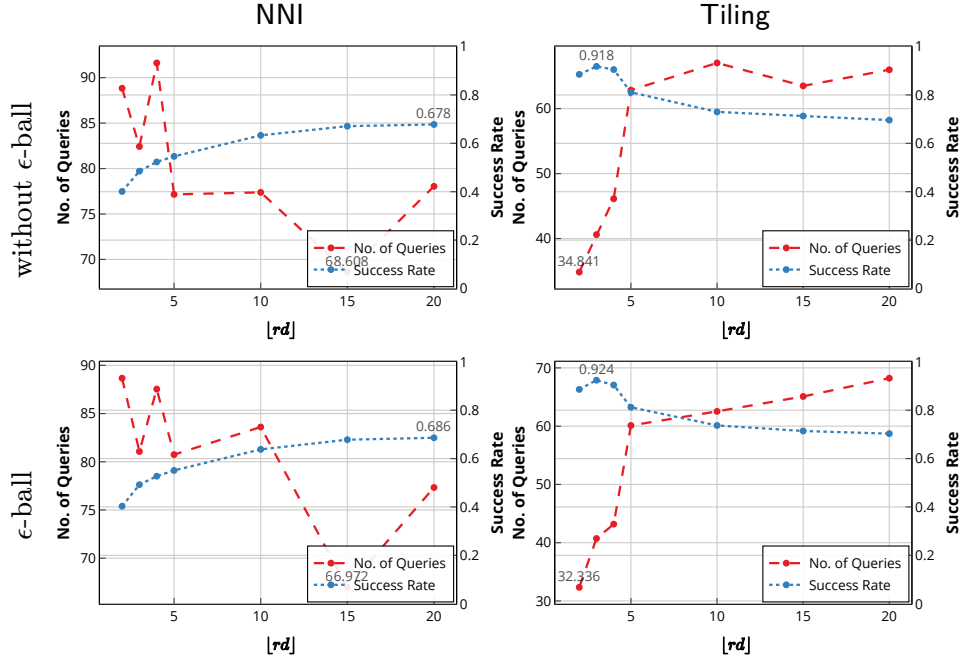


Figure C.4. Untargeted ℓ_∞ BO-Adv Attack on CIFAR-10, comparing NNI vs Tiling and with vs without ϵ -ball Projection.

C.4 Untargeted ℓ_∞ Attack Experiments

For MNIST, CIFAR-10, ResNet50, and VGG16-bn, our **Tiling** consistently outperforms NNI for both Rng-Adv and BO-Adv agents (See Figures C.5, C.6), achieving higher success rates with fewer queries and at lower latent dimensions $\lfloor rd \rfloor \sim 2$. Even considering the best results across different latent dimensions $\lfloor rd \rfloor$, **Tiling** is still superior; the highest success rate is often achieved at higher latent dimensions (*i.e.*, $\lfloor rd \rfloor = 20$) for NNI, whereas for **Tiling** the highest success rate is higher and is achieved at a lower $\lfloor rd \rfloor \leq 3$ while using drastically fewer queries. For Inception-v3, NNI is better but only at higher latent dimensions.

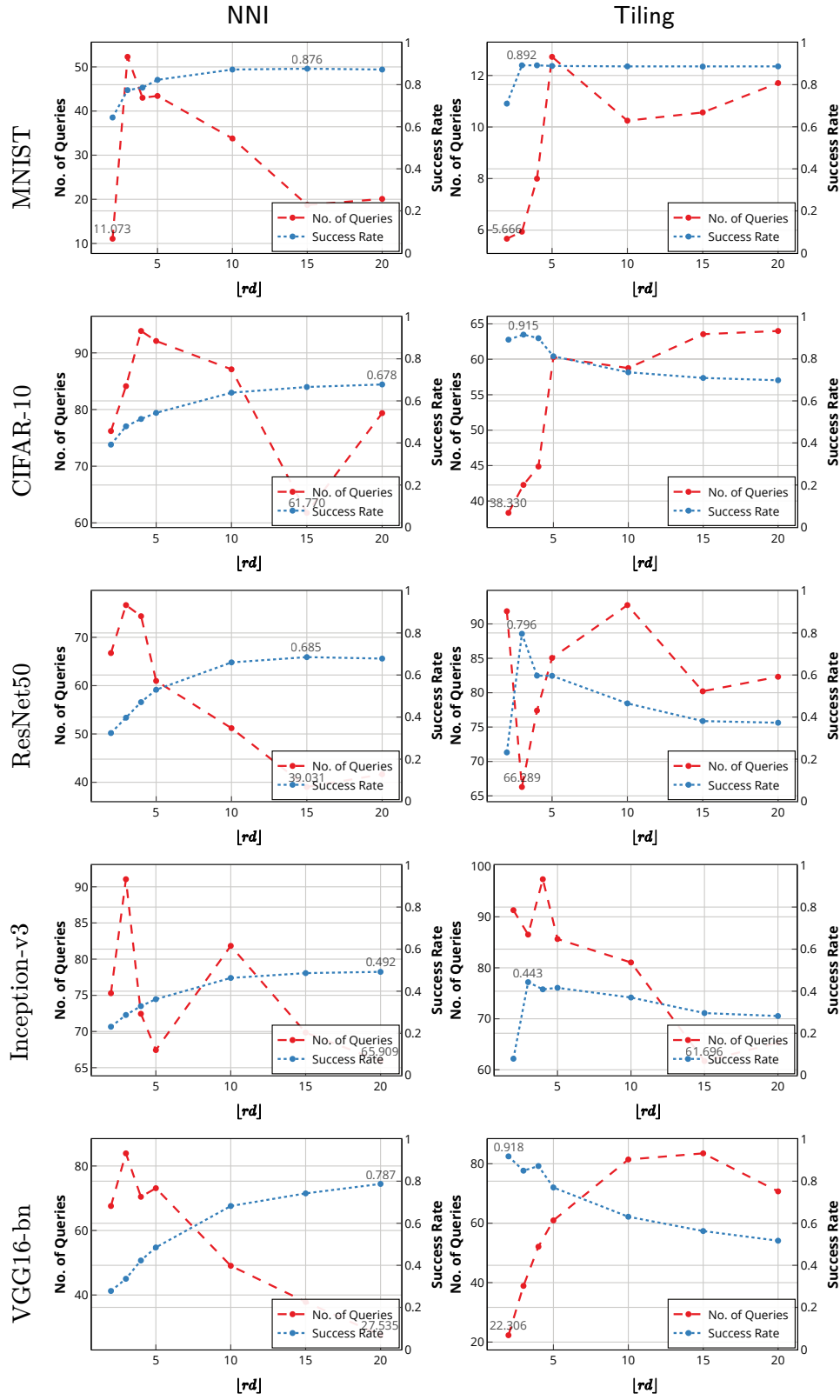


Figure C.5. Untargeted ℓ_∞ Rng-Adv Attack, comparing NNI and Tiling.

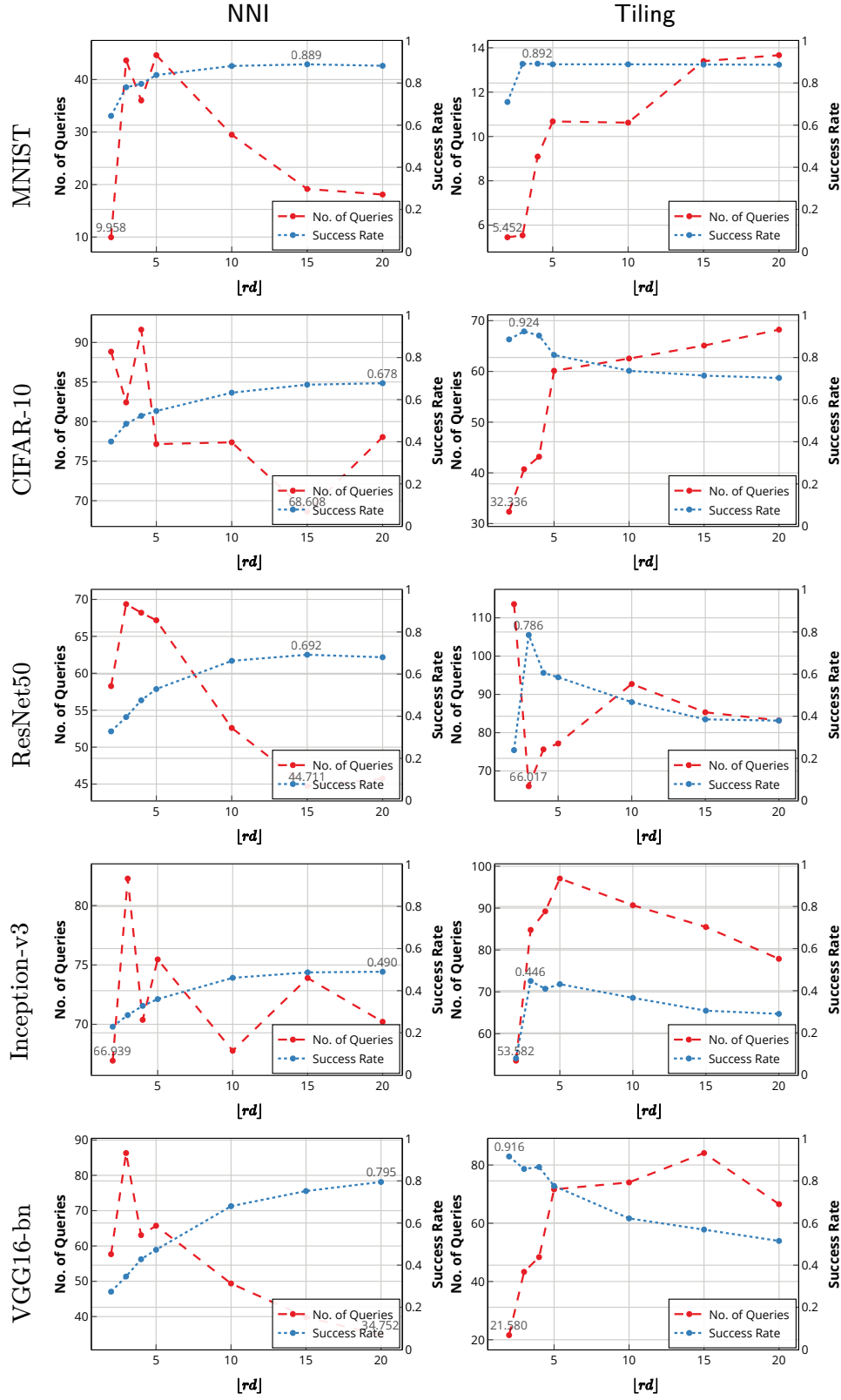


Figure C.6. Untargeted ℓ_∞ BO-Adv Attack, comparing NNI and Tiling.

C.5 Untargeted ℓ_2 Attack Experiments

Figures C.7, C.8 illustrate FSFT’s higher success rates, fewer query requirements, and operation at a low latent dimensionality of $\lfloor rd \rfloor = 1$. The resulting smaller latent perturbation size enhances scaling to larger images and boosts BO efficacy in low-dimensional spaces. It’s important to note that the latent perturbation size is determined by the chosen attack frequency, especially since some frequencies only have real components.

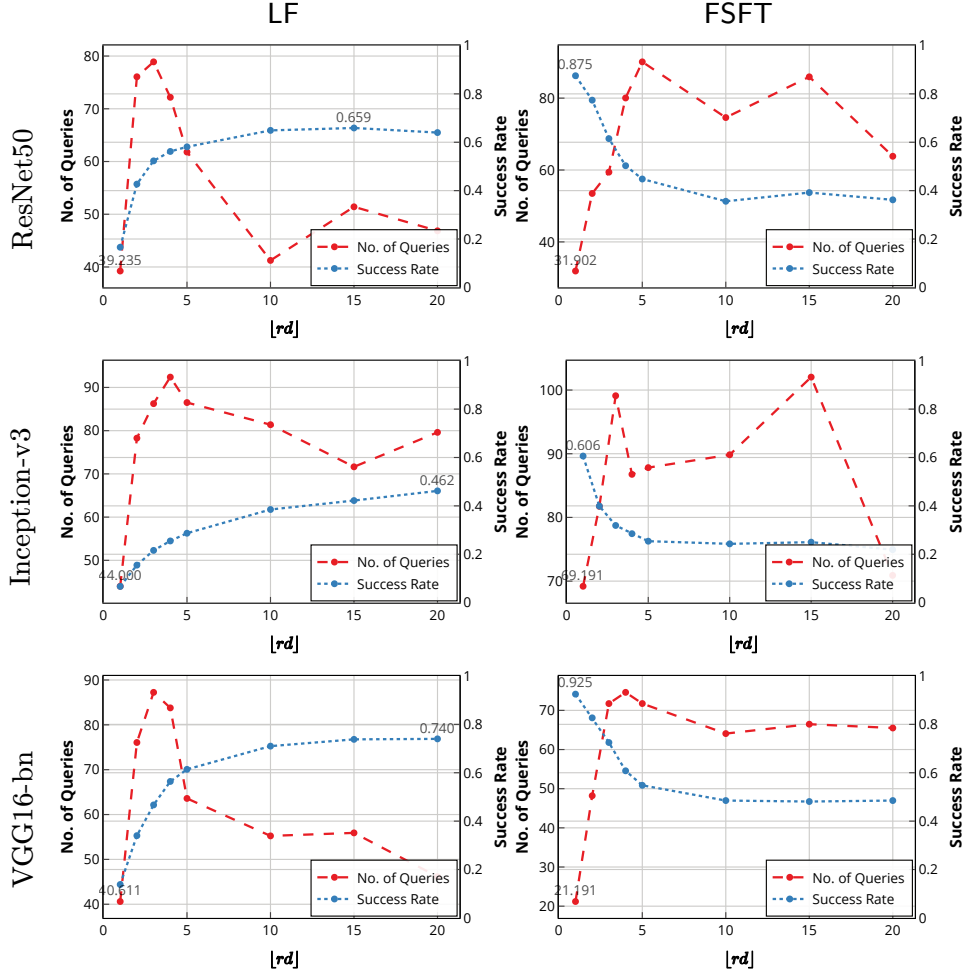


Figure C.7. Untargeted ℓ_2 Rng-Adv Attack on ImageNet CNN, comparing LF and FSFT.

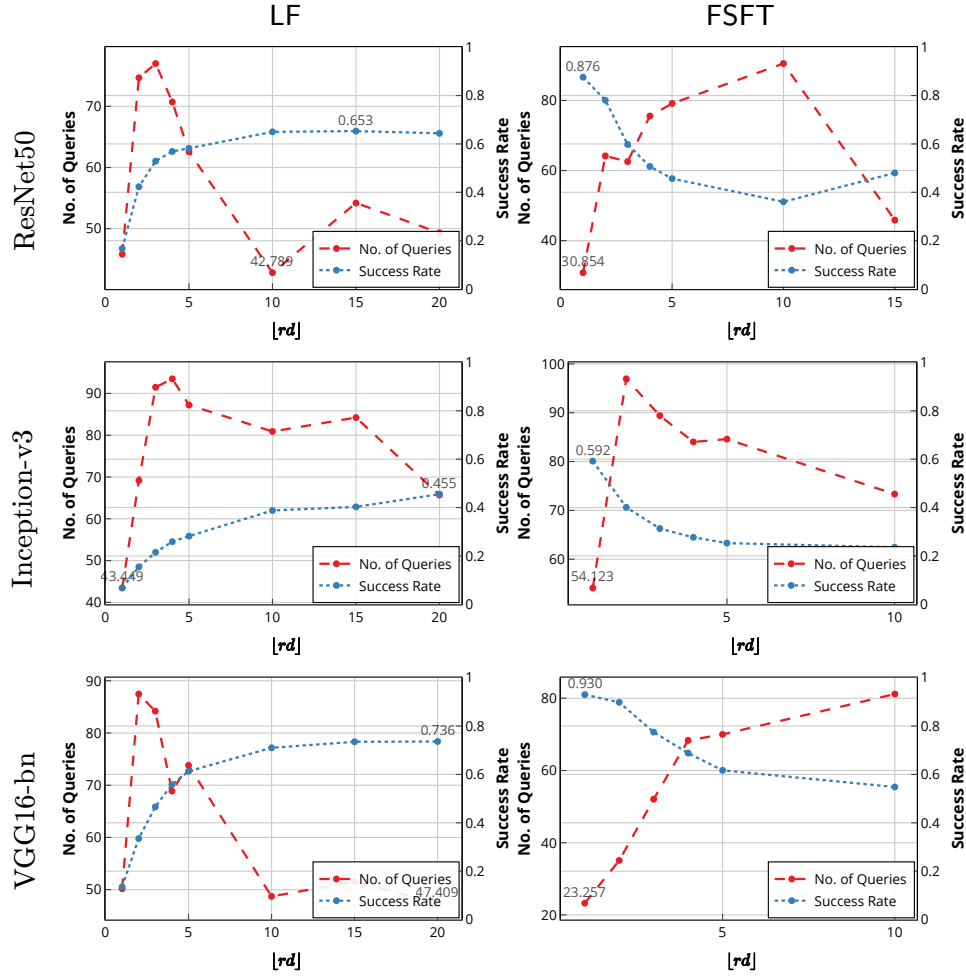


Figure C.8. Untargeted ℓ_2 BO-Adv Attack on ImageNet CNN, comparing LF and FSFT.